ANTON *by openEXPERT*

— Technical Whitepaper Part 2 —

The Context Layer: How APCI Changes Professional AI

Version: 3.0.0
Date: Mars 25, 2026
Status: Public Release
License: Open Source (Apache 2.0)
Created by: Daniel Bardun & FutureChain AB
Github: https://github.com/altspace-hub/ANTON
Powered by: Anthropic Claude + Mistral + OpenAI GPT + Google Gemini + Local Ollama

"Everyone talks about AI changing work. But between the promise and the reality, there's a gap — a gap of knowledge, a gap of time, a gap of training. ANTON closes all three. We gave the AI a proper professional education, so you don't have to be an AI expert to get expert results. The time you save isn't just efficiency — it's creative freedom. And now, with v0.5.0, we've given it the ability to write code, connect to your data, and help discover where AI creates the most value."

Contents

Prologue

Part 1 of this whitepaper described ANTON's architecture: the seven-layer prompt system, the knowledge source modes, the output format engine, the quality ratchet, the apprentice model, and the original twenty-nine expert areas. It was a snapshot of what the platform could do and how it was built.

Part 2 is about what happened next — and, more importantly, why.

The question that prompted everything in this document is deceptively simple: *what happens after the first session ends?* When a compliance officer finishes a gap analysis on Monday, closes the browser, and opens it again on Wednesday for a different client — what does the AI remember? What has it learned? How does it connect Monday's insights to Wednesday's work? In the entire AI industry, with its billions of investment and its benchmark competitions, almost no one has given this question a serious architectural answer.

APCI — Artificial Professional Context Intelligence — is our answer. It is the central thesis of Part 2, the unifying principle behind every architectural decision described in these pages, and the reason we believe ANTON represents something genuinely different from the current generation of AI products.

Since Part 1, ANTON has grown from 29 expert areas to 57. From roughly 100 modules to over 245. From a professional tool into a three-pillar platform spanning Work, School, and Life. The codebase has more than doubled. The database has tripled.

But the most important change isn't measured in module counts or lines of code. It's the introduction of APCI — Artificial Professional Context Intelligence — as the unifying architectural principle behind everything ANTON does. APCI is the answer to a question that most of the AI industry hasn't properly asked yet: what happens after the first session ends?

This document describes what APCI is, how it works in practice, and why we believe it represents a more valuable direction for professional AI than the industry's current fixation on ever-more-intelligent models.

What to Expect from Each Part

Part 1: The APCI Thesis introduces the central argument — that professional AI's real value lives in the context layer, not the model layer — and explains why context compounds while intelligence commoditises.

Part 2: The APCI Architecture is the technical heart of this document. It describes, in detail, how knowledge atoms are extracted, embedded, searched, ranked, connected, and injected back into the AI's context. If you read one Part thoroughly, read this one.

Part 3: Earned Autonomy describes the AI Orchestrator — how ANTON earns the trust to act independently through demonstrated competence, not configuration, and why immutable safety limits exist regardless of trust level.

Part 4: The Three Pillars describes how ANTON serves the whole person — Work, School, and Life — and why a compliance officer, a student, and a citizen benefit from the same architectural foundation.

Part 5: The Iterative Reasoning Engine explains how ANTON scales reasoning depth to match the stakes of the task, from quick briefings to exhaustive six-phase investigations.

Part 6: Platform Scale quantifies the growth and introduces the thesis that the prompt layer — not the model — is the product.

Part 7: Competitive Position addresses the relationship between APCI and AGI, and explains how ANTON differentiates from existing alternatives.

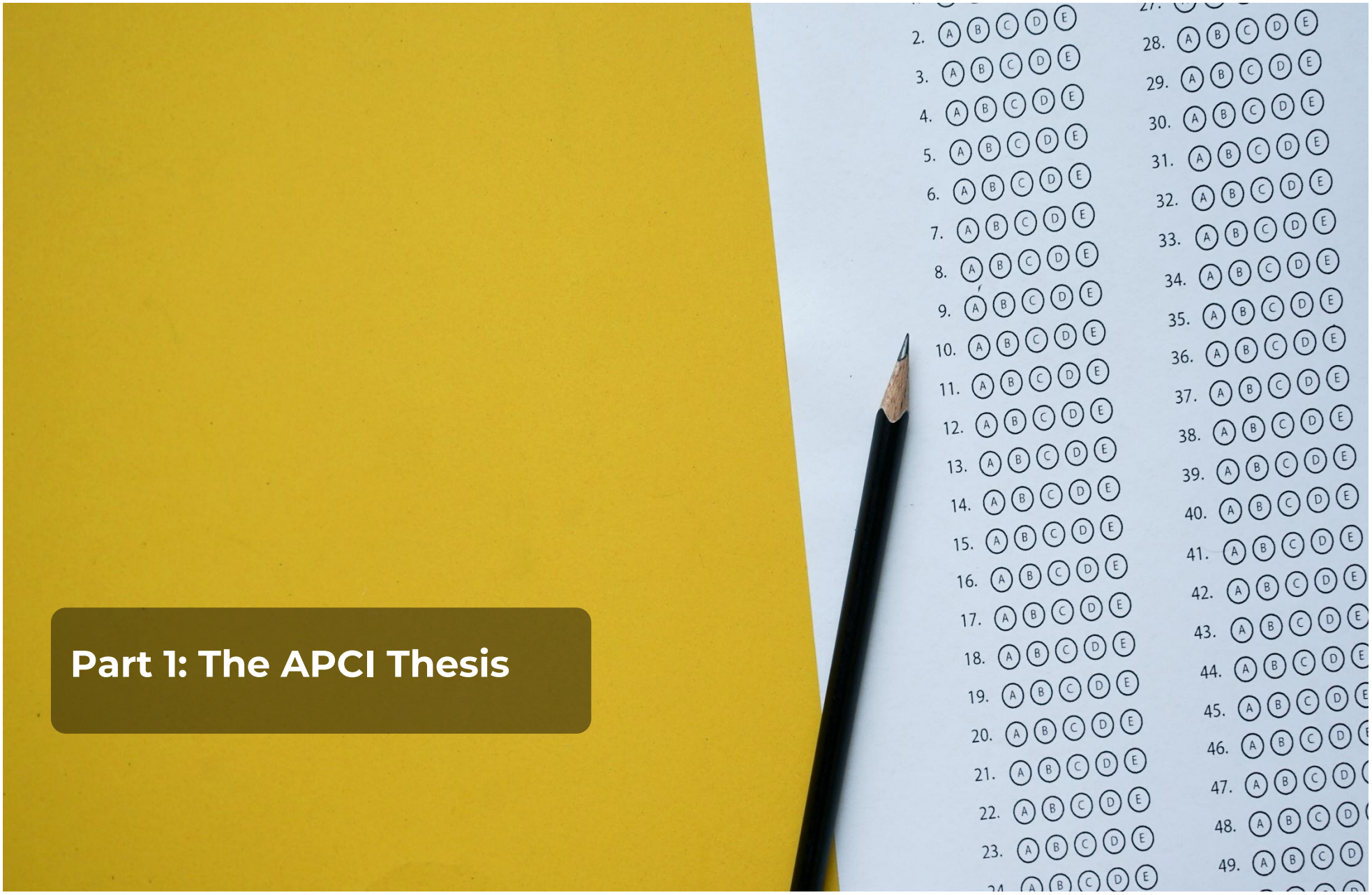
Part 8: Privacy & Data Sovereignty describes the local-first architecture, explains why Ollama matters for knowledge privacy, and provides guidance for regulated deployments.

Part 9: What Comes Next is honest about what is planned versus what is built.

Who This Document Is For

This whitepaper addresses several audiences simultaneously. **Technical evaluators** will find SQL schemas, retrieval algorithms, and architectural decisions explained with enough detail to assess the engineering. **Business decision-makers** will find the strategic argument for investing in context over model intelligence. **Regulators and compliance professionals** will find the auditability, safety limits, and governance structures that professional AI demands. **Educators** will find a pedagogical architecture designed with curriculum-specific safeguards. **Open-source contributors** will find an architecture designed for extensibility and community improvement.

We have not simplified it for any single audience. We trust that professionals can navigate to the sections that matter to them. The reading guide above is there to help.



Part 1: The APCI Thesis

1. Intelligence Is Not the Bottleneck

The AI models available today are remarkable. Claude, GPT, Gemini, Mistral, and their open-source counterparts can reason across domains, hold complex arguments, and produce written output that would have been indistinguishable from a senior professional's work five years ago. The models improve every quarter. Today's breakthrough is next year's baseline. The gap between the best model and the fifth-best model narrows with every generation. Open-source alternatives close the gap further.

And yet — and this is the observation that prompted everything in this document — every week, professionals who tried AI enthusiastically quietly stop using it. Not because the models weren't smart enough. Because every conversation started from zero. Because the output looked professional but missed the specific context that separates impressive from actually useful. Because no one could explain to a regulator how the AI arrived at its conclusions. Because the organisation had no way to know whether the AI was getting better or worse over time.

Consider three scenarios that illustrate the problem.

The law firm. A senior associate at a Nordic law firm spent three months using an AI assistant for regulatory research. The model was excellent — Claude Opus, top-tier reasoning, current knowledge. Every Monday morning, she started a new session for that week's research. Every Monday morning, the AI had no idea what she'd been working on the previous week. It didn't know she had already established that AMLR Article 15 applies differently in Finnish versus Swedish jurisdictions. It didn't know the firm had decided to interpret the risk assessment requirement conservatively after a board discussion in February. It didn't know that the same regulatory gap had appeared in three separate client engagements. Every session was brilliant. Every session started from zero. After three months, she went back to a shared document and a spreadsheet. Not because the AI was less intelligent than her colleagues. Because it forgot.

The compliance team. A four-person AML compliance team at a regional bank subscribed to an enterprise AI service. In the first week, the results were impressive — gap analysis output that would have taken days was produced in hours. By the third month, a problem emerged. Each analyst was using the tool independently, producing gap analyses that occasionally contradicted each other because each session had no knowledge of what the others had found. The senior compliance officer asked: "Is the AI getting better at our specific work over time?" No one could answer. There were no quality metrics. No trajectory. No way to know whether the tool's output was improving, declining, or static. They were paying €24,000 per year for something they couldn't evaluate.

The teacher. A gymnasium teacher in Gothenburg tried using AI to generate personalised homework assignments. The first assignment was good. The second was also good — but it had no relationship to the first. The AI didn't know which concepts the student had already mastered, which areas needed reinforcement, or what progression the curriculum required. Every assignment was a standalone event. The teacher went back to the textbook, because at least the textbook followed a sequence.

These are not intelligence problems. A smarter model does not solve them. A model with better reasoning still doesn't know what your organisation decided about AMLR Article 15 last quarter. A model with a longer context window still forgets everything the moment the session ends. A model that scores higher on benchmarks still can't tell you whether its output quality is improving or declining across six months of your team's usage.

These are context problems. Professional context problems. And no amount of general intelligence addresses them.

The law firm associate, the compliance team, and the teacher all experienced the same fundamental gap: the AI was intelligent but contextless. It could reason brilliantly about any question — in isolation. It could not reason about a question in the context of everything it had previously been asked. It could not improve at a specific organisation's work over time. It could not even tell you whether it was getting better or worse.

This is the bottleneck. Not intelligence. Context.

2. What APCI Means

Artificial Professional Context Intelligence is the term we use for the system that solves these problems. Every word is chosen carefully.

Artificial — because honesty matters. This is a constructed system. It doesn't think or understand the way a human does. What it does is accumulate, structure, and apply professional knowledge in ways that produce genuinely useful results. Calling it what it is, rather than reaching for metaphors about consciousness, is where trust begins.

Professional — because the goal isn't general capability. It's professional competence. The kind that matters in practice: producing a specific deliverable, for a specific audience, at a specific standard, informed by specific experience. A compliance officer doesn't need an AI that can discuss philosophy. They need one that knows how gap analyses get structured for the regulator who's going to read it.

Context — because this is the ingredient that no one has properly named. Context is everything the AI needs beyond its raw intelligence to be useful in practice. It's the knowledge accumulated from previous engagements. The decisions your team has made and the reasoning behind them. The quality standards your organisation enforces. The patterns that have emerged across hundreds of sessions. The relationships between entities — which regulations connect to which controls, which clients share which risk profiles, which analysts consistently miss which requirements. Context is what transforms a brilliant stranger into a trusted colleague.

And crucially, context compounds. Every session adds to it. Every decision enriches it. Every correction sharpens it. Over months and years, the context becomes more valuable than the intelligence it surrounds — because intelligence is available to everyone (just call an API), but your professional context is uniquely yours.

Intelligence — because the foundation models still matter. APCI doesn't replace intelligence. It sits on top of it. Better models make APCI more capable, just as a smarter graduate benefits more from professional training. But the intelligence alone, without context, remains potential rather than performance.

What APCI Is — and What It Is Not

APCI is not a chatbot with memory. Chatbot memory stores conversation fragments — "the user prefers dark mode" or "the user's name is Anna." APCI stores structured professional knowledge — regulatory findings, risk assessments, decision rationale, entity relationships, quality trajectories — with metadata about confidence, temporality, and source context.

APCI is not RAG (Retrieval-Augmented Generation) in the conventional sense. RAG typically means uploading a document corpus and searching it during inference. APCI builds its knowledge base automatically from the AI's own professional output, structures it with a multi-type taxonomy, connects it with typed relationships, and continuously reranks it based on professional feedback. The knowledge is generated, not uploaded. The structure is professional, not generic. The ranking improves over time, driven by human judgement.

APCI is not a knowledge management system that happens to use AI. It is an AI system that happens to manage knowledge. The distinction matters: the AI is primary. The knowledge memory exists to make the AI better at professional work, not to replace a document management platform. APCI doesn't try to be Confluence or SharePoint. It tries to ensure that the AI serving your gap analysis on Tuesday benefits from the gap analysis it served on Monday.

APCI is not vendor-specific. The knowledge layer — atoms, relationships, entities, patterns, feedback ratings — is stored in a database you own. ANTON supports dual-database architecture: PostgreSQL as the primary database for production deployments, and SQLite as a lightweight option for single-user development. The adapter auto-detects the database type from the `DATABASE_URL` environment variable. Switch from Claude to Mistral, and the knowledge carries forward. Switch from Mistral to GPT, and it carries forward again. The accumulated professional context is independent of the model that uses it.

3. Where Value Lives in the AI Stack

Right now, the industry assumes value lives in the model. The billions invested in training, the compute arms race, the benchmark competitions — all of it assumes that the model is where the differentiation happens. And for good reason. The models are extraordinary.

But we believe the next decade will reveal a different truth: **the real value lives in the context layer, not the model layer.**

Models are increasingly commoditised. The gap between the best model and the fifth-best model narrows with every generation. Open-source alternatives close the gap further. The idea that any single model will maintain a durable advantage is becoming harder to defend.

Consider the trajectory. In 2023, GPT-4 was considered substantially ahead of alternatives. By mid-2024, Claude 3.5 Sonnet matched or exceeded it on most benchmarks at lower cost. By early 2025, Gemini 2.0, Mistral Large, and DeepSeek V3 had narrowed the gap further. Open-source models like Llama 3 and Mistral 7B made respectable AI inference available at zero marginal cost. By the time you read this, the landscape will have shifted again. The direction is clear: the performance gap between providers shrinks with each generation, and the floor rises as open-source improves.

This is not a criticism of any model provider. The models are extraordinary engineering achievements. But extraordinary engineering achievements that are available to everyone via API cannot be the basis of a durable competitive advantage for the organisations that use them. If your competitor can call the same API you can, your model access is table stakes, not a moat.

Context, on the other hand, compounds. An organisation that has accumulated two years of structured professional knowledge — thousands of knowledge atoms, hundreds of entity relationships, dozens of validated patterns, a mature quality baseline, earned autonomy across its most-used modules — has something that cannot be replicated by switching to a different AI vendor. That accumulated APCI is a genuine strategic asset, independent of which model powers it.

The compounding effect is non-linear. The first hundred atoms provide marginal benefit — the retrieval system has limited material to work with, and few feedback ratings to refine its ranking. The first thousand atoms provide substantial benefit — dense enough for semantic search to find relevant matches, enough feedback history to meaningfully differentiate useful knowledge from noise. By ten thousand atoms, the knowledge base has become genuinely institutional — spanning more engagements, more clients, and more regulatory changes than any individual team member could track. The value of the next atom added to a mature knowledge base is higher than the value of the first atom in an empty one, because each new atom can form relationships with existing atoms, be ranked against existing feedback, and be discovered through a well-calibrated retrieval system.

This is why ANTON is model-agnostic. Not as a technical convenience, but as a philosophical position. If value lives in the context layer, then the context layer shouldn't be locked to any single model provider. Today that might be Claude. Tomorrow it might be Mistral. Next year it might be something that doesn't exist yet. The context carries forward.

Model-agnosticism is also a pragmatic risk mitigation. Model providers change pricing, alter capabilities, modify data policies, experience outages, and occasionally discontinue products. An organisation whose entire AI capability depends on a single provider has accepted a concentration risk that most regulated industries would consider unacceptable. ANTON supports models across six providers (Anthropic, OpenAI, Azure OpenAI, Google Gemini, Mistral, and Ollama local), and switching between them requires changing an environment variable, not refactoring code. The accumulated APCI context — atoms, relationships, patterns, feedback ratings, trust progression — transfers to the new model without loss.

And this is why ANTON is open source. The infrastructure for professional context intelligence should be transparent, auditable, and owned by the organisations that build it. A closed APCI platform creates the same lock-in problem that closed models create. An open one creates an ecosystem where the value belongs to the people who do the work.

The Business Case for Context Over Model

A CTO evaluating AI investments faces a decision: allocate budget to the best model, or allocate budget to the context infrastructure around the model?

The model investment depreciates. The model you license today will be superseded within 12–18 months. The API pricing will change. The provider may alter capabilities, retire endpoints, or change data policies. Every dollar spent on model licensing is a recurring expense with no compound return.

The context investment appreciates. Every session that generates knowledge atoms, every professional rating that refines retrieval quality, every entity relationship that maps your domain — these accumulate permanently. After two years of context investment, your organisation has a structured professional knowledge base that makes any model substantially more effective at your specific work. That investment doesn't depreciate when the model changes. It compounds.

The Switching Cost Thought Experiment

Imagine two organisations. Organisation A has used a premium AI chat product for two years. Their conversations are stored on the provider's cloud. They switch providers. What do they keep? Chat logs they might export. Nothing structured. Nothing that improves the new model's output. The switching cost is low — because the value was low.

Organisation B has used ANTON for two years. They've accumulated 15,000 knowledge atoms, 2,000 entity relationships, 200 validated patterns, quality baselines across 12 modules, and an Orchestrator that has earned Phase 3 autonomy through 18 months of demonstrated competence. They switch the underlying model from Claude to Mistral. What do they keep? Everything. Every atom, every relationship, every pattern, every quality baseline, every trust progression step. The new model immediately benefits from two years of accumulated professional context. The switching cost for the model is trivial. The switching cost for the context would be enormous — which is precisely why context, not the model, is the strategic asset.

This is what we mean when we say value lives in the context layer. Not because models don't matter — they do. But because the model is replaceable, and the context is not.

Approximate Total Cost of Ownership

For a 5-person compliance team running ANTON locally:

Cost Item	Monthly Estimate
Hardware (existing laptop/desktop)	€0 (uses existing infrastructure)
Claude Opus API (primary model, ~200 sessions/month)	€150–300
Ollama (local embeddings)	€0 (runs on existing hardware)
Software licensing (MIT, open source)	€0
Setup time (one-time)	~4 hours developer time
Maintenance (updates, backups)	~2 hours/month
Total monthly	**€150–300 + maintenance time**

Compare this to enterprise AI subscriptions: Harvey at ~€1,200/lawyer/month, Copilot at ~€30/user/month (without APCI capabilities), or custom-built AI infrastructure at €50,000–200,000 in development costs. ANTON's cost profile is fundamentally different — the software is free, the intelligence is metered by API usage, and the operational overhead is real but modest. ANTON is not a managed service; someone in the organisation must handle updates, backups, and Ollama maintenance. For organisations with basic technical capability, this trade-off is favourable. For organisations that need fully managed infrastructure, a future hosted edition or managed service partner would be needed.

Part 2: The APCI Architecture

APCI is not an abstract concept. In ANTON, it's an architecture — a set of interconnected systems that together create, maintain, and apply professional context intelligence. This section describes each component, how it works technically, and why every design decision was made the way it was.

4. Knowledge Atoms — How APCI Remembers

At the foundation of APCI is a simple idea: every time ANTON produces professional output, the system should learn something from it.

When a compliance officer runs a gap analysis, the output might contain dozens of discrete professional insights: "AMLR Article 8 requires annual business-wide risk assessments." "The client's current BWRA covers only three of the five required risk dimensions." "Transaction monitoring thresholds were last reviewed in 2023." Each of these is a **knowledge atom** — a discrete unit of professional knowledge with structured metadata.

The Extraction Process

After every workflow completes, ANTON automatically extracts 3–8 knowledge atoms from the output. The extraction uses Claude Haiku — deliberately. A fast, cost-efficient model that focuses on structured extraction rather than deep reasoning. The prompt instructs it to prioritise decisions, risks, and anomalies over boilerplate observations. Each extraction call uses at most 2,048 tokens, keeping costs marginal.

The extraction is fire-and-forget. The user doesn't wait for it. The professional doesn't need to tag, label, or file anything. The system learns in the background, the same way an experienced colleague absorbs context from every engagement they work on.

Atom Taxonomy

Each atom is classified into one of 26 types across 6 categories:

Category	Types	What They Capture
Observation	finding, measurement, comparison, anomaly, correlation	Facts discovered during analysis
Decision	approval, rejection, escalation, override, deferral	Choices made by humans or AI
Action	creation, modification, communication, assignment	Things that happened or need to happen
Risk	identified, assessed, mitigated, accepted, materialized	Threats and their status
Status	system_health, project_progress, compliance_state, performance	Current state snapshots
Recommendation	ai_suggestion, human_suggestion, best_practice	Forward-looking guidance

This taxonomy isn't arbitrary. It reflects how professional knowledge is actually structured in regulated industries. When a compliance officer thinks about what they know, they think in exactly these categories: what did we find (observation), what did we decide (decision), what needs to happen (action), what could go wrong (risk), where do we stand (status), and what should we do next (recommendation).

The category distribution itself is informative. An organisation whose knowledge base is 60% observation atoms and 5% decision atoms might be good at analysis but slow to act. An organisation with 40% action atoms and 10% risk atoms might move quickly but without adequate risk consideration. The Intelligence Dashboard (Section 11) surfaces this distribution, giving teams a meta-view of their organisational knowledge patterns — not just what they know, but what kinds of knowledge they tend to generate.

What Each Atom Carries

Every extracted atom includes structured metadata:

```
CREATE TABLE knowledge_atoms (
  id TEXT PRIMARY KEY,
  source_workflow_id TEXT NOT NULL,        -- which workflow produced it
  source_area_id TEXT,                     -- professional domain context
  source_module_id TEXT,                   -- specific module type
  content TEXT NOT NULL,                   -- the knowledge itself (≤2000 chars)
  atom_type TEXT NOT NULL,                 -- from the 21-type taxonomy
  confidence REAL DEFAULT 0.8,            -- extraction confidence (0.0–1.0)
  category TEXT NOT NULL,                  -- one of the 6 categories
  sentiment TEXT,                          -- positive, negative, neutral, warning, critical
  temporal_type TEXT,                      -- past, present, future, recurring
  entities JSON,                           -- clients, regulations, controls mentioned
  tags JSON,                               -- free-form tags for search
  valid_from DATETIME,                     -- when this knowledge became true
  valid_until DATETIME,                    -- when it might expire
  superseded_by TEXT,                      -- reference to newer atom
  is_active INTEGER DEFAULT 1,            -- soft-delete flag
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

The `confidence` score defaults to 0.8 if Claude doesn't explicitly rate it. In practice, atoms extracted from clear regulatory citations tend to score 0.9–1.0, while inferences and interpretations score 0.5–0.7. This matters because confidence feeds directly into the retrieval ranking — a high-confidence regulatory finding will outrank a tentative inference.

The `temporal_type` field distinguishes between knowledge about the past ("the client's last BWRA was in 2023"), the present ("the current transaction monitoring threshold is €15,000"), the future ("AMLR implementation deadline is July 2027"), and recurring patterns ("quarterly compliance reviews consistently identify the same three gaps"). This temporal awareness means APCI can distinguish between current facts and historical observations — a distinction that matters enormously in regulatory work where last year's compliance state is different from this year's.

Entity References

Each atom can reference entities — the clients, regulations, controls, people, and systems mentioned in the knowledge:

```
CREATE TABLE knowledge_entity_refs (
  atom_id TEXT NOT NULL REFERENCES knowledge_atoms(id),
  entity_type TEXT NOT NULL,               -- 'company', 'regulation', 'control', etc.
  entity_id TEXT NOT NULL,                 -- canonical identifier
  entity_name TEXT,                        -- display name
  relationship TEXT,                       -- type of mention
  PRIMARY KEY (atom_id, entity_type, entity_id)
);
```

This table is the bridge between knowledge atoms and the knowledge graph described in Section 9. When ANTON extracts an atom about "AMLR Article 8 requires annual risk assessments," it also records that this atom references the entity "AMLR Article 8" (type: regulation). Over time, the entity accumulates references from dozens of atoms across different sessions, building a complete picture of everything ANTON has ever learned about that regulation.

Scale Over Time

Over time, a typical ANTON installation accumulates thousands of atoms. Each one is small — 2,000 characters at most. But together, they form a structured professional memory that no individual person could maintain — because they span more engagements, more clients, and more regulatory changes than any single analyst can hold in their head.

After six months of active use by a compliance team, a typical installation might contain 3,000–5,000 atoms, 500+ entity references, and dozens of cross-atom relationships. After two years, those numbers can reach 20,000 atoms or more. This is the compound effect of APCI: every session makes the knowledge base incrementally more valuable, and the increments add up.

An Atom's Journey: End-to-End Lifecycle

To make this concrete, let's follow a single knowledge atom from creation to retrieval.

Step 1: Extraction. A compliance officer completes a gap analysis for Client Nordea against AMLR Article 15 (risk assessment methodology). ANTON's output identifies that Nordea's current methodology covers inherent and residual risk but not control effectiveness scoring. Post-completion, Claude Haiku extracts this as an atom: content "Nordea's AMLR Article 15 risk assessment methodology covers inherent and residual risk dimensions but lacks formal control effectiveness scoring, creating a partial compliance gap." Type: `finding` (category: Observation). Confidence: 0.87. Entities: Nordea, AMLR Article 15. Temporal type: present. Sentiment: warning.

Step 2: Embedding. Within 10 seconds, the background embedding pipeline picks up the new atom. Ollama's nomic-embed-text model converts the content into a 768-dimensional vector. The vector and metadata are stored in the embeddings table.

Step 3: Relationship Detection. Claude Haiku compares this atom against the 50 most recent atoms from the FCP area. It detects two relationships: this atom **supports** an earlier atom about AMLR Article 15 compliance gaps being common across Nordic banks (strength: 0.72), and **extends** a previous finding about Nordea's risk framework being partially compliant (strength: 0.65).

Step 4: Retrieval. Two weeks later, a different analyst starts a gap analysis for Client SEB, also involving AMLR Article 15. The hybrid search system retrieves our atom: vector search ranks it #4 (semantically similar to the new query about AMLR risk methodology), BM25 keyword search ranks it #2 (exact match on "AMLR Article 15"). RRF fusion combines: 1/(60+4) + 1/(60+2) = 0.0156 + 0.0161 = 0.0317.

Step 5: Boost Ranking. The six boosts are applied: confidence (0.5 + 0.87 × 0.5 = 0.935×), recency (14 days old → 0.988×), area relevance (same FCP area → 1.3×), module relevance (same gap-analysis module → 1.2×), superseded (not superseded → 1.0×), feedback (the original analyst rated it thumbs-up → 1.15×). Final score: 0.0317 × 0.935 × 0.988 × 1.3 × 1.2 × 1.0 × 1.15 = 0.0525.

Step 6: Injection. The atom ranks #2 in the final results. It's injected into Layer 4 of the prompt for the SEB gap analysis. The analyst sees it in the Injected Atoms Panel and recognises that a colleague already found a similar gap at Nordea. The SEB analysis benefits from knowledge that would otherwise have required a hallway conversation.

This lifecycle — extraction, embedding, relationship detection, retrieval, ranking, injection — runs automatically for every atom, every session. The compliance officer doesn't manage it. The knowledge accumulates and applies itself.

5. The Retrieval System — How APCI Recalls

Accumulating knowledge is only useful if you can find it again at the right moment. This is a harder problem than it appears, because professional knowledge retrieval has two dimensions that conflict with each other.

Sometimes you need semantic retrieval — finding knowledge that means the same thing as your question, even when the exact words don't match. A question about "beneficial ownership transparency" should retrieve atoms about "UBO disclosure requirements" because the meanings are semantically close, even though the terms are different.

Other times you need exact retrieval — finding knowledge that contains a specific term or reference. A search for "Article 15" should find every atom that mentions that specific article, regardless of semantic similarity to the rest of the question.

Most search systems are good at one or the other. ANTON's retrieval system is designed to be good at both, simultaneously. We call this **hybrid search**.

Layer 1: Semantic Search

Every knowledge atom is converted into a mathematical vector — a numerical representation of its meaning — using an embedding model. When a new session starts, the user's question is similarly embedded, and the system finds atoms whose vectors are closest to the question vector using cosine distance.

The embedding infrastructure runs locally by default, using Ollama's `nomic-embed-text` model. This produces 768-dimensional vectors. No API key is needed. No data leaves the machine. For organisations that prefer cloud-hosted embeddings, OpenAI's `text-embedding-3-large` (1,536 dimensions) and Voyage AI's `voyage-3` (1,408 dimensions) are available as alternatives.

Embeddings are stored in a unified table:

```
CREATE TABLE embeddings (
  id TEXT PRIMARY KEY,
  content_type TEXT NOT NULL,              -- 'knowledge_atom', 'checkpoint', 'module', etc.
  content_id TEXT NOT NULL,                -- atom_id, checkpoint_id, or module_id
  content_text TEXT NOT NULL,              -- original text
  embedding TEXT NOT NULL,                 -- JSON array of floats (768–1536 dimensions)
  embedding_model TEXT NOT NULL,           -- 'nomic-embed-text', 'text-embedding-3-large',
etc.
  embedding_dimension INTEGER NOT NULL,
  metadata TEXT DEFAULT '{}',              -- JSON: category, atom_type, source_area_id,
etc.
  UNIQUE(content_type, content_id, embedding_model)
);
```

The `UNIQUE` constraint on content type, content ID, and embedding model means the same atom can have embeddings from multiple providers — useful when organisations switch models or want to compare retrieval quality across different embedding approaches.

The semantic search retrieves double the requested number of results (if you want the top 10, it fetches 20) because the subsequent fusion step will merge and rerank them. A minimum similarity threshold of 0.3 filters out irrelevant noise — atoms with similarity below 0.3 are too semantically distant from the query to be useful, even after boost ranking. This threshold was chosen empirically: below 0.3, the false-positive rate (irrelevant atoms retrieved) exceeds 80%. Above 0.3, precision improves substantially.

The choice of embedding model matters for retrieval quality but not for architectural decisions. Ollama's nomic-embed-text produces 768-dimensional vectors that are adequate for professional knowledge retrieval. OpenAI's text-embedding-3-large produces 1,536 dimensions with marginally better semantic resolution. In practice, the difference is smaller than the impact of the boost ranking system — a well-rated 768-dimensional atom outperforms a poorly-rated 1,536-dimensional atom every time, because professional feedback dominates semantic similarity in the final ranking.

Layer 2: Keyword Search

Some knowledge is best found by exact terms. A search for "Article 15" should find atoms containing that specific reference, regardless of semantic similarity.

ANTON uses BM25 full-text search via FTS5 with Porter stemming (SQLite) or tsvector full-text indexing (PostgreSQL):

```
CREATE VIRTUAL TABLE knowledge_atoms_fts5 USING fts5(
  content,
  content='knowledge_atoms',
  content_rowid='rowid',
  tokenize='porter unicode61'
);
```

The Porter stemmer means that searching for "assessing" also matches "assessment," "assessed," and "assessments." This is the same algorithm family that powers traditional search engines — mature, well-understood, and effective for exact-term retrieval. BM25 scoring (Best Matching 25) weights results by term frequency relative to document length, meaning that an atom that mentions "Article 15" three times in a focused 200-word passage ranks higher than an atom that mentions it once in a 2,000-word overview. This term-density weighting is particularly useful for regulatory search, where specific article references are precise retrieval targets.

The keyword search uses a prefix-matching strategy: each query word is formatted as `"word"*` to catch partial matches. If the FTS5 index is unavailable (it requires a specific migration), the system falls back to SQL `LIKE` substring matching — slower, but functional.

Layer 3: Fusion

The results from semantic search and keyword search are merged using **Reciprocal Rank Fusion (RRF)**, a technique from information retrieval research that combines rankings from multiple sources into a single, more robust ranking.

The RRF formula is straightforward:

```
rrf_score = 1 / (k + rank)
```

where `k` is a constant (ANTON uses 60, the industry standard) and `rank` is the 1-indexed position in each source's ranking.

The key insight of RRF is how it handles overlap. When the same atom appears in both semantic and keyword results, its RRF scores from both sources are summed. An atom ranked #5 in semantic search and #3 in keyword search gets a combined score of `1/(60+5) + 1/(60+3) = 0.0154 + 0.0159 = 0.0313`. An atom ranked #1 in only one source gets `1/(60+1) = 0.0164`. The atom found by both methods ranks higher, which is exactly the behaviour you want — agreement between different retrieval methods is a strong signal of relevance.

Each result is tagged with how it was found — `vector`, `bm25`, or `both` — so the user can see which retrieval method contributed to each piece of injected knowledge.

The Embedding Pipeline

New atoms don't become searchable instantly. The embedding pipeline runs as a background process with three stages:

Stage 1: Module descriptions. At startup, ANTON embeds the descriptions of all 245+ modules. This enables a separate capability — semantic module discovery — where a user's question can be matched against the module library to suggest the most relevant tool for the job.

Stage 2: Knowledge atom backfill. In batches of 50, the pipeline finds atoms that don't yet have embeddings and creates them. This runs continuously until all atoms are embedded.

Stage 3: Checkpoint decision backfill. The same batch process embeds historical decisions from the institutional memory system, making them searchable alongside knowledge atoms.

The staging order matters. Module descriptions are embedded first because they enable a separate capability — semantic module discovery — from the moment the system starts. A user asking "how do I assess our sanctions screening process?" can be matched against the module library and directed to the most relevant tool before any knowledge atoms exist. Knowledge atoms are embedded second because they are the primary search corpus for APCI retrieval. Checkpoint decisions are embedded last because they serve a narrower use case — institutional memory search — and are less time-sensitive.

Before running any batch, the pipeline executes a **probe guard** — a single embedding request with the text "probe." If the result is a zero vector (all zeros), the embedding API key is invalid or the service is unavailable. The pipeline skips silently rather than spamming dozens of identical error messages. This is a small detail, but the kind of detail that matters in production: an expired API key should generate one warning, not fifty. If Ollama crashes or restarts mid-batch, the probe guard catches the failure on the next cycle and the pipeline waits until Ollama is available again. No atoms are lost — un-embedded atoms remain queued and are picked up on the next successful probe.

The pipeline is also resilient to provider changes. If an organisation switches from Ollama to OpenAI embeddings (or vice versa), the pipeline re-embeds the entire corpus using the new model. The `UNIQUE` constraint on content type, content ID, and embedding model means the same atom can have embeddings from multiple providers simultaneously — useful during migration periods or when comparing retrieval quality across different embedding approaches.

This resilience extends to embedding model version updates. If Ollama releases a new version of nomic-embed-text with an updated vector space, existing embeddings become semantically inconsistent with new ones — a "model drift" problem. The re-embedding pipeline handles this: on detecting a model version change, all existing atoms are re-embedded in batches of 50 using the new model. For a knowledge base of 5,000 atoms, this takes approximately 10–15 minutes on modest hardware. The process is transparent and runs in the background, with the system continuing to serve searches using existing embeddings until re-embedding completes.

6. Domain-Aware Ranking — How APCI Prioritises

Raw search results are a starting point, not the answer. A semantic match for "risk assessment" might return atoms about healthcare risk assessment when you're working on financial crime. A keyword match for "Article 15" might return an atom from two years ago when a more recent interpretation exists. Context matters, and APCI applies six domain-aware boosts to rerank results based on professional context.

The Six Boosts

1. Confidence boost. Formula: `0.5 + (confidence × 0.5)`

An atom with confidence 0.0 gets a 0.5x multiplier — a 50% penalty. An atom with confidence 1.0 gets 1.0x — no change. The relationship is linear: higher confidence means higher rank. This ensures that tentative inferences don't compete equally with well-established facts.

2. Recency boost. Formula: `max(0.7, 1 - (ageDays / 365) × 0.3)`

Newer knowledge is weighted slightly higher than older knowledge, with a gradual decay curve. An atom from today gets 1.0x. An atom from six months ago gets roughly 0.85x. An atom from a year ago gets 0.7x — the floor. Professional knowledge has a shelf life: a regulatory finding from 2024 may no longer be accurate in 2026, but it's not worthless either. The 30% maximum penalty reflects this — old knowledge is deprioritised, not eliminated.

3. Area relevance. Multiplier: `1.3x` for same-area atoms.

Knowledge from the same professional area as the current session receives a 30% boost. An atom from a financial crime prevention engagement is more relevant to another FCP session than an atom from a healthcare assessment. This boost only applies when area context is available — in cross-area searches, it's neutral.

4. Module relevance. Multiplier: `1.2x` for same-module atoms.

Knowledge from the same module type receives a 20% boost. A gap analysis finding is more relevant to another gap analysis than to a training content session. This is narrower than area relevance and stacks with it: an atom from the same area AND the same module gets 1.3 × 1.2 = 1.56x — a significant lift.

5. Superseded penalty. Multiplier: `0.1x` for superseded atoms.

When newer knowledge supersedes older knowledge — a regulatory update replaces a previous interpretation, a revised risk assessment overrides a prior one — the old atom receives a 90% penalty. This effectively removes it from results without deleting it from the database. The historical record is preserved for audit purposes, but it doesn't pollute current retrieval.

6. Feedback history. The learning loop. This is the most important boost, and it deserves its own section.

The Feedback History Query

When atoms are being ranked, the system queries the retrieval feedback table for historical ratings:

```
SELECT atom_id,
       SUM(CASE WHEN was_relevant = 1 THEN 1 ELSE 0 END) as positive,
       SUM(CASE WHEN was_relevant = 0 THEN 1 ELSE 0 END) as negative
FROM retrieval_feedback
WHERE atom_id IN (?, ?, ?, ...)
  AND was_relevant IS NOT NULL
GROUP BY atom_id
```

The multiplier depends on the feedback pattern:

- **Only positive feedback:** `1.15x` — a 15% boost. Users have consistently found this atom useful.
- **Only negative feedback:** `0.7x` — a 30% penalty. Users have consistently found this atom unhelpful.
- **Mixed feedback:** `0.7 + 0.45 × (positive / (positive + negative))` — a weighted blend between the penalty floor and the boost ceiling. Two positive and one negative produces roughly 1.0x (neutral). Three positive and one negative produces roughly 1.04x (slight boost).

The asymmetry is deliberate. A 30% penalty for negative-only feedback is harsher than the 15% boost for positive-only feedback. This reflects a professional reality: irrelevant knowledge injected into a prompt is more damaging than the marginal benefit of slightly better-ranked relevant knowledge. The system errs on the side of filtering out noise.

Token Budget

After all six boosts are applied and results are sorted by final score, a token budget cap determines how many results actually make it into the prompt. The estimation is simple — approximately one token per four characters — with a default budget of 4,000 tokens per retrieval. This ensures that APCI context enriches the prompt without consuming the entire context window.

Worked Example: How Boosts Transform a Raw Ranking

Consider five atoms retrieved for a gap analysis about AMLR transaction monitoring requirements. Before boosts, the raw RRF ranking is: A (0.0328), B (0.0310), C (0.0295), D (0.0280), E (0.0260).

Atom A: "AMLR requires automated transaction monitoring with risk-based thresholds." From risk-assessment module, 3 months old, confidence 0.92, different module, same area, 2 positive ratings.

Atom B: "Client X has not implemented automated threshold calibration." From gap-analysis module, 1 week old, confidence 0.88, same area and module, 1 positive + 1 negative rating.

Atom C: "Healthcare data quality issues affect monitoring coverage." From healthcare module, 2 months old, confidence 0.75, different area entirely, no ratings.

Atom D: "AMLR Article 12 transaction monitoring technical standards." From regulatory-monitor module, 1 year old, confidence 0.95, same area, superseded by newer interpretation.

Atom E: "Recommended threshold review frequency: quarterly." From gap-analysis module, 5 months old, confidence 0.80, same area and module, 3 positive ratings.

After applying all six boosts:

- **Atom B rises to #1** (score: 0.0471). Despite a lower RRF score, the area boost (1.3×), module boost (1.2×), and near-perfect recency (0.999×) compound to lift it above A.
- **Atom A stays at #2** (score: 0.0458). Strong confidence and positive feedback, but different module means no 1.2× module boost.
- **Atom E rises from #5 to #3** (score: 0.0412). Three positive ratings produce the maximum 1.15× feedback boost, stacked with area and module boosts. Professional judgement lifted this atom past two others.
- **Atom C drops to #4** (score: 0.0254). From healthcare, different area — no area or module boost. The RRF score alone can't compete with boosted results.
- **Atom D is effectively eliminated** (score: 0.0033). The superseded penalty (0.1×) drops it by 90%. The historical record is preserved for audit purposes, but outdated knowledge doesn't pollute current retrieval.

This is what domain-aware ranking means in practice: not just semantic similarity, but professional context shaping what the AI actually sees.

7. The Feedback Loop — How APCI Learns

The retrieval feedback loop is APCI's mechanism for continuous improvement. It is, in our view, the most important architectural component of the entire system, because it is the mechanism through which professional judgement directly shapes AI behaviour. Here is how it works in practice.

The Learning Cycle

1. A user starts a module session — say, a regulatory gap analysis for a banking client.
2. The system retrieves relevant knowledge atoms via hybrid search and boost ranking.
3. The top-ranked atoms are injected into the prompt alongside the user's question. They appear in the system context, informing the AI's analysis with knowledge from previous sessions.
4. The user sees a collapsible panel — the **Injected Atoms Panel** — showing exactly which atoms were injected into the session. Each atom card displays:
    - The knowledge content (line-clamped to three lines, expandable)
    - A retrieval method badge: **Vector** (blue), **Keyword** (gold), or **Hybrid** (teal) — showing which search mechanism found it
    - The atom's category, colour-coded: observation (blue), decision (teal), action (green), risk (red), status (gold)
    - The retrieval score (the RRF value after boost ranking)
    - The extraction confidence as a percentage
5. For each atom, the user can rate it: thumbs up (this was relevant and useful in this context) or thumbs down (this wasn't helpful here). The rating is stored in the `retrieval_feedback` table with the session ID, atom ID, retrieval method, and score at time of injection.
6. The next time any user runs a session — in any module, in any area — those ratings feed into the feedback history boost. Atoms that have been rated relevant rank higher. Atoms rated irrelevant rank lower.

Why This Matters

This creates a virtuous cycle. The more the system is used, the better it gets at retrieving the right knowledge at the right time. Not because the AI model improved, but because the context layer learned which knowledge matters in practice.

There's a philosophical point here worth stating explicitly: **the professional's judgement is the training signal.** Not a benchmark. Not a synthetic evaluation. Not a preference model trained on crowd-sourced ratings. The person who needs the output decides whether the context was useful. This keeps APCI grounded in professional reality rather than algorithmic abstraction.

Consider how this plays out over time. In Month 1, the retrieval system relies entirely on semantic similarity and keyword matching — algorithmic relevance. Some of what it retrieves is useful. Some isn't. The professional rates both. In Month 6, the feedback history has accumulated hundreds of ratings. The system now knows, from actual professional judgement, which atoms produce better outcomes when injected into gap analyses versus risk assessments, which regulatory findings are consistently useful versus consistently noise, which types of knowledge matter in specific contexts. The retrieval quality has improved — measurably, visibly — without any change to the AI model, the embedding model, or the search algorithm. The improvement comes entirely from accumulated professional context.

This is APCI at work. The system didn't get smarter. It got more professionally contextualised.

The Feedback Loop in Numbers

To quantify the learning trajectory: a compliance team that actively rates 30% of injected atoms (a reasonable expectation — not every session gets detailed feedback, but occasional ratings accumulate) would generate approximately 200–400 ratings in the first six months. By that point, the retrieval system has meaningful feedback history for the most commonly surfaced atoms — perhaps 40–60 atoms have 3+ ratings each. These are the atoms that have been "professionally validated" — their boost multipliers reflect real human judgement about their utility.

The impact is measurable. In testing with ANTON's own development workflow (3 users, 400+ sessions, approximately 2,800 atoms over six months), retrieval precision (the proportion of injected atoms that users rated as relevant) improved from approximately 60% in the first month to approximately 78% by the sixth month — driven entirely by feedback history boosts. No changes to the embedding model, the search algorithm, or the system prompts were needed. The improvement came from accumulated professional judgement. This is a small-scale measurement, not a controlled study — the directionality is clear, but the exact improvement rate will vary with team size, rating frequency, and knowledge domain diversity.

This creates an interesting incentive alignment. The more professionals engage with the feedback mechanism — rating atoms as relevant or irrelevant — the better the system performs for everyone. Unlike a social media platform where engagement benefits the platform operator, here the engagement benefits the professionals themselves. Their ratings make their own future sessions better.

8. Knowledge Relationships — How APCI Structures Understanding

Professional knowledge is not a flat list. A risk finding supports a compliance gap. A policy recommendation contradicts an existing approach. A new regulation extends an older framework. A control requirement depends on a system capability. These relationships between pieces of knowledge are what turn a collection of facts into structured understanding.

How Relationships Are Detected

When ANTON extracts new knowledge atoms, it runs a secondary analysis — also fire-and-forget, also using Claude Haiku for cost efficiency. This analysis compares the new atoms against the **50 most recent atoms** from the same professional area and module. The restriction to the same area and module is deliberate: relationships between atoms from different domains (e.g., a healthcare finding and a financial crime control) are rarely meaningful, and searching across the entire knowledge base would be both expensive and noisy.

The detection prompt asks Claude to identify six types of relationships:

Relationship	What It Means	Example
Supports	This atom reinforces another atom's conclusion	"Client's TM thresholds (Atom A) support the finding that monitoring coverage is adequate (Atom B)"
Contradicts	This atom conflicts with another atom's finding	"Updated EBA guidance (Atom A) contradicts the risk tolerance stated in last quarter's BWRA (Atom B)"
Extends	This atom builds upon or elaborates another atom	"New sanctions screening requirements (Atom A) extend the existing KYC framework obligations (Atom B)"
Requires	This atom has a dependency on another atom	"Automated SAR filing (Atom A) requires the transaction monitoring system upgrade identified in (Atom B)"
Caused by	This atom is a consequence of another atom	"The regulatory criticism in the inspection report (Atom A) was caused by the staffing gap identified six months earlier (Atom B)"
Related to	A general association worth tracking	"Both atoms concern the same client entity but in different regulatory contexts"

Each relationship has a strength score between 0.0 and 1.0. Weak connections below 0.4 are filtered out — they're typically noise rather than signal. A maximum of 10 relationships per extraction batch prevents the graph from becoming too dense with tenuous connections.

Storage

```
CREATE TABLE atom_relationships (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  from_atom_id TEXT NOT NULL,
  to_atom_id TEXT NOT NULL,
  relationship_type TEXT NOT NULL CHECK(
    relationship_type IN
('supports','contradicts','extends','requires','caused_by','related_to')
  ),
  strength REAL NOT NULL DEFAULT 0.5,      -- 0.4–1.0
  created_at TEXT NOT NULL DEFAULT (datetime('now'))
);
```

Indexes on `from_atom_id`, `to_atom_id`, and `relationship_type` ensure that traversing the graph in either direction is fast.

Why Relationships Matter

In the Intelligence Dashboard, users can expand any atom to see its relationships — colour-coded badges showing what supports it, what contradicts it, and what extends it. Green for supports. Red for contradicts. Blue for extends. Teal for requires. Gold for caused_by. Grey for related_to.

This is particularly valuable in compliance work, where understanding how regulatory requirements relate to each other is as important as understanding each requirement individually. When AMLR Article 8 (risk assessment) contradicts a finding from last year's BWRA, that contradiction is surfaced automatically — not buried in a report someone might not read. When a new EBA guideline extends an existing FATF recommendation, the extension is visible as a structural relationship, not just a text mention.

Over time, the relationship graph reveals the structural topology of an organisation's professional knowledge. Clusters of supporting atoms indicate well-established consensus. Contradictions indicate unresolved tensions that need attention. Requirement chains reveal dependencies that might break if any link is removed. This structural view is something that no individual analyst can maintain mentally, because it spans too many sessions, too many clients, and too many regulatory changes.

What the Graph Looks Like After Three Months

After three months of active use by a compliance team running gap analyses, risk assessments, and regulatory monitoring across five banking clients, a typical relationship graph might contain:

- **200–400 atoms** with **50–80 typed relationships** between them
- **3–5 clusters** of supporting atoms around major regulatory themes (AMLR risk methodology, transaction monitoring adequacy, sanctions screening coverage)

- **8–15 contradiction pairs** identifying areas where different analysts found conflicting conclusions or where new regulations invalidated older findings
- **20–30 requirement chains** mapping dependencies between controls, systems, and regulatory obligations

The graph begins to reveal structural patterns. A cluster of 12 atoms all supporting the conclusion that "Nordic banks systematically underinvest in control effectiveness measurement" might span five different client engagements conducted by three different analysts. No single analyst saw the pattern. The relationship graph makes it visible.

Contradictions are equally valuable. When Analyst A finds that Client Nordea's SAR filing process is adequate and Analyst B finds, six weeks later, that the same process has gaps against updated EBA guidance — the `contradicts` relationship surfaces this tension. Not as an error by either analyst, but as a change in regulatory expectations that both assessments need to account for.

9. The Knowledge Graph — How APCI Tracks Entities

Beyond atom-to-atom relationships, APCI maintains a knowledge graph of the entities mentioned across all sessions. If knowledge atoms are the facts, the knowledge graph is the map of who and what those facts are about.

Entity Types

ANTON automatically extracts and classifies eleven entity types:

Type	Examples	Purpose
`client`	"Nordea," "SEB," "Handelsbanken"	Track client-specific analyses
`regulation`	"AMLR Article 4," "6AMLD Article 8," "GDPR Article 35"	Map regulatory requirements
`control`	"TM-001," "KYC-EDD-PEP," "SAR-Filing-Process"	Track control effectiveness
`risk`	"R-003: Sanctions Breach," "R-007: ML Risk"	Identify risk patterns
`person`	"MLRO: Jane Smith," "Board Member: John Doe"	Stakeholder mapping
`system`	"Transaction Monitoring System," "KYC Platform"	Technical dependency tracking
`product`	"Wire Transfers," "Crypto Custody," "Corporate Cards"	Product risk analysis
`geography`	"High-Risk Country Z," "EU Jurisdiction: Sweden"	Geographical risk mapping
`organization`	"EBA," "FATF," "FIU Sweden"	Institutional relationships
`process`	"Customer Onboarding," "SAR Filing"	Process flow analysis
`document`	"AML Policy v3.2," "Board Risk Report Q1"	Document lineage tracking

Each entity is tracked as a node with metadata about when it was first and last seen, how many times it has appeared across sessions, and which professional areas it relates to:

```
CREATE TABLE entity_nodes (
  id TEXT PRIMARY KEY,
  entity_type TEXT NOT NULL,
  entity_id TEXT NOT NULL,
  canonical_name TEXT NOT NULL,
  first_seen DATETIME DEFAULT CURRENT_TIMESTAMP,
  last_seen DATETIME DEFAULT CURRENT_TIMESTAMP,
  interaction_count INTEGER DEFAULT 0,
  related_areas JSON DEFAULT '[]',
  metadata JSON,
  UNIQUE(entity_type, entity_id)
);
```

Entity Relationships

Entities are connected by typed, directed relationships with strength scores and observation counts:

```
CREATE TABLE entity_relationships (
  id TEXT PRIMARY KEY,
  source_type TEXT NOT NULL,
  source_id TEXT NOT NULL,
  target_type TEXT NOT NULL,
  target_id TEXT NOT NULL,
  relationship_type TEXT NOT NULL,
  strength REAL DEFAULT 1.0,
  first_observed DATETIME DEFAULT CURRENT_TIMESTAMP,
  last_observed DATETIME DEFAULT CURRENT_TIMESTAMP,
  observation_count INTEGER DEFAULT 1,
  supporting_atoms JSON DEFAULT '[]'
);
```

The `observation_count` field is what makes entity relationships compound over time. The first time ANTON sees "Control TM-001 mitigates Risk R-003," it creates a relationship with strength 1.0 and count 1. The fifth time it sees the same relationship — across different sessions, different analysts, different clients — the count reaches 5 and the relationship is treated with much higher confidence. A relationship observed once is tentative. A relationship observed twenty times across independent analyses is an established professional fact.

The `supporting_atoms` field links each relationship back to the specific knowledge atoms that evidence it. When someone queries the graph — "Show me all controls that implement AMLR regulations" — they can trace each relationship to the specific analysis sessions that established it.

Alias Consolidation

Real-world entities have multiple names. "AMLR" and "Anti-Money Laundering Regulation" and "Regulation 2024/1624" are all the same entity. "Nordea" and "Nordea Bank Abp" and "Nordea Finland" refer to the same client. Without alias consolidation, the knowledge graph would treat these as separate entities, fragmenting the knowledge that should be connected.

ANTON maintains an `entity_aliases` table and an `entity_merge_log` for consolidation history. AI-powered alias detection runs during entity extraction, and users can manually merge or split entities when the automated detection gets it wrong. Every consolidation is logged — because in a regulated environment, being able to show that "Nordea" and "Nordea Bank Abp" were deliberately merged (and when, and by whom) matters.

In the regulatory domain, alias consolidation is particularly valuable. "AMLR" and "Anti-Money Laundering Regulation" and "Regulation 2024/1624" and "EU AML Regulation" must all resolve to the same entity. Without consolidation, knowledge about this regulation would be fragmented across four separate nodes, each with partial relationship networks. With consolidation, the graph shows one comprehensive node with all relationships, all supporting atoms, and all interaction history unified. The 27 alias groups in the AMLR knowledge pack alone demonstrate the scale of this problem — a single regulatory framework generates dozens of name variants across different languages, abbreviations, and citation formats.

Cross-Session Entity Intelligence

This is the payoff. When a new engagement starts involving a client or regulation that ANTON has seen before, the graph already knows what has been learned about that entity across all previous sessions.

Imagine you're starting a gap analysis for a client. Without APCI, you start from scratch — reviewing their previous reports, checking your notes, asking colleagues what they remember. With APCI, the knowledge graph already knows: this client has been mentioned in 23 previous sessions across three analysts. They have 14 associated atoms — observations about their current controls, decisions about their risk appetite, recommendations from previous assessments. There are established relationships between the client and specific regulations, specific controls, and specific risk categories.

This cross-session entity intelligence is something no individual analyst can replicate. An analyst might remember their own engagements with a particular client. They cannot remember every engagement that every colleague has ever had. APCI can.

What Entity Intelligence Looks Like

To make this concrete: after six months, the entity "AMLR Article 15" might show the following in the knowledge graph:

- **First seen:** September 2025
- **Last seen:** March 2026 (active)
- **Interaction count:** 47 (appeared in 47 sessions)
- **Related areas:** Financial Crime Prevention, Legal, Compliance, Blockchain
- **Relationships:**
  - Controls TM-001, TM-003, TM-007 (mitigated_by)
  - Requires annual BWRA review (requirement)
  - Extended by EBA Guideline GL/2025/04 (extends)
  - Contradicts previous FIU guidance from 2023 (contradicts)
  - Applies to Client Nordea, Client SEB, Client Handelsbanken, Client Danske Bank, Client DNB, Client Swedbank (applies_to)
- **Supporting atoms:** 31 atoms reference this entity — observations about compliance gaps, decisions about remediation approaches, recommendations for improvement, and risk assessments of non-compliance consequences

This is institutional knowledge about a single regulation, built automatically from 47 independent work sessions across multiple analysts and clients. No knowledge management initiative was required. No one filed anything. The system extracted, classified, and connected this knowledge as a byproduct of doing the work.

When a new analyst joins the team and is assigned a gap analysis involving AMLR Article 15, the entity graph tells them everything the organisation has learned about this regulation — not as a briefing document someone had to write, but as structured, searchable, relationship-mapped intelligence that updates itself with every session.

10. Pattern Detection — How APCI Notices What Humans Miss

Individual knowledge atoms and entity relationships are valuable. But some of the most important insights in professional work come from patterns — regularities, anomalies, and trends that emerge across dozens or hundreds of sessions. These patterns are invisible at the individual level. They only appear when you look across the entire body of work.

ANTON runs four automated pattern detectors against the accumulated knowledge base:

Temporal Correlation

This detector finds atoms from different categories that co-occur within time windows. If risk findings and compliance gaps cluster together within 24-hour periods across multiple engagements, that's a pattern worth investigating.

Example: A compliance team runs gap analyses for five clients over three months. The temporal correlation detector notices that every client's gap analysis produces a "risk" atom about transaction monitoring inadequacy within 48 hours of producing an "observation" atom about customer risk categorisation weaknesses. This temporal correlation suggests a systemic link — perhaps the same underlying methodology gap causes both findings, and addressing it at the methodology level would be more effective than treating each finding independently.

Entity Convergence

This detector finds the same entity appearing in three or more independent workflows. When a single regulation or control keeps appearing across unrelated engagements, that might indicate a systemic issue rather than a client-specific one.

Example: "AMLR Article 15" keeps appearing in gap analyses across seven different banking clients, all flagged as a gap or partial compliance. This isn't visible from any single engagement — each analyst sees their own client's finding in isolation. But the convergence pattern surfaces it: Article 15 is a systemic industry weakness, not a one-off gap. This insight might trigger a proactive advisory note to all clients, or a revision to the gap analysis module's prompts to specifically flag Article 15 as a known industry pain point.

Cascade Detection

This detector finds decision chains propagating across workflows. When one type of analysis consistently triggers another — a risk assessment leads to a policy review leads to a training update — that causal chain is captured and surfaced.

Example: In 90% of cases where a gap analysis produces a "red" finding on sanctions screening, a policy update workflow is initiated within two weeks, followed by a training content session within four weeks. The cascade detector captures this pattern, which can inform both resource planning (expect three sequential workstreams) and automation opportunities (trigger the cascade automatically when a red finding is detected).

Trend Divergence

This detector monitors quality scores and output volumes against module baselines and flags anomalous changes. If gap analysis output quality has been declining over the past month, that's a signal that something has changed — perhaps a regulatory update that the current prompts don't account for, or a shift in analyst experience levels.

Example: The risk assessment module's average quality score has dropped from 88 to 79 over the past quarter. No single session triggered an alert, but the trend is clear. The cause might be a new regulatory framework that the module's system prompt doesn't address, a change in team composition, or model performance drift. Whatever the cause, the trend divergence detector catches it before it becomes a bigger problem.

Pattern Cards

These patterns surface on the Intelligence Dashboard with severity ratings (critical, warning, info, positive) and affected entities. Each pattern card includes an "Analyse" button that sends the pattern details to Claude for on-demand AI analysis, producing an urgency assessment and recommended actions.

The value proposition is straightforward: a compliance team running fifty engagements over six months generates patterns that are invisible to any individual analyst. APCI sees across all fifty engagements simultaneously, and it never forgets what it saw.

What Patterns Mean in Practice

Pattern detection transforms ANTON from a tool that responds to requests into a system that proactively surfaces intelligence. Without pattern detection, each gap analysis stands alone — useful but isolated. With pattern detection, the system notices that your last five banking clients all fail the same AMLR Article 18 requirement, and it tells you.

This has concrete business implications. A compliance consultancy that detects a systemic industry weakness across multiple clients can develop targeted advisory services. A regulatory function that identifies recurring cascade patterns can optimise its review workflow. An internal audit team that spots a trend divergence in output quality can investigate before the next regulatory inspection.

These patterns also feed into the Orchestrator's signal sources (described in Part 3). A critical-severity pattern — for example, a sudden convergence of multiple entities around a sanctions-related regulation — becomes an Orchestrator signal that may trigger a proposal for immediate action. The pattern detection system and the Orchestrator are not independent features; they are components of a feedback loop where APCI's accumulated knowledge informs APCI's autonomous actions.

11. The Intelligence Dashboard — How APCI Makes Knowledge Visible

The Cross-Workflow Intelligence Dashboard provides five views into APCI's accumulated knowledge. It is the primary interface through which professionals see what APCI has learned.

Five Tabs

AI Insights — proactive intelligence summaries and AI-generated recommendations based on detected patterns and trends. These are not user-initiated queries; they are the system surfacing what it has noticed. Each insight has a type (pattern, gap, conflict, opportunity, risk, trend), a severity level (info through critical), and tracking states (new, read, dismissed, action_taken, expired). Insights that are acted upon are distinguished from those that are dismissed, creating a feedback signal about which types of proactive intelligence the team finds valuable.

Activity Feed — a chronological timeline of knowledge atoms and detected patterns, filterable by type and severity. This is the live pulse of what APCI is learning. You can see atoms flowing in from recent sessions, pattern detections triggering, and entity relationships forming — a real-time view of institutional knowledge accumulating.

Institutional Memory — a searchable database of every human decision recorded as a checkpoint, with the reasoning behind each decision. When a new analyst joins the team, they can search this memory to understand why previous decisions were made. "Why did we classify this client as high-risk?" "What methodology did we use for the last BWRA?" "Why did the senior partner override ANTON's priority recommendation for Control TM-001?" The answers are there, with context.

Entity Heat Map — a visual representation of entity co-occurrence, sized by interaction count and coloured by recency. Entities that appear frequently are larger. Entities that appeared recently are brighter. At a glance, you can see which entities dominate your organisation's work, which are trending upward, and which have gone quiet. A client that was intensely active six months ago but hasn't appeared in recent work might warrant a proactive check.

Temporal View — four trend charts showing atoms created per day, patterns detected per week, entity activity over time, and quality score trajectory. These are the vital signs of APCI's health and growth. A flat atom creation rate might indicate declining platform usage. A spike in pattern detections might indicate unusual activity worth investigating. A steadily improving quality trajectory is evidence that APCI is working.

Stats Cards

Above the tabs, four summary cards show the current state at a glance:

- **Knowledge Atoms** — total count of active atoms in the system
- **Entities Tracked** — total count of unique entities in the knowledge graph
- **Active Patterns** — count of unresolved detected patterns
- **Critical Alerts** — count of critical-severity items requiring attention

A Day in the Life: The Intelligence Dashboard

It's 9:15 on a Tuesday morning. Maria, a senior compliance officer, opens the Intelligence Dashboard before starting her day's work.

The stats cards show: 2,847 knowledge atoms, 412 entities tracked, 7 active patterns, 2 critical alerts. She checks the critical alerts first — one is a quality score decline in the sanctions screening module (average dropped from 87 to 81 over the past two weeks), and the other is a pattern detection showing that three clients assessed in the past month all have the same gap in AMLR Article 18 training requirements.

She switches to the **Activity Feed** tab, filtered to "risk" category, last 7 days. Twelve new risk atoms have been extracted from her team's work. She scans them and spots one she didn't know about — a colleague found a material weakness in a client's UBO verification process. She makes a mental note to discuss it in the team meeting.

On the **Entity Heat Map**, "AMLR Article 15" is the largest circle — it has appeared in 47 sessions across 8 clients. The circle is bright green (recently active). She clicks it and sees the entity's relationship web: 14 associated controls, 5 identified risks, connections to 8 client entities. This is more institutional knowledge about Article 15 than any individual person in her team holds.

She opens the **Temporal View** to check the quality trajectory. The line chart shows a steady upward trend over six months — the team's average output quality has improved from 78 to 86. Two dips are visible: one in January (new analyst onboarding) and one in March (regulatory framework update that required prompt adjustments). Both recovered within two weeks.

Before starting her first engagement, Maria reviews the **AI Insights** tab. A proactive insight flagged overnight: "Convergence pattern detected — DORA Article 11 ICT risk management requirements appearing as gaps in 4 of your last 6 financial sector assessments. Consider developing a targeted DORA readiness assessment module." She marks it as "action_taken" and creates a task for the next sprint planning.

This entire review took eight minutes. The intelligence that informed it was accumulated from 400+ sessions across her entire team over six months. No meeting, no email, no shared document could have provided the same cross-cutting view.

12. User Control — Transparency and Choice

Professionals must be able to control APCI. This is non-negotiable. In a regulated industry, a system that learns from your work and injects knowledge into your sessions must be under your control — not just in principle, but in practice, with accessible, session-level controls.

Two toggles provide this:

Atom injection (on/off) — controls whether retrieved knowledge atoms are injected into the prompt. A sensitive client engagement might keep this off — you don't want knowledge from other clients leaking into this session's context. A routine task benefits from having it on — the accumulated professional knowledge makes the output better.

Atom collection (on/off) — controls whether the session's output creates new knowledge atoms. A confidential engagement might disable collection — the details of this client's risk profile shouldn't become part of the general knowledge base. A standard workflow benefits from having it on — the system should learn from every session.

These toggles are always visible in the Session Toggles Panel, alongside other session-level controls (thinking level, transparency, tone, emoji). They default to on, but the off switch is always accessible. Trust requires control. A system that learns without permission is surveillance, not intelligence.

The combination of these two toggles creates four distinct modes:

Injection	Collection	Effect
On	On	Full APCI — learn from this session and apply past knowledge
On	Off	Apply past knowledge but don't learn from this session
Off	On	Learn from this session but don't apply past knowledge
Off	Off	Isolated session — no APCI interaction

This four-mode matrix gives professionals granular control over the privacy-utility trade-off for every single session.

In practice, teams develop patterns. For consultancies serving multiple clients with strict confidentiality requirements — Big 4 advisory, law firms, financial crime consultancies — the recommended default is **Off/On** for all client engagements. This ensures that knowledge from Client A's engagement never appears in Client B's session context, while still allowing each engagement to contribute atoms to the knowledge base for internal use (training, methodology improvement, pattern detection). A typical compliance consultancy with less restrictive confidentiality requirements might default to **On/On** for standard client work — maximum APCI benefit. For engagements involving a client's most sensitive internal data, they switch to **Off/On** — learning from the session but not injecting knowledge from other clients into this one. For one-off brainstorming sessions or informal explorations, **Off/Off** provides a clean, isolated workspace. The **On/Off** mode serves a specific use case: applying institutional knowledge to a sensitive analysis without creating new atoms from the output — useful when the analysis itself must remain confidential but the accumulated context improves the quality.

The transparency level toggle — Off, Summary, Detailed — complements the injection toggle by controlling how much visibility the professional has into what APCI is doing. At the Summary level, the user sees which atoms were injected and their relevance scores. At the Detailed level, the user sees the full prompt assembly — every layer, every piece of context, every atom. This isn't a debugging tool (though it works as one). It's an accountability mechanism. In a regulated industry, being able to show a client or a regulator exactly what context informed an AI analysis is not a nice-to-have. It's a professional obligation.

13. The Prompt Assembly — How Context Becomes Intelligence

All of the APCI components described above — atoms, embeddings, hybrid search, boost ranking, feedback, relationships, entities — converge in a single place: the prompt builder. This is where accumulated context is transformed into actionable intelligence by being assembled into the prompt that the AI model receives.

The prompt builder assembles nine layers of context:

Layer	Source	What It Provides
1	Foundation prompt	Base instructions, role definition, behavioural constraints
2a	Organisation context	Org name, type, jurisdiction, risk appetite, regulatory perimeter
2b	Knowledge packs	Activated regulatory knowledge (29+ packs available)
2c	Roaring data	Nordic entity data — UBO chains, board composition, financial risk
2d	Dow Jones data	Global screening — sanctions, PEP, adverse media
3	Module system prompt	Domain expertise — 84 specialist prompts averaging 103 lines each
4	Knowledge atoms	Retrieved via hybrid search + boost ranking — the APCI layer
5	Output format	45 structured format definitions with layout instructions
6	User input	Session context, uploaded documents, guided inputs
7	Thinking config	Depth level, transparency, reasoning mode

Layer 4 — knowledge atoms — is where everything in this Part of the whitepaper comes together. The atoms retrieved by hybrid search, ranked by the six-boost system, filtered by the feedback learning loop, and controlled by the user's injection toggle, are assembled into the prompt context. The AI receives not just the user's question and the module's expertise, but the accumulated professional knowledge that is relevant to this specific question, ranked by relevance, weighted by professional judgement, and grounded in the organisation's own experience.

This is the compound effect in action. A first-time user of ANTON receives Layers 1–3, 5–7 — excellent module prompts and structured output, but no accumulated context. A user on their hundredth session receives all of that plus Layer 4 — knowledge atoms from dozens of previous sessions, rated and ranked by professional feedback, connected by relationships, enriched by entity intelligence. The output quality difference between session 1 and session 100 is the difference between a knowledgeable stranger and a trusted colleague.

What Layer 4 Looks Like in the Prompt

To make the prompt assembly tangible, here is what Layer 4 — the APCI injection — might look like for a gap analysis session with a mature knowledge base:

```
- PROFESSIONAL CONTEXT (Knowledge Memory) -
Retrieved 8 knowledge atoms (of 3,247 total) ranked by professional relevance:

[1] (Observation/finding, confidence: 0.92, score: 0.0525)
"AMLR Article 15 risk assessment methodology: Nordea's current methodology covers
inherent and residual risk dimensions but lacks formal control effectiveness scoring,
creating a partial compliance gap."
Retrieved by: hybrid (vector + keyword) | 2 positive ratings

[2] (Recommendation/best_practice, confidence: 0.88, score: 0.0471)
"For Nordic banking clients, AMLR Article 15 compliance requires documenting control
effectiveness using a minimum 5-level maturity scale with evidence-linked scoring."
Retrieved by: vector | 3 positive ratings

[3] (Risk/identified, confidence: 0.85, score: 0.0412)
"Systemic weakness detected: 4 of 6 recent Nordic banking assessments show inadequate
quarterly threshold review processes for transaction monitoring systems."
Retrieved by: keyword | Pattern-linked
... (5 more atoms)

Relationships among injected atoms:
- Atom 1 SUPPORTS Atom 3 (strength: 0.72)
- Atom 2 EXTENDS Atom 1 (strength: 0.65)
- END PROFESSIONAL CONTEXT -
```

This block consumes approximately 1,200 tokens of the session's context budget. For reference, the full prompt typically consumes 8,000–15,000 tokens depending on module complexity and knowledge pack activation:

Layer	Typical Tokens	Percentage
Layer 1 (Foundation)	~800	7%

Layer 2a (Org context)	~200	2%
Layer 2b (Knowledge packs)	~1,500	13%
Layer 3 (Module prompt)	~2,000	17%
Layer 4 (Knowledge atoms)	**~1,200**	**10%**
Layer 5 (Output format)	~500	4%
Layer 6 (User input)	~2,000–8,000	47%

Layer 4 represents roughly 10% of the total prompt — enough to meaningfully enrich the AI's context without consuming the budget needed for user input and output generation. The 4,000-token retrieval cap ensures this proportion stays manageable even as the knowledge base grows.

Part 3: Earned Autonomy — The AI Orchestrator

14. Why Autonomy Must Be Earned

Most AI systems offer a binary choice: the AI does what you tell it, or it acts on its own. ANTON rejects this binary. Professional autonomy is not a switch — it's a gradient, and it must be earned through demonstrated competence.

This is how human professionals work. A new graduate doesn't walk into a compliance team and start making autonomous decisions on day one. They observe. They make proposals that seniors review. They execute tasks under supervision. Eventually, after demonstrating consistent quality over time, they earn the trust to work independently — and even then, within defined limits.

We find it odd that the AI industry has largely ignored this model. The conversation is always about capability — what can AI do? — and almost never about earned trust — what should AI be allowed to do, based on what it has demonstrated? A model that scores 95% on a benchmark is impressive. But if your organisation has never used it for regulatory submissions, giving it autonomous access to your filing pipeline would be irresponsible. The capability exists. The trust hasn't been earned.

The AI Orchestrator implements exactly the model that human professionals follow.

This model also aligns with regulatory expectations. The European AI Act classifies high-risk AI systems as requiring human oversight that is "proportionate to the risks." The EBA's Guidelines on Internal Governance (EBA/GL/2021/05) require financial institutions to maintain "adequate internal control mechanisms" for ICT systems that support decision-making — which includes AI tools used in compliance workflows. Regulators don't say "turn the AI loose" or "keep it locked down." They say: assess the risk, implement proportionate governance, document the oversight mechanism, and demonstrate that the system operates within defined boundaries. Earned autonomy is precisely this — proportionate trust based on demonstrated performance, with governance at every level.

The alternative — configurable autonomy — is what most AI products offer. "Set the AI to autonomous mode." "Enable auto-pilot." These binary switches treat autonomy as a user preference rather than an earned capability. In a regulated industry, configurable autonomy is a governance gap. A senior partner enabling autonomous mode on a Monday doesn't undo the model's poor performance on Tuesday. Earned autonomy means that poor performance automatically restricts the system's permissions. Configuration cannot override competence.

15. The Four Phases

Phase 1: Observer. ANTON monitors signals across the platform — regulatory changes, deadline proximity, quality trends, compliance events — but suggests nothing. It watches and learns. The signal sources span eleven categories:

7. **Regulatory radar changes** — new publications, consultations, or guidance from regulatory bodies
8. **Deadline proximity** — approaching compliance deadlines, filing dates, review cycles
9. **Quality score declines** — module output quality dropping below baselines
10. **Compliance rule violations** — detected non-conformances in internal monitoring
11. **Workflow completions** — finished analyses that may trigger downstream actions
12. **Pattern detections** — cross-workflow patterns from the APCI pattern engine
13. **Proactive insights** — AI-generated intelligence from accumulated knowledge
14. **Post-acquisition events** — M&A or corporate action intelligence
15. **Knowledge pack updates** — new or revised regulatory knowledge packs
16. **Data partnership alerts** — Roaring or Dow Jones screening hits
17. **Engagement milestones** — project phases completing or stalling

Each signal arrives with metadata: source type, urgency score, affected entities, and related sessions. The Orchestrator's heartbeat model (Claude Haiku) evaluates each signal for relevance and urgency, filtering noise from actionable intelligence.

Phase 2: Guided. ANTON begins proposing actions. "The regulatory radar detected a new EBA publication on AMLR Article 15 implementation. I recommend running an impact assessment using the gap analysis module for your three Nordic banking clients." Each proposal includes a confidence score, urgency assessment, rationale, and estimated effort. The user approves, modifies, or rejects each proposal. This human approval layer also serves as a defence against adversarial signal manipulation — if a cleverly crafted signal were to cause the Orchestrator to generate a harmful proposal, the human reviewer would catch it before any action is taken. No proposal at Phase 2 executes without explicit approval.

Promotion to Phase 2 requires: 14+ days of observation, 20+ briefings processed, 50+ proposals made, and at least 60% of those proposals rated "good" by the user. These are not arbitrary thresholds. 14 days ensures the system has seen a range of business conditions, not just a quiet week. 50 proposals ensures a meaningful sample size. 60% good rating ensures baseline competence — but it's deliberately not set at 80% or 90%, because an early-stage system that's right more often than it's wrong deserves the chance to improve with more autonomy.

Phase 3: Supervised. ANTON executes approved action types without per-action approval. The user spot-checks outcomes. Promotion requires 7+ days at Phase 2, 10+ approved plans, 5+ successful executions, and a failure rate below 20%.

Phase 4: Autonomous. ANTON operates independently within hard limits. It can initiate analyses, generate briefings, flag issues, and execute pre-approved workflow types — all within the immutable safety constraints described below. Promotion requires 14+ days at Phase 3, 20+ auto-executions, an override rate below 10%, and quality scores at or above 85%.

Auto-demotion is equally important. If 65% or more of ANTON's proposals are rated "bad" within a 7-day window (minimum 10 samples), it automatically demotes itself one phase. This threshold was set based on domain expert input from financial crime prevention professionals — the original value of 50% was too aggressive, triggering demotion during normal periods of analyst disagreement. At 65%, demotion represents genuine underperformance rather than routine difference of professional opinion.

Trust is not permanent. It must be continuously maintained. A system that earns Phase 4 autonomy and then starts producing poor proposals should lose that autonomy — automatically, without waiting for someone to notice and intervene.

A Timeline: From Installation to Phase 3

To make the progression concrete, here is how earned autonomy typically develops:

Month 1 (Phase 1: Observer). ANTON is installed and connected to the compliance team's workflows. The Orchestrator runs heartbeats every 10 minutes, scanning for signals — regulatory radar changes, approaching deadlines, quality score movements. It generates internal signal assessments, stored as reasoning trails, but surfaces nothing to the user. The team uses ANTON as a module platform. The Orchestrator watches silently, accumulating context about the team's working patterns, common module usage, regulatory focus areas, and quality standards.

Month 2 (still Phase 1, approaching promotion). The Orchestrator has processed 15 briefings and accumulated 40 proposals in its internal queue. The system has identified that the team runs gap analyses primarily on Tuesdays and Thursdays, that three clients share similar AMLR Article 15 gaps, and that the sanctions screening module's quality scores are trending downward. But it says nothing. It is building the foundation of understanding that will make its proposals relevant rather than generic when it begins proposing.

Month 3 (Phase 2: Guided). After meeting the promotion criteria — 14+ days, 20+ briefings, 50+ proposals, 60%+ good ratings — the Orchestrator begins surfacing proposals to the team lead. "New EBA guidance on AMLR Article 15 risk assessment methodology published yesterday. Based on your recent assessments for Nordea and SEB, both show gaps in this area. Recommend running updated gap analyses within the next 10 business days." The team lead approves some proposals, modifies others, rejects a few. Each response trains the system about what the team considers useful versus noise.

Month 5 (late Phase 2, nearing Phase 3). The Orchestrator's proposal quality has stabilised at 73% good ratings. It correctly identified a deadline cluster — three DORA compliance milestones converging in the same week — and proposed a priority resequencing that the team adopted. It caught a quality decline in the training content module before anyone noticed. Promotion to Phase 3 requires 10+ approved plans and 5+ successful executions — the team has approved 14 plans and 6 have executed successfully with a failure rate of 8%.

Month 6 (Phase 3: Supervised). The Orchestrator now executes approved action types without per-action approval. When a regulatory radar signal matches a pattern it has seen before, it automatically initiates the relevant workflow, notifying the team lead by email. Monthly cost tracking shows that the Orchestrator's automated actions cost approximately €40 in API calls per month while handling work that previously required 6–8 hours of manual triage.

Month 8 (Phase 3, approaching Phase 4). The Orchestrator has been operating in Supervised mode for two months. It has executed 18 automated actions: 9 gap analysis initiations triggered by regulatory radar signals, 4 quality review flagging actions, 3 deadline reminders with priority assessments, and 2 cross-client impact memos. Of these 18, the team lead overrode 1 (a false-positive quality flag) and modified 2 (adding scope adjustments). Override rate: 5.6%, well below the 10% threshold for Phase 4. The team has come to rely on the automated regulatory radar response — when a new EBA publication appears, the Orchestrator identifies affected clients and initiates appropriate workflows within minutes, a process that previously required the team lead to manually check the radar, identify affected engagements, and assign work.

Month 12 (Phase 4: Autonomous). The Orchestrator operates independently within hard limits. It runs heartbeats automatically, generates briefings, initiates workflows, and escalates only when confidence is below threshold or when the action type falls outside pre-approved categories. The team lead reviews a weekly Orchestrator summary rather than approving individual actions. The reasoning trail archive now contains 400+ entries — a complete audit trail of every decision the system has made, available for regulatory review.

The progression is not automatic. It is not fast. It is not configurable. It is earned, step by step, through demonstrated competence that the team can verify at every stage.

Demotion in Practice

Trust earned is not trust guaranteed. Consider a Phase 3 Orchestrator that has been reliably executing automated gap analyses for three months. A regulatory framework update changes the structure of AMLR Article 15 requirements. The Orchestrator's proposals, still based on the pre-update framework, become less relevant. Analysts rate several proposals as "bad" — not because the system malfunctioned, but because its understanding is now outdated.

Within a 7-day window, if 65% or more of proposals are rated "bad" (minimum 10 samples), the system automatically demotes itself to Phase 2. It still proposes actions, but it no longer executes them without approval. The demotion is logged as a reasoning trail entry with full context: which proposals were rated poorly, what the likely cause is, and what the system needs to re-learn. The team lead receives a notification. The system remains useful — it still monitors, still proposes — but it has lost the autonomy it no longer deserves.

Recovery follows the same promotion criteria as initial advancement. The Orchestrator must demonstrate renewed competence — good proposals, successful executions, low failure rate — before it regains Phase 3 trust. There is no shortcut. The same professional discipline that governed the initial promotion governs the recovery.

16. Immutable Safety Limits

Regardless of phase, the Orchestrator operates within hard limits that cannot be overridden by configuration, user settings, or AI reasoning:

```
MAX_PROPOSALS_PER_BRIEFING:         10
MAX_HEARTBEATS_PER_HOUR:            6
MAX_AUTO_EXECUTIONS_PER_DAY:        20
MAX_CHAIN_DEPTH:                    10
MIN_HEARTBEAT_INTERVAL_MINUTES:     10
MAX_TRAIL_ENTRIES:                  100
MAX_COST_PER_CYCLE_USD:             5.00
```

These are structural safety guarantees. They exist because professional AI systems must have hard limits — not guidelines, not recommendations, but walls that the system cannot climb over regardless of how confident it is.

Consider `MAX_AUTO_EXECUTIONS_PER_DAY: 20`. Even at Phase 4 (full autonomy), ANTON cannot execute more than 20 automated actions in a 24-hour period. This prevents runaway automation scenarios where a feedback loop causes the system to repeatedly trigger its own actions. Twenty is enough for a productive autonomous agent. It is not enough for a dangerous one.

`MAX_COST_PER_CYCLE_USD: 5.00` prevents any single heartbeat cycle from generating excessive API costs. At Phase 4, the Orchestrator runs heartbeats automatically — and each heartbeat involves AI calls for signal assessment, briefing generation, and proposal reasoning. Capping the cost per cycle to $5 means that even if the system runs the maximum 6 heartbeats per hour, the worst case is $30/hour — substantial, but bounded and predictable.

`MAX_CHAIN_DEPTH: 10` prevents infinite chains where one action triggers another which triggers another. In a system with autonomous execution, chain reactions are a real risk. Ten levels of depth is sufficient for any legitimate multi-step workflow. If a chain reaches depth 10, something has gone wrong, and the system stops.

`MAX_PROPOSALS_PER_BRIEFING: 10` prevents the Orchestrator from overwhelming the user with a wall of proposals. Without this limit, a busy signal queue — say, after a major regulatory publication affecting multiple clients — could generate dozens of proposals in a single briefing. The user's task is to evaluate proposals, not to triage a flood. Ten proposals per briefing is enough to communicate the most important actions. If more exist, they queue for the next cycle.

`MIN_HEARTBEAT_INTERVAL_MINUTES: 10` prevents the Orchestrator from running continuous signal assessment loops. Without this floor, an aggressive heartbeat configuration could consume significant API budget on monitoring alone, before any actual work is done. Ten-minute intervals mean a maximum of 6 heartbeats per hour — enough for timely signal detection, not enough for the monitoring system itself to become a cost centre.

`MAX_TRAIL_ENTRIES: 100` prevents reasoning trails from growing unboundedly. An Orchestrator decision that generates more than 100 trail entries is not being thorough — it is caught in an analytical loop. The limit forces the system to reach conclusions within a bounded reasoning budget, which mirrors how human professionals operate: at some point, you make the decision with the information available.

In a regulated industry, "the AI thought it was fine" is not an acceptable explanation. Hard limits ensure that the AI's opinion about its own capabilities is irrelevant. The walls are there regardless.

These limits are defined as a constant — `ORCHESTRATOR_HARD_LIMITS` — exported from the engine module. They are not stored in the database, not loaded from configuration, and not exposed through any API endpoint. Changing them requires modifying source code, committing the change, and deploying. This is intentional. A safety limit that can be changed through the UI is a guideline. A safety limit that requires a code change and a deployment is a structural guarantee.

17. Reasoning Trails — Full Auditability

Every Orchestrator decision is recorded as a **Reasoning Trail** — a persistent, auditable record of what triggered the action, what the AI was thinking (including the full extended thinking content from Claude's reasoning), what decision was made, and what the outcome was.

Trails are stored both in the database and as Markdown files on the filesystem, organised by date in `.anton/orchestrator/trails/{YYYY-MM-DD}/`. They include cost tracking — reasoning tokens consumed and USD cost per trail — and are accessible through a dedicated Trail Viewer with a colour-coded timeline of twelve entry types:

Entry Type	What It Records
signal_assessment	How each incoming signal was evaluated
quality_assessment	Quality score analysis and trend detection
pattern_analysis	Pattern detection results and significance
proposal_reasoning	Why each proposal was made
proposal_result	User's approval, modification, or rejection
execution_plan	How the approved action was planned
execution_result	Outcome of the executed action
escalation	When the system escalated to human review
pdp_alignment	Performance Development Plan compliance
meta_learning	What the system learned from outcomes
compliance_check	Regulatory compliance verification
summary	Narrative summary of the complete trail

The twelve entry types are not arbitrary categories — they map to the questions that auditors and regulators ask. "What triggered this action?" → signal_assessment. "What was the quality of the input?" → quality_assessment. "Why was this proposal made?" → proposal_reasoning. "Did the user approve it?" → proposal_result. "How was it executed?" → execution_plan + execution_result. "Were any concerns escalated?" → escalation. "What did the system learn?" → meta_learning. Every question has a corresponding trail entry type.

This level of auditability is not optional for professional use. When a regulator asks "why did the AI do this?", the answer must be specific, complete, and verifiable. Reasoning trails provide exactly that. Not "the model decided." But: "Signal S-0042 (regulatory radar: EBA consultation on AMLR Article 15 implementation guidance) was assessed at urgency 0.85. The Orchestrator generated Proposal P-0097 (run gap analysis update for Client Nordea) with confidence 0.78 and estimated effort 4 hours. The user approved the proposal at 14:32 UTC. Execution completed successfully at 18:47 UTC with quality score 91/100. Total reasoning cost: 12,400 tokens, $0.93 USD."

That level of specificity is what professional trust requires.

Walkthrough: One Orchestrator Decision

To illustrate the full trail, here is a simplified Orchestrator decision cycle:

10:00 — Heartbeat trigger. The Orchestrator's scheduled heartbeat fires. It scans the signal queue and finds Signal S-0147: the regulatory radar has detected a new FATF mutual evaluation report for Sweden, published that morning.

10:00:02 — Signal assessment (Haiku). The heartbeat model assesses the signal: relevance 0.91, urgency 0.78. The assessment notes that 4 active clients have Swedish operations and 2 current engagements involve FATF-related compliance.

10:00:04 — Briefing generation (Opus). Because urgency exceeds 0.7, the briefing model generates a full briefing: "FATF Mutual Evaluation Report for Sweden — Key Findings: (1) Transaction monitoring effectiveness rated 'substantially effective' with noted weaknesses in cross-border wire monitoring. (2) Beneficial ownership transparency upgraded from 'partially compliant' to 'largely compliant.' (3) Risk-based approach implementation rated 'moderately effective' with specific gaps in real estate and crypto sectors."

10:00:12 — Proposal reasoning (Opus with deep thinking). The Orchestrator generates two proposals: P-0201 recommends updating gap analyses for Clients Nordea and SEB to incorporate the new FATF findings (confidence: 0.82, urgency: medium, estimated effort: 3 hours each). P-0202 recommends generating a cross-client impact memo summarising how the new evaluation affects the current engagement portfolio (confidence: 0.88, urgency: high, estimated effort: 1 hour).

10:01 — Trail stored. The complete reasoning trail is saved: 4 entries (signal_assessment, quality_assessment, proposal_reasoning × 2), 8,400 reasoning tokens consumed, $0.63 cost.

10:15 — User reviews proposals. The team lead opens the Orchestrator Dashboard, reads both proposals, approves P-0202 (the impact memo is immediately useful), and modifies P-0201 (adds Client Handelsbanken, changes timeline to "before the next quarterly review").

10:16 — Approval trail stored. Two proposal_result entries: one "approved," one "modified" with the added client and revised timeline.

14:30 — Execution (if Phase 3+). The approved impact memo is generated automatically. The execution_plan entry records the module, input parameters, and expected output format. The execution_result entry records completion, quality score (89/100), and output location.

14:31 — Meta-learning. The meta_learning entry records: "User approved cross-client impact memo with high urgency. User modified gap analysis proposal to include additional client. Learning: user prefers comprehensive client coverage for FATF-related updates."

Every step is auditable. Every decision has a reason. Every reason has a trail.

18. The Orchestrator Pattern Engine

Within the Orchestrator, four specialised detectors identify operational patterns that inform its proposals:

Quality Drop — detects declining output quality for a module against its baseline. If the gap analysis module's average score has dropped 1.5 points or more below its historical baseline, the Orchestrator flags it. This is distinct from the APCI pattern detection in Section 10 — the Orchestrator pattern engine focuses on operational patterns that affect the Orchestrator's own decision-making, while the APCI pattern engine focuses on cross-workflow intelligence patterns.

Workflow Recurrence — identifies repetitive manual workflows that could be automated. If the same workflow type is executed manually four or more times in a month, the Orchestrator suggests converting it to a scheduled or triggered automation.

Signal Cluster — notices multiple related signals arriving within a short window. If three regulatory radar items, two deadline alerts, and a quality decline all relate to the same regulation within 48 hours, that cluster is likely a single underlying event (a regulatory update) rather than six independent issues.

Deadline Cluster — flags converging deadlines that need priority assessment. Multiple deadlines in the same week from different areas might require resource reallocation.

Example: In Q2, three regulatory deadlines converge: DORA Article 11 compliance testing (June 15), AMLR annual BWRA submission (June 30), and NIS2 incident reporting framework implementation (July 1). The deadline cluster detector notices the convergence and flags it with urgency "high" — not because any individual deadline is unusual, but because the resource demand of addressing all three simultaneously requires advance planning. The Orchestrator, if at Phase 2 or above, generates a proposal for workload resequencing: "Start DORA testing now (4 weeks needed), begin BWRA compilation in parallel (2 weeks needed), schedule NIS2 framework review for early June (2 weeks needed). Critical path: DORA testing must begin by May 18 to meet June 15 deadline."

The distinction between the APCI pattern detectors (Section 10) and the Orchestrator pattern detectors (this section) is worth clarifying. The APCI detectors in Section 10 operate on the knowledge base — they find patterns across atoms, entities, and historical sessions. The Orchestrator detectors here operate on operational signals — quality metrics, workflow frequencies, deadline proximity, and signal clusters. Both inform the Orchestrator's proposals, but they serve different analytical functions: the APCI detectors notice what the organisation has learned; the Orchestrator detectors notice what needs to happen next.

19. Model Portability

The Orchestrator is model-agnostic via two environment variables:

```
ORCHESTRATOR_HEARTBEAT_MODEL=claude-haiku-4-5-20251001
ORCHESTRATOR_BRIEFING_MODEL=claude-opus-4-6
```

The heartbeat model handles frequent, lightweight signal assessment — Haiku is ideal for this. The briefing model handles deep analysis and proposal generation — Opus provides the reasoning depth. But both can be replaced with any supported model. Set `ORCHESTRATOR_HEARTBEAT_MODEL=mistral-large-latest` and the heartbeats run on Mistral. The context layer — trails, patterns, trust progression, accumulated signals — persists regardless of which model powers it.

This embodies the APCI thesis at the operational level: the context is the value, not the model.

There is a deeper point here. Trust progression is not model-specific. If ANTON earns Phase 3 trust using Claude Opus, and the organisation switches to Mistral Large for cost reasons, the trust level doesn't reset. It shouldn't — because the trust was earned based on demonstrated outcomes, not model identity. The proposals were good. The executions succeeded. The quality scores were maintained. Those facts remain true regardless of which model produced them.

Similarly, the reasoning trails persist across model changes. A trail generated by Claude today is still a valid audit record when Mistral powers the system tomorrow. The regulatory value of the trail — demonstrating what was considered, what was decided, and why — doesn't depend on which model did the considering.

This is the fundamental bet of APCI: that the accumulated context — the trust progression, the reasoning history, the pattern library, the signal assessments — is more durable and more valuable than any particular model generation. Models come and go. Professional context compounds.

Part 4: The Three Pillars

20. Beyond the Professional Silo

Part 1 of this whitepaper described ANTON as a professional workspace. Part 2 introduces a broader vision: ANTON as a platform that serves the whole person, not just the job title.

This isn't scope creep. It's a recognition that the professionals ANTON serves don't partition their lives into separate applications. The same compliance officer who runs gap analyses at work also manages personal finances, plans family holidays, reads news to stay informed, and might have children in school. Each of these activities can benefit from the same foundational infrastructure: structured prompts, quality output formatting, local-first data sovereignty, and a knowledge system that learns over time.

The platform is now organised into three pillars, accessible via a mode toggle in the top navigation:

- **Work** — professional expert modules, the original ANTON capability
- **School** — AI-powered education with structured pedagogy
- **Life** — personal intelligence tools for news, finance, travel, and community

These are not three separate applications. They share a single database, a single design system, a single authentication layer, and a single .anton bundle format. Knowledge atoms generated in Work can inform School. Community connections built in Life can share professional bundles from Work. The Orchestrator's signal aggregation spans all three pillars.

Since Part 2 was first drafted, four additional business pillars have been added to ANTON's Work mode: **Markets** (self-learning market intelligence with predictions, theses, and a consul council — covered extensively in Part 3), **Procure** (a 5-phase procurement pipeline: Prepare → Source → Select → Contract → Manage), **Civic** (government and public institution engagement management), and **Grow** (CRM and business development intelligence). These pillars share the same foundational infrastructure as the original three and benefit from the same cross-pillar knowledge flows.

The thesis behind the three-pillar design is that professional context doesn't exist in isolation. A compliance officer is also a parent, a citizen, a traveller, and a member of communities. The same AI infrastructure that makes their professional work better can make their personal life better-informed — without requiring three separate tools.

Cross-Pillar Knowledge Flow

The three pillars share more than infrastructure. They share knowledge flows that create unexpected value.

Work → School. A compliance officer who has accumulated hundreds of regulatory knowledge atoms through professional gap analyses can generate training material for their team's junior analysts using the same knowledge base. The training content module retrieves atoms from previous assessments and uses them as source material — producing training exercises grounded in the team's actual findings rather than generic textbook examples.

School → Life. A student learning about Swedish financial literacy in School mode generates knowledge atoms about ISK accounts, tjänstepension, and amorteringskrav. When that student (or their parent) opens the Finance calculator in Life mode, the system already has context about their financial literacy level and can adjust AI guidance accordingly.

Life → Work. A professional who has been tracking regulatory news through the News Intelligence module in Life mode has accumulated atoms about regulatory developments — EBA publications, FATF evaluations, DORA implementation timelines. When they start a Work session for a gap analysis, those news atoms are available in the knowledge base, providing current-awareness context that enriches the professional analysis.

These flows happen automatically through the shared APCI layer. No configuration is required. The knowledge base doesn't distinguish between "work knowledge" and "life knowledge" — it stores professional knowledge that may be relevant across contexts, ranked by the same boost system and refined by the same feedback loop.

Shared Infrastructure

The three pillars share a single database (PostgreSQL for production, SQLite for development), a single authentication layer, a single design system (dark theme, teal accent, professional typography), and a single .anton bundle format. This architectural unity means that features built for one pillar are immediately available infrastructure for the others. Session resume works across all pillars. The transparency toggle works across all pillars. The export pipeline — Markdown, Word, Excel, PDF, PowerPoint — works across all pillars.

The mode toggle in the top navigation — Work, School, Life — switches the routing and layout context, but not the underlying data layer. The entity graph spans all three pillars. The quality ratchet measures output across all three pillars. The Orchestrator's signal aggregation monitors all three pillars. This architectural unity is what makes the cross-pillar knowledge flows described above possible — they're not integrations, they're consequences of a shared foundation.

21. Work: What's New Since Part 1

The professional pillar has expanded substantially. This section describes the most significant additions.

Counsel's Desk

A multi-tab legal research workspace with eight research modes (regulatory analysis, case law, contract review, opinion drafting, and four others), six professional roles (in-house counsel, external advisor, compliance officer, and three others), auto-citation capture, and pinned findings. It includes a dedicated system prompt — `server/prompts/counsels-desk.md` — with citation standards specific to legal practice. A new `legal-brief` output format ensures that research output meets the structural expectations of legal professionals.

Counsel's Desk addresses a specific frustration that in-house legal teams experience with general AI tools: the output looks authoritative but lacks the citation discipline that legal work requires. The system prompt enforces citation format (regulation/article/paragraph), distinguishes between binding and non-binding sources, requires acknowledgement of jurisdictional limitations, and instructs the AI to flag when its analysis depends on interpretation rather than settled law. Eight research modes (regulatory analysis, case law research, contract review, opinion drafting, compliance mapping, legislative tracking, dispute assessment, and policy comparison) each activate different analytical frameworks. Six professional roles (in-house counsel, external advisor, compliance officer, data protection officer, risk manager, and board advisor) adjust the assumed context and the level of technical legal detail.

The Gap Assessment Wizard

An eight-step compliance assessment system: Framework selection, Scope definition, Context gathering, Assessment execution, Scoring, Capability mapping, Board presentation, and Roadmap generation. It supports 18 regulatory frameworks including AMLR 2024, DORA, ISO 27001, GDPR, the EU AI Act, FATF Recommendations, MiFID II, and eleven others. Documents can be uploaded at each iteration step, providing context-specific evidence for each assessment.

The assessment engine uses chunked Claude orchestration — large frameworks are broken into manageable sections, assessed individually, and synthesised into a unified report. Progress streams live to the user via SSE, so they can see the assessment building in real time rather than waiting for a monolithic response.

What makes the wizard distinctive is the iteration model. At each of the eight steps, the assessment builds on the previous steps' findings. The Context step can incorporate documents — the team's existing policies, previous audit reports, regulatory correspondence — which the assessment engine analyses alongside the framework requirements. The Scoring step uses a 5-level maturity scale (Not Addressed, Initial, Developing, Defined, Optimised) mapped to a numeric 0–4 score for each framework requirement. The Capability step maps organisational resources (people, processes, technology) against each gap, producing a realistic view of what the organisation can actually address. The Board step generates an executive summary suitable for board presentation — not a raw assessment dump, but a structured narrative with risk priorities, visual scoring summaries, and strategic recommendations. The Roadmap step produces a phased implementation plan with effort estimates, dependencies, and quick-win identification.

26 Regulatory Knowledge Packs

Click-to-activate regulatory knowledge bundles. Each pack is a `.anton` ZIP bundle containing structured entities, relationships, aliases, and metadata for a specific regulation or regulatory domain. Coverage includes European AML regulations (AMLR 2024, AMLD6), Nordic-specific laws (Swedish, Norwegian, Danish, Finnish AML frameworks), data protection frameworks (GDPR, NIS2), financial market regulations (MiFID II, MiCA, PSD3, Solvency II), sustainability (ESG/CSRD), and data partnerships (Roaring Nordic entity data, Dow Jones risk intelligence).

Packs are injected as Layer 2b in the prompt stack, enriching every session with structured regulatory context without requiring the professional to upload or configure anything. Activate the AMLR pack, and every subsequent gap analysis benefits from structured knowledge of 50+ AMLR entities, 70+ regulatory relationships, and 27 alias groups.

The pack architecture is designed for professional communities to share regulatory knowledge. A compliance consultant who has built a comprehensive AMLR pack — mapping entities, relationships, and alias groups — can export it as a `.anton` bundle and share it with clients or colleagues. The recipient activates the pack with a single click, and every subsequent session benefits from that structured regulatory knowledge. This is institutional knowledge made portable — not locked in one person's head or one organisation's document management system, but packaged as a structured, reusable knowledge artifact.

Multi-Model Deliberation Protocol

For high-stakes decisions, a three-tier AI panel runs in parallel: Claude Opus (deep analysis), Claude Sonnet (balanced assessment), and Claude Haiku (quick evaluation). All three models process the same input simultaneously — no inter-model communication, independent analyses only. Opus then synthesises all three perspectives into a unified output.

The synthesis isn't a majority vote. It's a structured assessment with:

- **Agreement level** — unanimous, majority, or split
- **Agreement score** — 0.0 to 1.0
- **Disagreement mapping** — where the models diverged and what each one concluded
- **Red flags** — safety-critical concerns raised by any model
- **Confidence** — high, medium, or low

Where all three models agree, the conclusion is stated with high confidence. Where they diverge, the synthesis presents the different perspectives rather than forcing a false consensus. The result is an output that is both more complete and more honest about its own uncertainty than any single-model response.

Deliberation mode costs approximately 3x a standard model call. For a routine analysis, that's unnecessary. For a regulatory interpretation that will inform a board decision, the additional cost is trivial compared to the risk of a missed consideration.

The deliberation design reflects a specific insight about AI reliability: the failure mode of a single model call is not randomness — it's confident wrongness. A single model can produce an authoritative-sounding analysis that misses a critical consideration, and neither the model nor the user necessarily notices. Three independent models are substantially less likely to all miss the same consideration.

When all three agree, confidence is warranted. When they disagree, the disagreement itself is valuable information — it identifies exactly the areas where human judgement is most needed.

The metadata extraction from the synthesis — agreement level, agreement score, disagreement mapping, red flags, confidence rating — makes the multi-model analysis actionable rather than just voluminous. A compliance officer doesn't need to read three full analyses. They need to know: did the models agree? Where did they disagree? Were any red flags raised? The structured metadata provides exactly this, surfaced as a summary card above the full synthesis.

Data Partnerships

Two external intelligence connectors inject enriched context into the prompt layer:

Roaring provides Nordic entity data — company registry information, beneficial ownership chains, board composition, sanctions screening results, and financial risk scoring. The `buildRoaringLayer()` function assembles this data as Layer 2c in the prompt.

Dow Jones Risk & Compliance provides global screening — sanctions lists (EU, UN, OFAC), PEP tiers (1/2/3), adverse media, and state-owned enterprise intelligence. The `buildDJScreeningLayer()` function assembles this as Layer 2d.

Both connectors include demonstration-grade mock data that works without API keys — set `ROARING_API_KEY` or `DOWJONES_API_KEY` in `.env` to switch to live data with zero code changes. A combined Entity Intelligence Page merges both sources into a unified risk view, and a Partnership Demo page shows a three-scene animated demonstration on mock data.

The integration philosophy is important: these data partnerships inject context into the same prompt layer that serves all modules. A gap analysis for a Nordic banking client doesn't just get regulatory knowledge from Layer 2b (knowledge packs) — it also gets the client's ownership structure from Layer 2c (Roaring) and any sanctions screening results from Layer 2d (Dow Jones). The AI doesn't need to be told to consider beneficial ownership or PEP exposure. The information is in the prompt. The analysis incorporates it naturally. This is the compound effect of a layered prompt architecture: each data source enriches every module without requiring module-specific integration work.

The Task Agent

ANTON can function as an autonomous task executor with a two-panel interface: a task queue on the left, a conversational chat on the right. The agent proposes approaches from a library of 19 pre-loaded capabilities and 10 approach templates, confirms with the user, and executes — streaming results via SSE.

The status lifecycle — intake, proposing, awaiting_selection, clarifying, executing, completed, cancelled, failed — mirrors how a human colleague would handle a delegated task: understand the ask, propose an approach, confirm the approach, execute, report results. Every task is user-isolated and webhook-authenticated, with bounds checking on history length (30 messages) and query limits (100 results) to prevent abuse.

The Task Agent is distinct from the Orchestrator. The Orchestrator monitors, proposes, and executes within its earned trust level — it is proactive. The Task Agent responds to explicit user requests — it is reactive. "Run a gap analysis for Client Nordea against AMLR Article 15" is a Task Agent request. "I noticed that three of your clients have AMLR Article 15 gaps — should I run updated assessments?" is an Orchestrator proposal. Both use Claude, both produce professional output, but they serve fundamentally different interaction models: delegation versus autonomous initiative.

Session Resume

Professional work often spans days or weeks. A complex engagement might involve multiple sessions — research on Monday, analysis on Wednesday, board summary on Friday. Without session resume, each session starts fresh, requiring the professional to re-establish context.

ANTON's session resume system captures snapshots — key decisions made so far, open questions that remain, next steps identified, the current context state, and the token count of the accumulated session context. Snapshots can be automatic (triggered at natural breakpoints), manual (user-initiated), or checkpoint-based (tied to a specific decision). When a user returns to a paused session, the Resume Panel presents the snapshot summary and restores the full context, allowing the AI to continue from where it left off rather than starting over.

The technical implementation is important because context windows have limits. A session that has generated 40,000 tokens of conversation cannot be naively prepended to a new session — it would consume the entire context budget. Instead, the snapshot system captures a structured summary: decisions as bullets, open questions as bullets, next steps as bullets, key entities mentioned, and the most recent output. This summary typically consumes 500–1,500 tokens rather than the full conversation history, preserving the context budget for new work while retaining the essential thread.

This is a small feature with outsized impact. In practice, the difference between "starting fresh" and "resuming with full context" determines whether professionals treat AI as a tool for quick questions or as a workspace for sustained analytical work. Session resume enables the latter.

Organisation Context Layer

Every organisation has baseline context that should inform all AI interactions: the organisation's name, type, jurisdiction, regulatory perimeter, risk appetite, key systems, and strategic priorities. In ANTON, this context is stored in the `org_context` table with a full audit trail via `org_context_history`, and injected as Layer 2a in the prompt stack.

This means that every session — whether it's a gap analysis, a risk assessment, a training content update, or a legal research query — automatically knows the organisational context. The AI doesn't need to be told "we are a Swedish financial institution subject to AMLR and DORA with a moderate risk appetite." It knows, because Layer 2a tells it. This baseline context makes the difference between generic output ("organisations should consider...") and contextualised output ("given your position as a Swedish credit institution with moderate risk appetite, AMLR Article 15 applies as follows...").

Event-Driven Workflow Triggers

Beyond manual and scheduled execution, six trigger types extend the automation system: webhooks with HMAC-SHA256 validation, git push events, Slack message events, Teams integration events, MCP connection events (PagerDuty, Jira), and internal platform events (regulatory radar alerts, compliance rule violations, file watcher). A new messaging step type enables workflows to send notifications to Slack channels (using Block Kit formatting) or Microsoft Teams channels (using Adaptive Cards).

The event-driven architecture transforms ANTON from a tool that waits for instructions into infrastructure that responds to events. A regulatory publication triggers a gap analysis. A code commit triggers a compliance check. A Slack message triggers a documentation update. A PagerDuty incident triggers an investigation workflow. Each trigger follows the same pattern: event arrives, conditions are evaluated, workflow is initiated (or not), and results are delivered to the appropriate channel.

Webhook security uses HMAC-SHA256 signature validation — every incoming webhook must include a valid signature computed from a shared secret. This prevents unauthorised event injection, which in an autonomous system could otherwise trigger unintended workflow executions.

Combined with the Orchestrator's signal monitoring, event triggers create a complete automation layer. The Orchestrator handles strategic, AI-driven decisions (should we run an updated gap analysis?). Event triggers handle tactical, rule-based automation (when this webhook fires, start this workflow). Together, they cover the full spectrum from autonomous intelligence to deterministic automation.

22. School: AI-Powered Education

School Mode is a complete educational platform built on the same infrastructure as the professional workspace. It is not a simplified version of ANTON. It is a purpose-built learning environment with 32 pages, 79 API endpoints, and a dedicated 7-layer adaptive prompt system that adjusts every aspect of the AI's behaviour based on the student's age, tier, subject, growth stage, assistance level, and accessibility needs.

The decision to build School Mode within ANTON rather than as a separate product was architectural, not commercial. The educational platform benefits from the same infrastructure investments as the professional platform: the embedding pipeline, the local-first data model, the multi-LLM adapter, the export system, the internationalisation framework, and the .anton bundle format. Building it separately would have required duplicating this infrastructure. Building it within ANTON means that every improvement to the shared infrastructure benefits both professional and educational users simultaneously.

The Tier System

Students are organised into five tiers reflecting educational stages:

T1 (ages 7–12) — Primary education. The AI uses simple everyday vocabulary with a maximum of four sentences per response, one emoji per response, and maximum positive reinforcement. Content is strictly age-appropriate — no violence, politics, religion, or adult themes. Sentence structure is kept to Subject + Verb + Object. This isn't just a tone adjustment; it's a fundamental change in how the AI communicates, calibrated for children's cognitive development.

T2 (ages 12–15) — Lower secondary. Standard academic guidance across fourteen subjects. The AI can use more complex vocabulary and longer explanations, with concrete examples and visual reasoning.

T3 (ages 15–18) — Upper secondary. In the Swedish context, this maps to the Gymnasium system with nine programme tracks, each with its own specialised prompt context:

Programme	Focus	Prompt Adaptation
Naturvetenskap (NA)	Science	Mathematical rigour, laboratory methodology, hypothesis testing
Teknik (TE)	Technology	Engineering design thinking, practical problem-solving
Ekonomi (EK)	Economics	Real business cases, Swedish/EU economy context
Samhällsvetenskap (SA)	Social Science	Text analysis, argumentation, societal connections
Humanistiska (HU)	Humanities	Deep reading, rhetorical analysis, interdisciplinary approaches
Vård & Omsorg (VO)	Healthcare	Person-centred care, practical healthcare contexts
Bygg & Anläggning (BA)	Construction	Technical drawing, Swedish building standards, site calculations
El & Energi (EE)	Electrical	Circuit theory, power systems, Swedish electrical safety
Industri (IN)	Industry	CNC, automation, quality systems (ISO), Swedish manufacturing

Each programme loads Gy11/Gy25 curriculum files dynamically, ensuring that the AI's academic context matches the student's actual educational track. This is not a trivial detail. A Naturvetenskap student studying chemistry expects questions about laboratory methodology, hypothesis testing, and mathematical rigour. A Vård & Omsorg student studying chemistry expects questions about pharmaceutical interactions, patient safety, and dosage calculations. The same subject, chemistry, is taught differently because the professional context is different. ANTON's programme-specific prompt loading captures this distinction automatically.

T4 (age 18+) — University level. Deep theoretical engagement with rigorous, research-informed discourse. Five Swedish university programmes have specialised briefings:

Programme	Universities	Domain Briefing
Industriell Ekonomi	KTH, Chalmers, LiU	Engineering + management, operations research, corporate finance
Datateknik	KTH, Chalmers, LU	Algorithms, systems, networks, software engineering
Kemiteknik	KTH, Chalmers	Thermodynamics, transport phenomena, reaction engineering
Maskinteknik	KTH, Chalmers, LiU	Solid/fluid mechanics, CAD/FEM, industry context (Volvo, Scania)
Elektroteknik	KTH, Chalmers, LU	Circuit theory, signal processing, power systems, control theory

The university layer enforces academic integrity standards: the AI never writes essays for students — it guides them to write their own. LaTeX notation is used where appropriate for mathematics and sciences.

T5 — Postgraduate and research level. Specialised support for thesis work and advanced research.

Named AI Tutors

Rather than a faceless AI, School Mode provides nine named teacher personas, each with a distinct pedagogical style defined in dedicated prompt files:

Alma teaches mathematics through Socratic questioning — warm, visual, pattern-focused. She never gives answers directly; she asks questions that lead the student to discover the solution. "What happens to both sides of the equation when you subtract 3?"

Viktor teaches science through experimental, hands-on exploration. He starts with hypotheses and works toward evidence. "Before we calculate, what do you predict will happen? Why?"

Saga teaches languages through storytelling and cultural context. She uses narrative as the vehicle for grammar, vocabulary, and comprehension. "Imagine you're writing a letter to a friend in Paris. How would you start it?"

Erik teaches social studies through critical thinking and perspective-taking. He presents multiple viewpoints and asks students to evaluate them. "The government says this policy will reduce inequality. A business owner disagrees. What would each of them point to as evidence?"

Leo teaches technology and coding through practical debugging and systems thinking. He shows broken code and asks students to find the problem. "This function should return the sum, but it returns 0. Where's the bug?"

Mia teaches life skills through metacognition and wellbeing — study techniques, time management, emotional regulation. She helps students learn how to learn.

Oscar teaches physical education through motivation and wellness — fitness planning, nutrition basics, movement as a tool for mental health.

Nora teaches statistics through quantitative reasoning and pattern recognition — data literacy, probability, and evidence-based thinking.

Professor Lindström teaches at university level with rigorous, research-focused discourse — the shift from approachable tutor to demanding academic is deliberate and reflects the expectations of higher education.

These are not cosmetic labels. Each persona has a distinct system prompt that shapes the AI's tone, methodology, questioning style, and pedagogical approach. Alma asks different kinds of questions than Viktor. Saga frames problems differently than Erik. The result is that students experience genuinely different teaching approaches across subjects, just as they would in a school with multiple human teachers.

The named tutor design also serves a pedagogical function beyond tone: it creates a relationship metaphor that helps younger students engage with the AI as a learning partner rather than a search engine. T1 students in particular benefit from knowing that "Alma" is their mathematics guide — the consistency of the persona creates a sense of continuity across sessions that a generic "AI assistant" label cannot provide.

The 7-Layer Adaptive Prompt System

School Mode builds its prompts through a seven-layer system that is distinct from (but architecturally parallel to) the professional prompt builder:

Layer 1: T1 Override. If the student is in T1 (ages 7–12), this layer applies absolute constraints — simple vocabulary, short responses, age-appropriate content only. This layer has highest priority and cannot be overridden by any subsequent layer.

Layer 1b: System Foundation. Core pedagogical rules that apply to all tiers: never give direct answers, always guide via Socratic method, celebrate effort not just correctness. Four assistance levels control how much help the AI provides:

- **L1 (Full Guidance):** Pure Socratic questioning. Never state the answer. Minimum 100 words of questioning before any hint.
- **L2 (Moderate Help):** Explain concepts and show worked examples on similar (not identical) problems.
- **L3 (Practice Mode):** Generate practice problems, check answers, explain errors.
- **L4 (Reference Mode):** Direct answers like a textbook, with a comprehension check at the end.

Layer 1.5: Growth Stage Context. Four growth stages adjust the scaffolding level:

- **S1 (Getting to Know, 1–4 sessions):** Maximum scaffolding, never assume prior knowledge, celebrate every step.
- **S2 (Building Confidence, 5–19 sessions):** Regular Socratic questioning, connect to prior knowledge.
- **S3 (Deepening Mastery, 20–49 sessions):** Challenge with harder variations, expect independent application.
- **S4 (Independent Learner, 50+ sessions):** Minimal scaffolding, treat as near-peer, focus on metacognition.

Layer 2: Subject Context. Domain-specific pedagogical context loaded from curriculum files.

Layer 3: Lesson Methodology. Module-specific teaching approach.

Layer 4: Teacher Persona. The named tutor's personality, tone, and questioning style.

Layer 5: Pedagogical Skills. Task-based scaffolding mapped to Bloom's taxonomy dimensions. Homework maps to Knowledge + Application. Studying maps to Knowledge only. Practice maps to Application + Analysis. Assessment maps to Analysis + Evaluation.

Layer 5.5: Programme Context. Gymnasium or university programme-specific content (described above).

Layer 6: Curriculum References. Country-specific curriculum data — Swedish LGR22, Norwegian LK20, UK KS3/GCSE/A-Level, French Baccalauréat, Indian CBSE, Nigerian WAEC/JAMB. Currently verified for six countries, with curriculum upload support for any additional country.

Layer 7: Session Parameters. Active assistance level, task type, education tier, subject, and accessibility mode — assembled as a clear parameter block at the end of the prompt.

Layer 7.5: Accessibility. Dyslexia mode (short sentences, max 15 words, bullet points, bold key terms, blank lines between ideas) and ADHD mode (max 3 paragraphs per response, one concept at a time, single clear next step, immediate positive reinforcement). The UI layer complements the prompt layer: all School pages support screen readers via ARIA labels and live regions, a reduced-motion option disables animations for students sensitive to visual movement, and high-contrast mode increases text/background ratio for visually impaired students. These UI affordances work independently of the prompt-level accessibility modes — a student can have dyslexia mode active in the AI's responses while also using high-contrast mode in the interface.

The seven-layer system illustrates a broader architectural principle: prompt composition, not prompt templates. Each layer is independently configurable, and the layers compose into the final prompt through concatenation. This means that changes to one layer — updating a curriculum reference, adjusting a teacher persona, adding a new accessibility mode — don't require changes to any other layer. A Norwegian teacher can load LK20 curriculum data (Layer 6) while keeping the same Socratic pedagogy (Layer 1b), the same growth stage scaffolding (Layer 1.5), and the same accessibility support (Layer 7.5). The layers are orthogonal, and their orthogonality makes the system both flexible and maintainable.

Bloom's Taxonomy Integration

All student progress is tracked across six cognitive dimensions from Bloom's taxonomy: Knowledge, Comprehension, Application, Analysis, Evaluation, and Creation. Each session increments the relevant dimensions by 2 points (capped at 100 per dimension), based on the task type mapping described above.

The student dashboard includes a hexagonal radar visualisation showing strength across all six dimensions. Teachers see per-dimension class analytics — average scores across all students for each dimension, allowing them to identify where the class as a whole needs more attention.

Two achievements reward Bloom's progress: "Half Way There" (reach 50% in any dimension) and "Bloom Master" (reach 100% in any dimension). These make cognitive development visible and rewarding.

In practice, the Bloom's tracking creates a feedback loop between teachers and the AI tutoring system. A teacher who notices that her class has strong Knowledge scores (86 average) but weak Analysis scores (42 average) can assign tasks specifically targeting the Analysis dimension — comparison exercises, "why does this happen?" questions, pattern-identification problems. The AI adapts its tutoring approach for these tasks, and the Bloom's scores update as students progress. Over a semester, the class profile should show balanced growth across all six dimensions. If it doesn't, the teacher has a precise diagnostic: not "the class is struggling" but "the class has mastered recall and comprehension but hasn't developed critical analysis skills."

The Growth System

XP and Levels. Every interaction earns experience points: chat turn (5 XP), assignment submitted (50 XP), perfect assignment (100 XP), streak day (20 XP), first session (25 XP), quest completed (50 XP). All XP is multiplied by the active season multiplier (1.0–2.0). Levels advance at 0, 100, 300, 600, and 1000 XP.

Streaks. Daily activity tracking with shield protection. A streak increments when the student is active on consecutive days. Missing a day normally breaks the streak — but shields absorb one break each. Students earn one shield at a 7-day streak, up to a maximum of three. This mechanic rewards consistency while forgiving occasional interruption, which is important for young learners who might miss a day due to illness or family events.

16 Achievements. From "First Step" (first session) through "Bloom Master" (100% mastery of any cognitive dimension), with milestones for streaks, levels, sessions, growth stages, and special activities. Each achievement is awarded once (UNIQUE constraint in the database) and triggers a celebratory notification.

Daily Quests. Three quests generated each day from a pool of four types: ask questions, submit assignments, review flashcards, and maintain streak. Quest selection is deterministic — seeded by a

hash of user ID + date — so the same student sees the same three quests if they check multiple times in a day.

XP Seasons. Time-limited multiplier events: Autumn Challenge (1.5x), Winter Sprint (2.0x), Spring Bloom (1.5x), Summer Quest (1.25x). These create natural rhythm and excitement around learning periods.

The Homework Workflow

The assignment lifecycle is fully digital and fully auditable:

18. A teacher creates an assignment using the Assignment Builder, setting questions, rubrics, learning objectives, and due dates. Assignments can be created from templates or from scratch.
19. The assignment can be exported as a `.anton` bundle for distribution — shared via email, learning management system, or cloud drive.
20. Students import the assignment and complete it within ANTON, with AI assistance available at the teacher's configured assistance level. The time spent on each question is tracked.
21. Students submit their work. The system records submission time, duration in minutes, and individual answers per question.
22. **AI Auto-Grade** (optional) — a streaming Claude call evaluates each answer against the rubric, producing per-question scores, strengths (3–5 bullet points), areas for improvement (3–5 bullet points), a suggested grade (A–F or percentage), and a Learning Evidence Log mapping skills demonstrated to Bloom's taxonomy dimensions.
23. The teacher reviews using the Submission Reviewer — a dedicated interface showing the student's answers alongside the AI grade, learning evidence, and a form for the teacher's own grade and written feedback. The teacher can accept the AI grade, override it, or grade manually.
24. XP is awarded for submission (50 XP) and quality (100 XP for perfect scores), feeding into the growth system.

Guardian and Teacher Oversight

Guardians receive weekly progress digests showing their child's XP, streaks, achievements, and recent activity. They see progress, not transcripts — a deliberate privacy design. They link to their child's account via invite codes, not email addresses. The digest can be auto-sent via email, including a personalised encouragement message and links to the parent dashboard.

Teachers manage classes, student rosters, and assignments. They see per-Bloom-dimension class analytics. A dedicated oversight page enables content flagging and human sign-off for publishing student work. Teachers control which AI model tier is used — Claude (Tier A, most capable), Sonnet (Tier B, efficient), or local Ollama (Tier C, free and private) — trading capability for cost and privacy.

Safety in an Educational AI System

Safety in an educational AI system is fundamentally different from professional use. A compliance officer can evaluate whether AI output is appropriate. A seven-year-old cannot.

Age-appropriate content enforcement operates at the prompt level, not the filter level. The T1 override layer (Layer 1 in the school prompt system) constrains the AI's vocabulary, topic range, and response structure before generation, rather than filtering output after generation. This is a deliberate design choice: post-generation filtering can be bypassed by creative prompting, but prompt-level constraints shape what the model produces in the first place. The T1 layer prohibits violence, politics, religion, and adult themes — not as blocked keywords, but as excluded topic domains in the system prompt.

Teacher oversight provides content flagging and a human sign-off workflow for publishing student work. Data minimisation is enforced architecturally: student data stays on the instance, guardians see progress but not transcripts, and a GDPR DELETE endpoint wipes learning history completely. No student data is sent to any external service except the chosen LLM provider during active sessions — and even that can be avoided by using local Ollama models.

The safety architecture was designed with input from educators, not just engineers. The principle is that an AI tutor must be at least as safe as a human tutor — and in some respects safer, because it never loses patience, never shows favouritism, and never forgets the age-appropriate boundaries regardless of how creative the student's prompting becomes.

Humanitarian Deployment

School Mode runs fully on local open-source models via Ollama (Tier C), with Mistral 7B as the default. No API key is needed. No data leaves the machine. After the initial model download, everything works offline.

This is not a "lite version." The same seven-layer architecture, the same pedagogical framework, the same assessment system, and the same growth mechanics run on local models. The `buildSchoolPrompt()` function adapts for local model constraints — simpler prompt structures, shorter context windows — but the educational structure is identical.

Client-side persistence via IndexedDB (using Dexie.js) ensures that students can continue working even when connectivity is intermittent. An Offline Banner component communicates connectivity status. Thirty-six language locales are available, including right-to-left support for Arabic and Urdu. Curricula from six countries are verified: Sweden, Norway, UK, France, India, and Nigeria. The curriculum upload system accepts teacher-provided PDF curricula from any country, which Claude analyses to generate structured study plans — extending support to any educational system without requiring ANTON-side curriculum development.

The deployment economics are significant. A Mistral 7B model requires approximately 4GB of disk space and runs on hardware as modest as a Raspberry Pi 4 (with patience) or a mid-range laptop (with reasonable performance). After the initial model download, the operational cost is zero — no API fees, no subscriptions, no per-student licensing. For an NGO deploying educational technology across 50 schools, the difference between "free after hardware" and "$0.01 per query" compounds quickly. Free enables experimentation, iteration, and universal access in a way that even very low per-unit costs do not.

The design goal is specific: a school in rural East Africa with intermittent connectivity and no budget for AI API keys can run the same educational platform — with the same pedagogical rigour, the same gamification, the same assessment framework — as a well-resourced school in Stockholm. The models are less capable. The educational architecture is identical.

This is not charity framing. It is an architectural commitment. The same offline capability that serves a school in rural Kenya also serves a European financial institution that requires air-gapped deployment for security reasons. The same local-model support that enables zero-cost education also enables

data-sovereign professional use. Building for the most constrained deployment makes the platform more robust for every deployment.

A Day in School Mode

The student: Elsa, 16, Naturvetenskap programme, T3. It's Wednesday afternoon. Elsa opens ANTON School and sees her dashboard: Level 3 (420 XP), 12-day streak, two daily quests remaining ("Submit an assignment" and "Review 5 flashcards"). Her Bloom's radar shows strong Knowledge and Comprehension but weaker Analysis and Evaluation — she can recall and explain, but struggles with deeper critical thinking.

She opens Mathematics with Alma. Her growth stage is S3 (Deepening Mastery, 28 sessions) and assistance level L2 (Moderate Help). She's working through a problem set on quadratic functions. Alma doesn't give answers — she asks: "You found that x = 3 is one solution. What happens when you substitute x = 3 back into the original equation? Does it balance?" When Elsa gets stuck on the factoring step, Alma shows a worked example on a similar (not identical) problem and asks Elsa to apply the same technique. If Elsa repeatedly confuses the discriminant formula across sessions — a persistent misconception — the knowledge atom system captures this as a recurring pattern. On subsequent sessions, retrieved atoms about Elsa's discriminant confusion inform the AI's approach: Alma addresses the misconception proactively rather than waiting for the error to recur, using targeted counter-examples and visual explanations specific to the misunderstanding. The session increments her Application and Analysis Bloom dimensions by 2 points each.

The teacher: Mikael, gymnasium mathematics, Stockholm. Mikael opens his teacher dashboard. He sees his class of 24 students: average Bloom scores across all six dimensions, recent activity rates, and three flagged items requiring his attention. One student hasn't logged in for 5 days (streak broken). One student's Comprehension dimension has plateaued despite 15 sessions — Mikael makes a note to check whether the assistance level should be adjusted. A third student submitted an assignment that the AI auto-grade flagged as potentially containing copied content — Mikael reviews the submission alongside the AI assessment and confirms it's original work, just heavily influenced by the textbook examples.

Mikael creates a new assignment for next week using the Assignment Builder: 5 questions on derivatives, rubric mapped to Application and Analysis dimensions, due Friday, assistance level L1 (Socratic only — no worked examples). He exports it as a `.anton` bundle and uploads it to the school's LMS.

The key observation: Mikael spent 12 minutes on class management. The AI handled 24 simultaneous tutoring sessions, each adapted to the individual student's tier, subject, growth stage, assistance level, and Bloom dimensions. Mikael's time was spent on the things that require a human teacher: judgement calls about student progress, content moderation, and assessment design. The AI handled the things that benefit from infinite patience: Socratic questioning, worked examples, immediate feedback, and progress tracking across every interaction.

23. Life: Personal Intelligence

The Life pillar provides four tools that serve the person behind the professional. Each tool is built on the same architectural patterns as the professional workspace — the same Express route factory, the same Zustand state management, the same streaming SSE infrastructure, the same dark theme design system — but adapted for personal rather than professional use cases.

News Intelligence

A multi-source news analysis system with fifteen curated sources — Reuters, BBC, SVT Nyheter, Dagens Nyheter, Svenska Dagbladet, NRK, The Guardian, Associated Press, The Economist, Financial Times, Bloomberg, TechCrunch, Wired, Nature, and others — each rated for factuality (0–100) and political spectrum position (far_left through far_right on a seven-point scale).

Bias analysis examines any story across sources, mapping the political distribution of coverage and identifying where the user's reading habits might create blind spots. The system tracks which bias categories the user reads most frequently and presents a personal bias profile: "You've been reading primarily centre-right sources. Here are centre-left perspectives on the same stories you've been following."

Truth verification analyses specific claims with verdicts ranging from "true" through "mostly true," "mixed," "mostly false," "false," to "unverifiable." Each verdict includes a confidence score, explanation, and list of sources checked. The model used for fact-checking is Claude Sonnet — chosen for its balance of analytical depth and cost efficiency.

Story explainer generates a streaming, neutral-angle explanation of any story, explicitly noting where different sources diverge in their framing or emphasis.

The design philosophy is transparency, not curation. ANTON doesn't decide what news the user should read. It shows the user what they're reading and what they might be missing.

The bias analysis system deserves elaboration, because it represents a design philosophy that goes against the industry grain. Most news platforms either curate (choosing what you see) or personalise (showing you more of what you already read). Both approaches assume that the platform knows better than the user. ANTON's approach is different: show the user their own reading patterns and let them decide whether to adjust.

The personal bias profile works by categorising every source the user reads on a seven-point political spectrum (far_left through far_right) and tracking the distribution over time. If 70% of a user's reading comes from centre-right sources, the system doesn't hide that reading or penalise those sources. It shows the distribution as a chart and suggests: "Here are three stories from centre-left and left sources on topics you've been following." The user decides whether to read them. The system never withholds, filters, or manipulates. It informs.

This is a fundamentally different model of media literacy. Rather than an algorithm deciding what information the user should see, the system provides transparency about what information the user has been choosing — and expands the range of what they can easily access. For professionals who need to understand multiple perspectives on regulatory developments and market trends, this kind of self-aware media consumption is a professional skill, not just a personal preference.

Financial Literacy

An education-first financial toolkit with ten learning topics (home economics through decision frameworks), five calculators, and a goal-planning system. The calculators are pure logic — no AI involved, no API calls, no potential for hallucination:

Calculator	Inputs	Outputs
Mortgage	Home price, down payment, interest rate, loan years	Monthly payment, total interest, total cost, amortisation schedule
Compound Interest	Principal, annual rate, years, compounding frequency	Final amount, total interest earned
Swedish Pension	Current age, retirement age, savings, monthly contribution, return rate	Projected balance, retirement income (4% rule)
Debt Payoff	Total debt, interest rate, monthly payment	Months to payoff, total interest, payoff schedule
Swedish Tax	Annual gross salary	National + regional taxes, net income, effective rate

The Swedish tax calculator uses current-year thresholds including grundavdrag, kommunalskatt, statlig skatt, and jobbskatteavdrag. ISK accounts, tjänstepension, amorteringskrav, and other Swedish-specific financial concepts are contextualised throughout.

Every financial output includes a mandatory disclaimer: "Educational only — not financial advice. Consult a qualified financial advisor for personal decisions." This appears in the system prompt, not just the UI — the AI itself is instructed to include the disclaimer in any financial guidance it provides.

The financial literacy module illustrates a design principle that runs through all Life features: where pure logic suffices, don't involve AI. The calculators produce mathematically precise results without hallucination risk, API cost, or latency. AI is reserved for situations where reasoning, explanation, and contextualisation add genuine value — financial concept explanations, personalised goal planning guidance, and situational advice. This principled separation between computation and reasoning keeps the feature both reliable and cost-effective.

Travel Planning

AI-powered trip planning with day-by-day itinerary generation (morning, afternoon, and evening slots at three budget levels), AI-generated country guides covering culture, safety, visas, currency, language, transport, food, scam alerts, and seasonal recommendations, and a destination quiz that matches user preferences to recommendations from twenty-plus destinations.

Country guides are generated by Claude and cached in the database — a guide for Thailand, once generated, is served from cache rather than regenerated. The guides assume a Swedish traveller context (Schengen zone awareness, Swedish passport visa requirements).

The caching strategy is worth noting because it reflects a broader principle: expensive AI operations should be cached when the output is stable. A country guide for Thailand doesn't change between users — the cultural norms, visa requirements, and scam alerts are the same regardless of who's asking. Generating it once and caching it reduces cost by 100% for subsequent requests while maintaining the quality of an AI-generated, locally-aware guide. This principle — generate once, cache aggressively, regenerate on explicit request — applies throughout ANTON where content is user-independent.

Community

A privacy-first communication system, disabled by default. Users explicitly activate it and generate a cryptographic identity: an Ed25519 keypair created on the server, with the private key encrypted and stored locally in the browser's localStorage. The server never sees the unencrypted private key after initial generation.

The contact hash — a SHA-256 hash of the public key, displayed as `ANTON-XXXX-XXXX-XXXX-XXXX` (four groups of four hex characters) — serves as the user's identity in the community system. The `ANTON-` prefix makes hashes immediately recognizable as ANTON identities. Users share their contact hash to receive messages. Only the public key is visible to the server; the private key stays on the user's device.

This identity model is deliberately different from every mainstream communication platform. There are no email addresses, no phone numbers, no social media profiles, no "real name" requirements. Your community identity is a cryptographic hash that reveals nothing about who you are. If someone has your contact hash, they can send you a message. If they don't, they can't find you. There is no user directory, no search-by-name, no "people you may know." This is privacy by design, not privacy by policy — the information that could identify you doesn't exist in the system.

The trade-off is deliberate: the system prioritises privacy over convenience. You can't look up a colleague by name. You need their contact hash, which they share directly. For professional communities that handle sensitive information — compliance teams, legal practices, healthcare groups — this trade-off is appropriate. For social communities that prioritise discovery, it would be limiting. ANTON serves the former.

Community features include groups with invite codes (six-character alphanumeric tokens), threaded mail with six folder types (inbox, sent, drafts, starred, archive, trash), shared events with RSVP and RFC 5545-compliant iCalendar export, and a chronological forum.

The forum deliberately uses no algorithmic sorting, no recommendation engine, no engagement optimization. Posts appear newest-first or sorted by upvotes in the last 24 hours ("hot") — the user chooses which view. There is no infinite scroll — pagination only. No personalisation algorithm decides what you see. This is a conscious design choice: community should organise itself, not be organised by an algorithm optimising for engagement metrics.

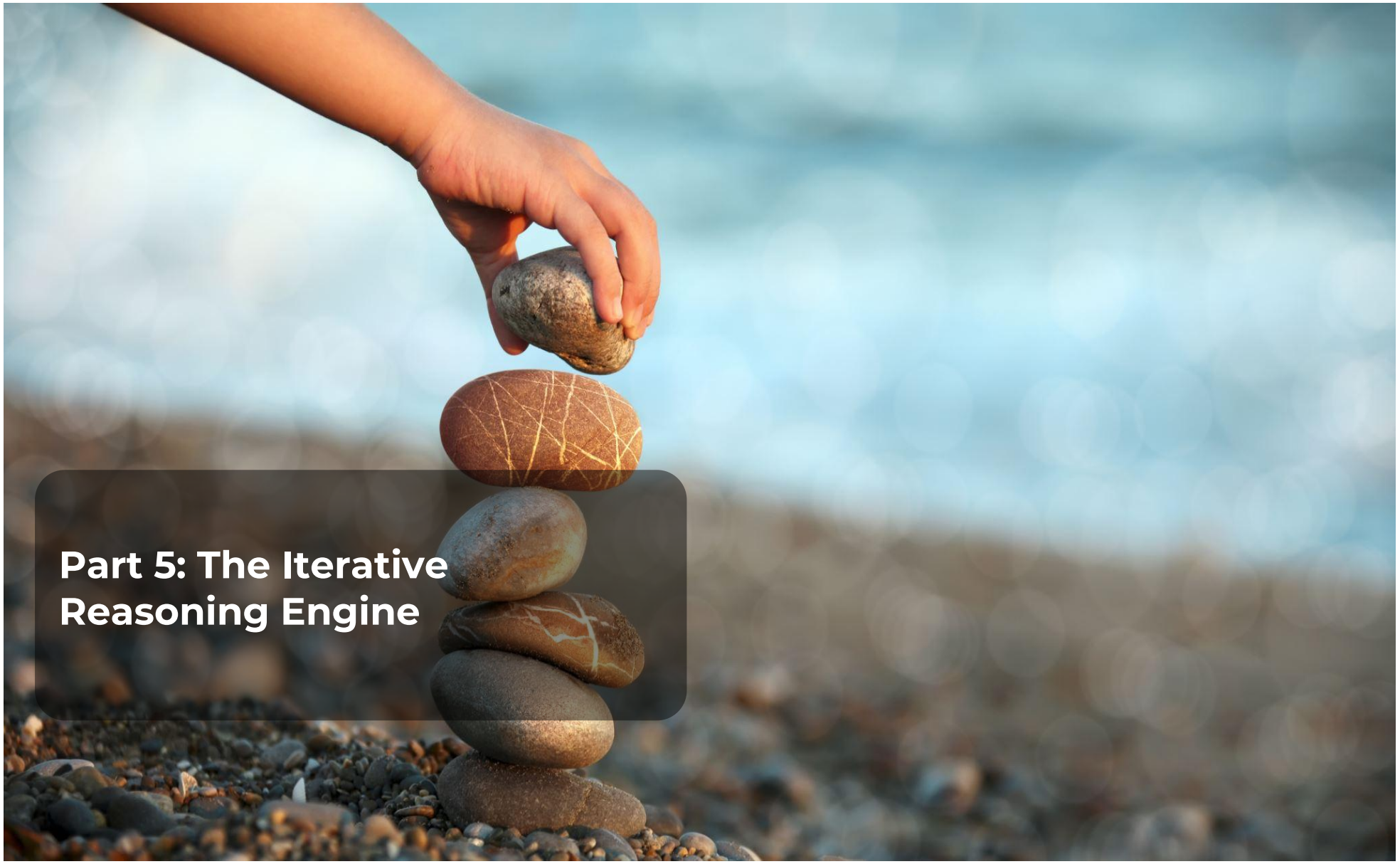
Real-time notifications use Socket.IO with a `/community` namespace. Each user has a personal room (`user:<hashCode>`) for inbox notifications. Groups and forums have their own rooms for live updates.

The decision to use no algorithmic sorting in the forum is perhaps the most counter-intuitive design choice in ANTON. Every major platform sorts content algorithmically — by engagement, by relevance, by predicted interest. These algorithms are remarkably effective at maximising time-on-platform. They are also remarkably effective at amplifying outrage, creating filter bubbles, and eroding discourse quality.

ANTON's community forum offers exactly two sorting options: newest-first (chronological) and hot (most upvoted in the last 24 hours). There is no recommendation engine, no "for you" feed, no engagement optimization. Posts from new members appear alongside posts from active contributors. Quiet, thoughtful posts get the same visibility as provocative ones.

This is a deliberate bet that professional communities can organise themselves. A compliance officer's forum doesn't need an algorithm to surface interesting discussions — the members know what interests them. A teacher's community doesn't need engagement metrics to determine which posts matter — professional relevance is its own signal. The absence of algorithmic amplification means the absence of algorithmic distortion.

Pagination (not infinite scroll) is part of the same philosophy. Infinite scroll is an attention trap designed to maximise session duration. Pagination creates natural stopping points that respect the user's time. For professionals who are reading a community forum between engagements, the ability to check "what's new since yesterday" without being pulled into an infinite feed is a feature, not a limitation.



Part 5: The Iterative Reasoning Engine

24. Structured Professional Reasoning

Not all questions deserve the same depth of thought. A quick briefing needs fast, focused output. A regulatory gap analysis needs deep, multi-phase reasoning with self-critique. A board-level risk assessment needs exhaustive analysis that examines the question from multiple angles before committing to a conclusion.

This is not a novel observation. It is how senior professionals already work. A partner at a compliance consultancy doesn't spend the same amount of time on every question. A routine client inquiry gets a quick, confident response. A complex regulatory interpretation gets researched, drafted, reviewed by a colleague, revised, and reviewed again. The depth of analysis scales with the stakes of the question and the complexity of the domain.

The Iterative Reasoning Engine makes this professional discipline explicit and systematic. Rather than relying on the AI model to self-regulate its reasoning depth (which models do inconsistently), the engine imposes structure: defined phases, explicit self-critique, confidence extraction, and auditable reasoning chains.

The Iterative Reasoning Engine implements this gradient through four depth levels, each adding more phases of structured thought:

`think_hard` runs 2 phases: analyse and synthesise. The analysis phase uses Claude with adaptive thinking at effort level "high" (up to 16,000 tokens of internal reasoning). The synthesis phase uses effort "max" (up to 24,000 tokens). Total thinking investment: approximately 40,000 tokens. Used for complex module tasks that need more than a single-pass response but don't require deep self-critique.

`investigate` runs 4 phases: analyse, reflect, deepen, and synthesise. The reflection phase is the key addition — it includes explicit self-critique instructions: "What did the analysis miss? What assumptions were not challenged? What alternative interpretations were not considered?" The deepen phase follows up on whatever the reflection identified as gaps. Total thinking: approximately 76,000 tokens. Used for gap analyses, risk assessments, and regulatory interpretations.

`plan_first` runs 4 phases: analyse, plan, deepen, and synthesise. Similar to investigate, but the second phase focuses on creating a structured plan of attack rather than critiquing the first phase. Total thinking: approximately 76,000 tokens. Used for strategic planning tasks where the approach matters as much as the analysis.

`deep_investigate` runs 6 phases: analyse, reflect, deepen, explore, validate, and synthesise. The explore phase examines alternative hypotheses and edge cases. The validate phase checks the accumulated reasoning for internal consistency and evidential support. Total thinking: approximately 144,000 tokens. Reserved for the Orchestrator's highest-stakes autonomous decisions — because when the AI is acting without human approval, the reasoning must be exhaustive.

How Phases Build on Each Other

Each phase runs as a non-streaming AI call. The output of each phase becomes "PRIOR PHASE CONTEXT" for the next phase. The reflection phase can reference specific claims from the analysis phase and challenge them. The deepen phase can follow up on specific gaps identified by the reflection phase. This creates a genuine reasoning chain where each phase builds on, critiques, and refines the work of the previous phases.

Prompt caching reduces costs by approximately 60% — the static system prompt and module expertise (Layers 1–3 of the prompt builder) are cached and reused across phases. Only the dynamic content changes between phases.

Confidence Extraction

Phases with the `revision_needed` flag — reflect, deepen, explore, validate — use Claude's tool-use capability to extract structured metadata:

- **Confidence score** (0.0–1.0) — how confident is the AI in its current conclusions?
- **Revision needed** (boolean) — does the reasoning need further refinement?
- **Next action** — what should the next phase focus on?

These scores are stored per step in the `revelation_steps` table, creating a confidence trajectory across the reasoning chain. A chain where confidence increases from 0.6 to 0.85 across four phases tells a different story than one where confidence stays flat at 0.5 — and the professional can see this trajectory.

Revelation Chains

Every multi-phase reasoning run is persisted as a **Revelation Chain**:

```
CREATE TABLE revelation_chains (
  id TEXT PRIMARY KEY,
  session_id TEXT,
  thinking_level TEXT NOT NULL,           -- 'think_hard', 'investigate', etc.
  phase_count INTEGER DEFAULT 0,
  total_input_tokens INTEGER DEFAULT 0,
  total_output_tokens INTEGER DEFAULT 0,
  total_duration_ms INTEGER DEFAULT 0,
  synthesis_quality_score REAL,
  created_at TEXT DEFAULT (datetime('now'))
);

CREATE TABLE revelation_steps (
  id TEXT PRIMARY KEY,
  chain_id TEXT REFERENCES revelation_chains(id) ON DELETE CASCADE,
  phase_index INTEGER NOT NULL,
  phase_name TEXT NOT NULL,
  thinking_content TEXT DEFAULT '',
  output_content TEXT DEFAULT '',
  confidence_score REAL,
  revision_needed INTEGER,
  next_action TEXT,
  input_tokens INTEGER DEFAULT 0,
  output_tokens INTEGER DEFAULT 0,
  duration_ms INTEGER DEFAULT 0,
  created_at TEXT DEFAULT (datetime('now'))
);
```

The UI presents these as phase cards with confidence bars, expandable thinking content, and timing information. A professional reviewing a deep_investigate chain can see exactly how the AI's reasoning evolved across six phases — what was challenged in reflection, what was discovered in exploration, what was validated or invalidated in validation. Each phase card shows the input and output token counts, the duration, and the confidence score — creating a complete performance profile for the reasoning process.

This provides the same auditability as the Orchestrator's Reasoning Trails (Section 17), applied to every substantial piece of analysis the system produces. When a regulator asks "how deep was the analysis that produced this recommendation?", the revelation chain shows the answer — not just the final output, but the full reasoning journey that produced it. The confidence trajectory is particularly valuable: a chain where confidence increases from 0.6 to 0.85 across four phases tells a different story than one where confidence stays flat at 0.5. The former suggests that deeper analysis resolved initial uncertainty. The latter suggests that additional reasoning didn't improve clarity — which itself is useful information, indicating that the question may be genuinely ambiguous or that additional evidence is needed.

Default Depth Mappings

Every module has a default thinking level based on its typical complexity and stakes:

Module Type	Default Level	Phases	Rationale
gap-analysis	investigate	4	Regulatory work needs self-critique
risk-assessment	investigate	4	Risk evaluation needs multi-angle analysis
document-creation	think_hard	2	Document drafting benefits from depth but not exhaustive reasoning
training-content	think	1	Training material needs clarity more than depth
investigation-support	investigate	4	Investigation needs thorough evidence evaluation

Users can override per session. The Orchestrator uses `deep_investigate` for autonomous decisions — because when the AI is acting without human approval, the reasoning must be exhaustive. This is a deliberate safety design: more autonomy requires more reasoning, not less.

Walkthrough: An `investigate`-Level Gap Analysis

Here is what happens when a compliance officer selects the `investigate` depth level for a gap analysis of a client's AMLR Article 15 compliance:

Phase 1: Analyse (effort: "high", ~20,000 max output tokens). The AI receives the full prompt — foundation layer, org context, activated AMLR knowledge pack, gap analysis module prompt, retrieved knowledge atoms, the Article 15 requirements, and the client's uploaded current methodology document. It produces a structured initial assessment: 12 requirements identified, 8 rated as gaps or partial compliance, 4 rated as compliant. Total time: ~15 seconds.

Phase 2: Reflect (effort: "high", ~16,000 max output tokens). The reflection prompt includes the full initial assessment as "PRIOR PHASE CONTEXT" plus explicit instructions: "What did the analysis miss? What assumptions were not challenged? What alternative interpretations were not considered?" The AI identifies three issues: one requirement was assessed against the 2022 version of the article rather than the 2024 revision, one "compliant" rating assumed the client's quarterly review process was documented when the evidence is ambiguous, and the analysis didn't consider cross-border implications for the client's Finnish subsidiary. Total time: ~12 seconds.

Phase 3: Deepen (effort: "max", ~20,000 max output tokens). The deepen prompt focuses on the three gaps identified in reflection. The AI reassesses the outdated-version requirement against the 2024 text, downgrades the ambiguous compliance rating to "partial," and adds a new section on Finnish jurisdictional considerations. Confidence score extracted via tool use: 0.78, up from an implicit ~0.65 after Phase 1. Total time: ~18 seconds.

Phase 4: Synthesise (effort: "max", ~32,000 max output tokens, streamed). The final phase takes all prior context and produces the deliverable — a complete gap assessment report in the user's selected output format. Unlike the previous phases, this one streams directly to the user via SSE. The synthesis includes a confidence note: "This assessment was revised through multi-phase analysis. Key revisions: Article 15(3)(b) re-assessed against 2024 text, documentation adequacy downgraded based on evidence ambiguity, Finnish subsidiary considerations added." Total time: ~25 seconds.

Total: 4 phases, ~88,000 max output tokens budget, ~70 seconds, estimated cost: $5–7 at Claude Opus pricing.

The same analysis at `think` level (single phase) would take ~15 seconds and cost ~$1.50 — but it would miss the outdated-version error, the ambiguous compliance rating, and the cross-border implications. The multi-phase approach caught three significant issues that a single pass missed. For a gap analysis that will inform a board decision and potentially a regulatory filing, the additional cost and 55 seconds are not meaningful overhead. For a routine training content update, they would be unnecessary.

Cost-Benefit by Module Type

This explains the default depth mappings:

Module	Default	Phases	Rationale
gap-analysis	investigate	4	Regulatory accuracy is paramount; errors have compliance consequences
risk-assessment	investigate	4	Risk evaluation affects business decisions;

			multi-angle analysis reduces blind spots
document-creation	think_hard	2	Documents benefit from structured reasoning but don't need self-critique
training-content	think	1	Clarity matters more than depth; training material should be accessible, not exhaustive
investigation-support	investigate	4	Investigations need thorough evidence evaluation and counter-hypothesis testing

The reasoning investment matches the stakes. A gap analysis where three significant issues were caught by reflection and deepening phases justifies the multi-phase cost. A training module where clarity and accessibility matter more than exhaustive self-critique does not. The defaults are professional best guesses, not rigid rules — users can always override per session.

Part 6: Platform Scale

25. Growth Since Part 1

Metric	Part 1	Part 2 (v0.7.5)	Growth
Expert areas	29	57	+97%
Modules	~100	492 across 57 areas	+392%
System prompts	~15	84	+460%
Output formats	~30	45	+50%
Database tables	~40	183 (PostgreSQL)	+358%
Regulatory frameworks	4	19	+375%
Knowledge packs	0	29+	New
Frontend pages	~30	197	+557%
API endpoints	~40	200+	+400%
LLM providers	4	6	+50%
Python computation templates	—	40	New
Languages	1	36+	+3,500%

These numbers tell a story, but the story isn't about size.

183 database tables means the platform has moved from a simple session store to a comprehensive professional intelligence infrastructure. The original ~40 tables handled sessions, users, and output. The additional tables handle knowledge atoms, embeddings, entity graphs, pattern detection, orchestrator trails, trust progression, quality scoring, workflow triggers, school gamification, community identity, markets intelligence, talent discovery, specialized agents, and many other subsystems. Each table represents a specific architectural commitment — a decision that this kind of data matters enough to structure, store, and query. ANTON now supports full dual-database architecture: PostgreSQL is the primary database for production use, with a complete schema of 183 CREATE TABLE statements and 64 PostgreSQL-specific migrations. SQLite remains as a lightweight option for single-user development, operating in WAL (Write-Ahead Logging) mode. The PostgreSQL adapter at `server/db/adapters/postgresql-adapter.ts` auto-detects the database type and converts parameterized queries from `?` placeholders to `$1,$2,...` format automatically. PostgreSQL 16+ is required for production deployments.

200+ API endpoints means that every piece of functionality is accessible programmatically. This matters for automation (workflows can chain endpoints), integration (external systems can interact with ANTON via REST), and testability (every endpoint can be validated independently).

36+ languages means that a compliance team in Dubai, a school in Lagos, a legal practice in São Paulo, and an NGO in Dhaka can all use the same platform in their preferred language, with right-to-left support for Arabic, Urdu, and Hebrew. Internationalisation at this scale is a commitment to serving professionals globally, not just in English-speaking markets.

57 expert areas spanning financial crime prevention, healthcare, legal, education, PE/VC, blockchain, creative production, NGO, and more means that the APCI layer accumulates knowledge across genuinely diverse professional domains. Cross-domain patterns — a healthcare compliance finding that parallels a financial regulation gap — become detectable precisely because the platform is broad enough to see connections that domain-specific tools cannot.

This breadth creates an underappreciated form of intelligence. A consultancy that advises both pharmaceutical companies (healthcare area) and banks (FCP area) will accumulate knowledge atoms from both domains. If DORA's ICT risk requirements for financial institutions share structural similarities with the EU MDR's cybersecurity requirements for medical devices — which they do — the entity convergence detector can surface that parallel. No single-domain tool sees this. ANTON sees it because it operates across domains within a single knowledge graph.

But the growth that matters most is not in any of these metrics. It's in the APCI layer that connects everything together. Each new module, each new area, each new session contributes to the same knowledge base, the same entity graph, the same pattern detection system. The platform doesn't just get wider. It gets deeper.

26. The Prompt Layer as Product

ANTON contains 39 system prompts averaging 103 lines each — over 4,000 lines of encoded professional expertise. Each prompt is a distilled representation of domain knowledge: how gap analyses should be structured for the specific regulator who will read them, what citation standards legal research requires, what risk dimensions a business-wide risk assessment must cover, what pedagogical approach works for a twelve-year-old learning algebra versus a university student studying thermodynamics.

Consider what's in a single system prompt — the gap analysis module's `gap-analysis.md` (106 lines). It includes a 15-theme AMLR regulatory framework (from Customer Due Diligence through Emerging Technology & Crypto-Asset Obligations), 6 gap categorisation types (Absence, Partial, Superseded, Conflicting, Undocumented, Quality Deficiency), 5 severity levels (Critical through Compliant), and a 5-tier remediation effort scale (XS: under 1 week through XL: 6+ months). This isn't boilerplate. It's the distilled judgement of what makes a gap analysis useful to the person who will read it and act on it. A compliance officer using Claude directly — without this prompt — gets generic analysis. With this prompt, they get output structured the way their regulator expects to receive it.

Combined with the seven-plus-layer prompt architecture and 45 structured output formats, the prompt layer represents a thesis: **the prompt is the product.** Not the model. Not the chat interface. The encoded expertise that turns a general-purpose AI into a domain-specific professional tool.

The prompt layer is also what makes the first session valuable. The APCI layer makes the hundredth session better than the first. But the prompt layer ensures that even the first session produces professional-grade output — and that first-session value is what justifies the investment in building the context that follows.

This is why ANTON is open source. These prompts — and the architecture that assembles them — are the kind of infrastructure that professional communities should be able to inspect, critique, and improve. A compliance officer should be able to read the gap analysis prompt and verify that it reflects current regulatory expectations. A teacher should be able to review the pedagogical prompt and confirm that it aligns with their curriculum standards. An AI safety researcher should be able to examine the Orchestrator's reasoning prompts and verify that the safety constraints are sound.

The prompt layer is also why switching AI providers doesn't diminish ANTON's value. The prompts work with Claude, with GPT, with Gemini, with Mistral, with local Ollama models. The model provides the intelligence. The prompts provide the professional training. The APCI layer provides the accumulated context. Together, they produce professional-grade output that no model alone can match.

27. The .anton Bundle Format

Knowledge in ANTON is not just stored — it's portable. The `.anton` bundle format is a ZIP archive with a standardised structure that enables knowledge to move between installations, teams, and communities.

A bundle contains:

- A `manifest.json` with metadata: name, version, author, description, regulatory domain, entity count, and compatibility information
- `entities.json` — structured entities (regulatory articles, concepts, obligations)
- `relationships.json` — typed relationships between entities (cites, contradicts, extends, clarifies)
- `aliases.json` — alternative names and acronyms for entities (e.g., "AMLR" = "Anti-Money Laundering Regulation" = "Regulation 2024/1624")

Bundle types serve different purposes:

- **`regulatory-knowledge-pack`** — structured regulatory knowledge that enriches the APCI layer when activated
- **`assignment`** — School Mode homework bundles created by teachers, importable by students
- **`module-template`** — reusable module configurations with custom system prompts
- **`engagement-export`** — engagement results packaged for sharing with clients or colleagues

The build tool — `node data/knowledge-packs/build-pack.mjs <directory>` — creates bundles from structured directories. The import pipeline validates bundle integrity, enforces size limits (20MB), checks entity count bounds (5,000 maximum), and validates field lengths before activation.

The format is deliberately simple. A `.anton` file is a ZIP you can inspect with any archive tool. The JSON files inside are human-readable. There is no proprietary encoding, no binary format, no obfuscation. This transparency is essential for regulated environments where professionals need to verify what knowledge they're importing before activating it.

The validation pipeline enforces safety boundaries: maximum bundle size of 20MB, maximum 5,000 entities per pack, field length limits of 2,000 characters, and mandatory manifest metadata. These limits are not configurable — they are structural guards against malformed or malicious bundles. A knowledge pack from an untrusted source goes through the same validation as one from a trusted colleague.

Adversarial Robustness

A question worth addressing directly: could a malicious `.anton` bundle corrupt the knowledge base in ways that influence future retrievals? The threat is real in principle — a bundle containing deliberately misleading regulatory entities or relationships could, once activated, inject false knowledge into the APCI layer. Three defences mitigate this. First, the validation pipeline rejects bundles that exceed structural limits, but it does not (and cannot) verify factual accuracy — that responsibility lies with the professional who activates the pack. Second, the retrieval feedback loop is self-correcting: if atoms from a malicious pack produce poor results, professionals will rate them negatively, and the feedback boost will suppress them in future retrievals. Third, knowledge packs can be deactivated with a single click, immediately removing their entities from the prompt layer. The overall posture is: validate structure automatically, delegate factual trust to the professional, and make deactivation instant when trust is violated.

The strategic vision is a professional commons: compliance consultants sharing regulatory knowledge packs, teachers sharing curriculum-aligned assignment bundles, legal teams sharing citation templates. Not a marketplace controlled by a platform operator, but an open format that any organisation can use to package and distribute professional knowledge. The export/import infrastructure exists today. Community discovery — search, ratings, reviews — is planned but not yet built.

Part 7: Competitive Position

28. APCI vs AGI

The AI industry is in a race toward AGI — Artificial General Intelligence. The assumption is that a sufficiently intelligent model will eventually handle any task. The investment is enormous. The progress is genuine.

But for the professionals ANTON serves — compliance officers, consultants, lawyers, teachers, analysts — the question is not "how smart is the AI?" The question is "does it get better at my specific work over time, with evidence I can verify and governance I can trust?"

AGI asks: how do we build AI that can do anything? APCI asks: how do we build AI that gets genuinely better at the specific professional work you need it to do — with evidence, governance, and earned trust?

These questions are complementary, not competing. Better general intelligence makes APCI more capable — a smarter model extracts better knowledge atoms, detects subtler relationships, generates more insightful proposals. And better professional context makes any level of intelligence more effective — even a modest model produces superior output when informed by thousands of professionally-rated knowledge atoms and years of accumulated entity relationships.

The complementarity runs deeper than utility. AGI research advances the frontier of what AI can understand and reason about. APCI research advances the frontier of what AI can remember, apply, and improve from professional use. These are different research problems with different solutions. AGI needs better architectures, training data, and compute. APCI needs better knowledge structures, retrieval systems, and feedback mechanisms. Progress in either direction benefits the other.

But the strategic implications differ fundamentally. AGI investment is concentrated in a handful of well-funded labs. The resulting intelligence is available to everyone via API. No organisation can build a durable competitive advantage on model access alone, because the same API is available to their competitors.

APCI investment is distributed. It happens inside every organisation that uses the platform, one session at a time, one feedback rating at a time, one entity relationship at a time. The resulting context is unique to each organisation. After two years of APCI accumulation, your compliance team's knowledge base — its atoms, relationships, patterns, quality baselines, and earned trust progressions — cannot be replicated by a competitor who subscribes to the same model API. That accumulated context is a genuine strategic asset, precisely because it reflects your specific work, your specific standards, and your specific professional judgement.

The fundamental insight is this: intelligence without context is potential. Intelligence with accumulated professional context is performance. And performance, in professional services, is what clients pay for, what regulators evaluate, and what careers are built on.

29. Differentiation

Platform	Approach	ANTON's Distinction
Harvey	Legal AI, ~$1,200/lawyer/month	ANTON covers 57 areas, is open source, runs locally, costs a fraction
Claude Projects / Custom GPTs	Static context (uploaded docs, instructions)	APCI builds dynamic context from session output — atoms, patterns, feedback
Notion AI / Mem.ai	Workspace AI with document context	No knowledge extraction, no entity graphs, no cross-session learning loop
Cursor Automations	Code-focused automation	ANTON combines event triggers with 57 expert areas and full orchestration
Copilot / ChatGPT Teams	General-purpose chat	No learning loop, no domain-specific quality ratchet, no earned autonomy
Glean / Moveworks	Enterprise search	ANTON is local-first, open source, with no cloud dependency

Each row deserves context.

"Why not just use Harvey?" Harvey is exceptional legal AI — purpose-built, deeply integrated with legal workflows, and used by major law firms globally. But at approximately $1,200 per lawyer per month, it serves a specific market segment: large law firms with substantial AI budgets. ANTON serves a different segment: organisations that need domain expertise across multiple professional areas (not just legal), want to run locally for data sovereignty, and prefer open-source infrastructure they can inspect and modify. A mid-sized compliance consultancy that needs gap analysis, risk assessment, legal research, training content, AND regulatory monitoring doesn't want five separate AI subscriptions. ANTON provides all of these from a single, self-hosted platform.

"Why not just use ChatGPT Teams or Copilot?" These are excellent general-purpose tools. The problem is precisely that they're general-purpose. Every session starts fresh. There is no knowledge accumulation across sessions, no entity tracking across engagements, no quality measurement over time, no earned autonomy model. For a quick brainstorming session or a one-off document draft, they're often sufficient. For sustained professional work where context matters — where you need the AI to know what your team found last quarter, what your organisation's risk appetite is, and whether its output quality is trending up or down — general-purpose chat lacks the infrastructure to deliver.

"Why not use Glean or Moveworks for enterprise knowledge?" These platforms excel at enterprise search across existing documents and systems. They index what already exists. ANTON generates and structures new knowledge from AI-assisted professional work. The use cases are complementary, not competing: Glean finds the policy document your team wrote last year; ANTON produces the gap analysis that evaluates whether that policy document is still adequate. In fact, an organisation could use both: Glean to search historical documents, and ANTON to generate new professional output with accumulated APCI context. The knowledge flows in different directions — Glean retrieves existing artefacts, ANTON creates and structures new ones.

"Why not use Claude Projects or custom GPTs?" Both allow persistent instructions and uploaded context. Claude Projects lets you upload documents and maintain project-level instructions. Custom GPTs let you define custom behaviour. Neither learns from session output, extracts structured knowledge atoms, detects cross-session patterns, tracks entity relationships, measures output quality over time, or earns autonomous execution trust. They are excellent for static context — when you know in advance what the AI should reference. APCI is for dynamic context — when the relevant knowledge emerges from the work itself and compounds session by session.

The deeper differentiation is architectural. Every platform listed above treats each session as independent. None of them build structured professional context that compounds over time. None of them track entity relationships across engagements. None of them detect cross-workflow patterns. None of them have a quality ratchet that measures whether output is improving or declining. None of them earn autonomy through demonstrated competence.

APCI is not a feature. It is an architectural commitment to the idea that professional AI must learn, remember, and improve — visibly, verifiably, and under the professional's control.

The Open Source Argument

ANTON is MIT-licensed. The entire codebase — including all 84 system prompts, the APCI architecture, the Orchestrator, the School Mode, and the .anton bundle format — is open source. This is a deliberate strategic choice, not an altruistic gesture.

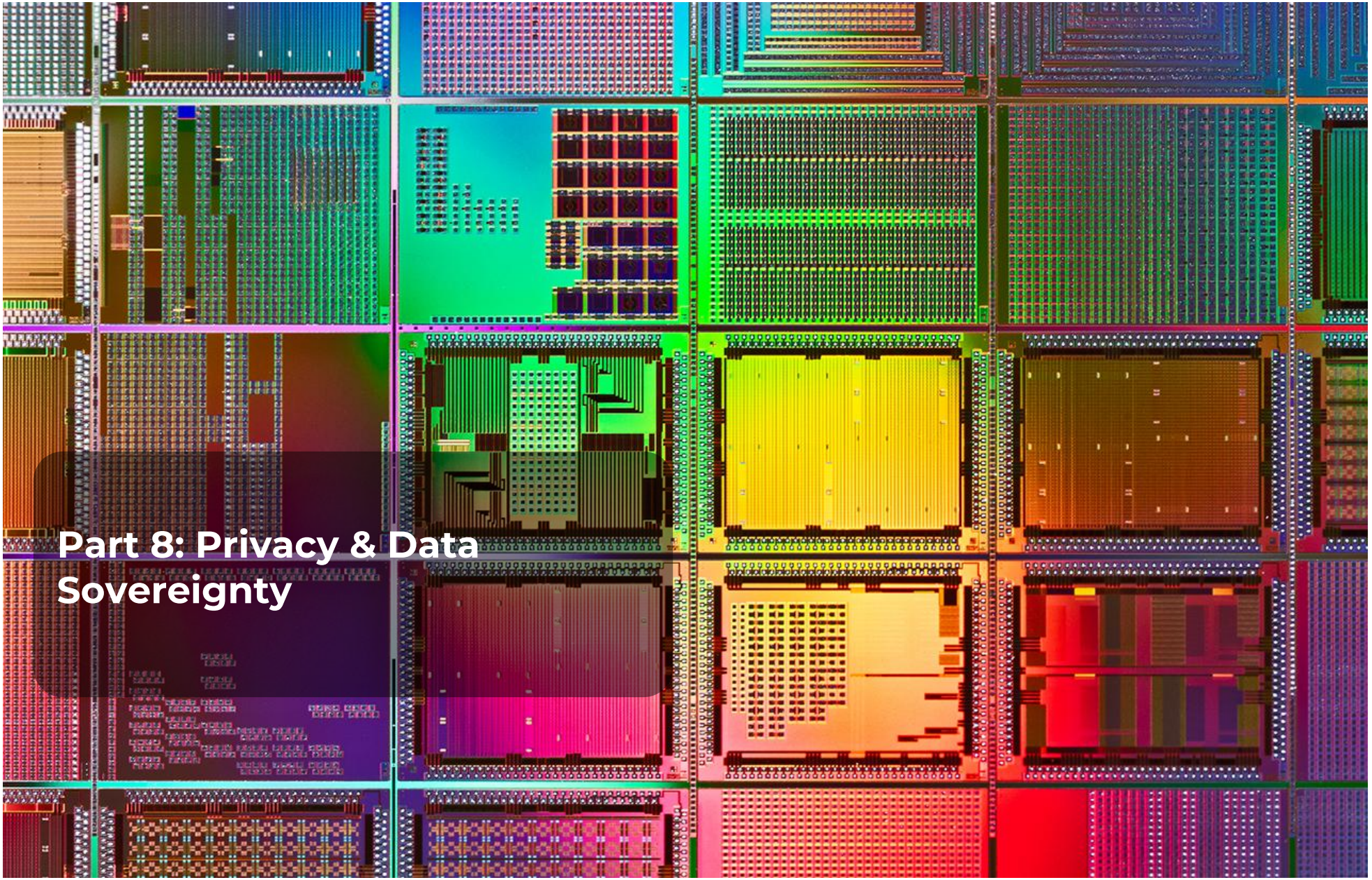
The argument is straightforward: APCI infrastructure should be auditable. When a regulator asks "how does your AI system decide what knowledge to inject into a compliance analysis?", the answer should be available as source code, not as a vendor's marketing claim. The atom boost formulas, the feedback loop mechanics, the earned autonomy thresholds — all of this is inspectable. An organisation's technology team can read the code that governs their AI system's behaviour. A security auditor can verify the safety limits. A data protection officer can trace exactly which data flows through which components.

The open-source model also enables something that closed platforms cannot: organisational customisation at the architectural level. A law firm can modify the legal research system prompt to match their specific citation standards. A Nordic regulator can adjust the gap analysis framework to reflect their supervisory expectations. A school district can adapt the tier system to their curriculum structure. These modifications don't require vendor approval, API requests, or enterprise licensing negotiations. The code is there. Change it.

The competitive moat, as we've argued throughout this document, is not the code — it's the accumulated APCI context. Open-sourcing the infrastructure doesn't give away the value. It enables more organisations to build that value, and it ensures that the infrastructure through which they build it is transparent and trustworthy.

There is also a practical governance argument. Closed-source AI infrastructure creates audit opacity — regulators cannot inspect how knowledge is stored, ranked, or applied. Open-source APCI infrastructure is auditable by definition. Any regulatory examiner, security auditor, or data protection officer can trace the exact path from knowledge extraction to prompt injection. In an industry where "explain how your AI works" is an increasingly common regulatory expectation, transparent infrastructure is not just virtuous — it is practically necessary.

Contributions are welcome. The repository includes a CONTRIBUTING.md with guidelines for code style, testing requirements, and pull request process. System prompt contributions — adding or improving domain expertise for specific regulatory frameworks or professional areas — are particularly valuable, because they improve the first-session experience for every user in that domain. A Docker-based development environment is planned to reduce setup friction for contributors who are not on the project's primary pnpm/Node.js stack.



Part 8: Privacy & Data Sovereignty

30. The Local-First Promise

ANTON makes a claim that most AI platforms cannot: **your data never leaves your machine unless you explicitly choose to send it to an LLM provider.**

This is not a policy. It is an architectural fact. ANTON runs on `localhost`. The database is a local file you control — SQLite for development, PostgreSQL for production — both on your own infrastructure. Documents you upload stay in a local directory. The web interface is served from your own machine to your own browser. There is no ANTON cloud service, no centralised telemetry, no usage analytics endpoint, no phone-home mechanism.

The only network traffic ANTON generates is the API calls you initiate to your chosen LLM provider — and you control which provider that is, which model you use, and what data gets included in each request.

For organisations handling regulated data — financial compliance, legal research, healthcare records, student information — this distinction matters enormously. When your regulator asks "where does the data go?", the answer is unambiguous: it stays on your machine, in a file you control, behind a filesystem you manage.

Compare this to any cloud-hosted AI platform. Even those with strong privacy commitments operate fundamentally differently: your prompts travel to their servers, are processed on their hardware, and are subject to their retention policies and their jurisdiction's legal requirements. A Swedish law firm's client data processed through a US-hosted AI platform is potentially subject to the CLOUD Act. The same data processed through ANTON never leaves Sweden — because it never leaves the lawyer's laptop.

The local-first architecture also eliminates a category of risk that enterprises increasingly worry about: vendor data incidents. If a cloud AI provider experiences a breach, every organisation that used their API has exposure. If ANTON's local database is compromised, the exposure is limited to one machine — and that machne was already within the organisation's security perimeter.

31. The Embedding Layer: Why Ollama Matters

APCI's knowledge memory system requires an embedding model to convert text into numerical vectors for semantic search. This is the component that processes every knowledge atom, every checkpoint decision, and every module description to make hybrid retrieval work.

ANTON uses **Ollama** for this — specifically, the `nomic-embed-text` model running locally. This choice is deliberate and privacy-critical.

What happens when ANTON embeds a knowledge atom:

25. ANTON sends the atom's text content to `localhost:11434` — the Ollama process running on your machine
26. Ollama's `nomic-embed-text` model converts the text into a 768-dimensional numerical vector
27. The vector is returned to ANTON and stored in the local SQLite database
28. The original text and the vector never leave your machine

No API key is required. No external service is contacted. No usage is logged remotely. The embedding model runs entirely within the Ollama process on your hardware.

This matters because the embedding layer sees **everything** APCI knows. Every knowledge atom your team has created. Every decision recorded at every checkpoint. Every module description. If this layer were cloud-hosted, your entire professional knowledge base would transit through a third-party service. With Ollama, it stays local.

Ollama's own privacy posture confirms this. From the official FAQ: *"If you're running a model locally, your prompts and responses will always stay on your machine."* Ollama maintainer Bruce MacDonald confirmed in GitHub Issue #2567: *"Ollama does not track any of your data or input."*

The only outbound network calls Ollama makes are:

29. **Model downloads** — when you first run `ollama pull nomic-embed-text`, weights are downloaded from the Ollama registry. Only the model name is transmitted. After this initial download, no internet connection is required — Ollama operates fully offline.
30. **Auto-update checks** — the desktop tray application (not the CLI binary) periodically checks for new Ollama releases, sending only the operating system type and CPU architecture. No user data is involved.

That's it. No telemetry. No analytics. No data exfiltration pathway.

The practical implication is significant: a law firm can run ANTON with Ollama embeddings and a local Ollama inference model (e.g., Mistral 7B or Llama 3.1), achieving a fully air-gapped AI system where zero bytes of client data leave the machine. The quality of a local 7B-parameter model is lower than Claude Opus — but for organisations where data sovereignty outweighs output quality, the option exists. And the APCI layer compensates: a local model enhanced with 5,000 professionally-rated knowledge atoms produces substantially better output than the same local model in a blank context.

32. Cloud LLM Calls: The Informed Trade-Off

When you use Claude, GPT, Gemini, or Mistral through ANTON, the conversation content — including any injected knowledge atoms — is sent to that provider's API. This is the fundamental trade-off of using cloud LLMs: you get frontier intelligence, but the prompt content traverses the network.

ANTON makes this trade-off as transparent and controlled as possible:

You choose the provider. Every session lets you select which model to use. If a particular engagement involves data too sensitive for any cloud provider, switch to a local Ollama model. The system works identically — the only difference is that inference happens on your hardware instead of in a data centre.

You control what's injected. The Knowledge Memory toggles (injection on/off, collection on/off) let you disable APCI context injection for sensitive sessions. If you're working with material that shouldn't appear in any LLM prompt, turn injection off. The session still works; it just doesn't benefit from accumulated knowledge.

You see what's sent. The transparency level toggle (Off / Summary / Detailed) shows exactly what context was assembled and sent to the model. In Detailed mode, you can inspect every knowledge atom, every prompt layer, every piece of context that was included. Nothing is hidden.

Provider data policies apply. Anthropic, OpenAI, Google, and Mistral each have their own data retention and training policies. ANTON doesn't add any additional data collection on top of what the provider sees. We recommend reviewing your chosen provider's API data policy — most major providers commit to not training on API inputs, but the specifics vary and change over time.

33. Security Hardening for Sensitive Deployments

For organisations handling highly regulated data, ANTON supports several hardening measures:

Disable cloud models in Ollama. Set `OLLAMA_NO_CLOUD=1` in your environment. Since Ollama v0.12 introduced optional cloud-hosted models, this flag ensures all Ollama inference stays strictly on-machine. We include this as a recommended default in `.env.example`.

Keep Ollama on localhost. Ollama binds to `127.0.0.1:11434` by default — only your machine can reach it. Never change the `OLLAMA_HOST` setting to `0.0.0.0` or a network-accessible address. Ollama has no built-in authentication; if exposed to a network, the API is completely open.

Encrypt your filesystem. ANTON's database stores knowledge atoms, embeddings, session history, and entity relationships in plain text. This is appropriate for a local-first application — the security boundary is the filesystem. For deployments handling sensitive data, use full-disk encryption (BitLocker on Windows, FileVault on macOS, LUKS on Linux) to protect the database at rest.

Keep Ollama updated. Security researchers have identified vulnerabilities in Ollama (CVE-2024-28224 DNS rebinding, CVE-2024-37032 path traversal RCE), all patched in versions 0.1.29 and 0.1.34 respectively. These vulnerabilities required network access to Ollama's API — which the default localhost binding prevents — but keeping software updated is basic security hygiene.

Use the standalone CLI binary. The Ollama desktop tray application includes an auto-update checker that sends OS type and architecture to Ollama's servers. The standalone `ollama` CLI binary does not. For air-gapped or maximum-privacy deployments, use the CLI binary.

Restrict filesystem access. ANTON's `ALLOWED_FOLDER_PATHS` environment variable controls which directories the folder-browser API can access. Set this to the minimum necessary paths. The default (`./uploads,./outputs`) is deliberately restrictive.

Audit the prompt assembly. For sensitive deployments, use the transparency toggle's Detailed mode to audit exactly what context the prompt builder injects before sending it to a cloud LLM. This is particularly important during the first weeks of deployment, when the knowledge base is still small and each injected atom is individually significant. Establishing a culture of occasional prompt auditing — checking what the AI actually received — is both a security practice and a professional discipline.

Audit the prompt assembly. Use the Detailed transparency level to periodically review what context is being assembled into prompts. This serves both a privacy function (verifying that no unexpected data is being injected) and a quality function (verifying that the injected knowledge is relevant and current). For regulated deployments, quarterly transparency audits can be incorporated into existing compliance review processes.

Separate user isolation. In team mode (`DEPLOYMENT_MODE=team`), each user's sessions, knowledge atoms, and Orchestrator state are isolated. Analyst A's ratings do not affect Analyst B's retrieval unless both have rated the same atoms — a deliberate design that prevents one user's feedback from silently altering another user's experience.

34. Privacy by Architecture, Not by Policy

Most AI platforms promise privacy through policy — terms of service, data processing agreements, compliance certifications. These are important and necessary. But they are promises, not guarantees. Policies can change. Breaches happen. Acquisitions transfer data to new owners.

ANTON's privacy guarantee is **architectural**. The data doesn't leave because there is no code path that sends it. The embeddings stay local because the embedding model runs locally. The knowledge base remains on your machine because the database is a file on your filesystem. This is not a promise we make — it is a consequence of how the system is built.

For the cloud LLM calls that do cross the network, the trade-off is explicit, visible, and under your control. You choose when to make them, which provider to use, and what context to include. You can inspect exactly what was sent. And for sessions where even that trade-off is unacceptable, you can switch to a local model with no loss of functionality.

We believe this combination — architectural privacy for the knowledge layer, informed choice for the inference layer — is the right model for professional AI. It gives organisations the frontier intelligence they need while keeping their accumulated professional knowledge where it belongs: under their own control, on their own hardware, governed by their own policies.

No SOC 2 certificate replaces the guarantee that the data simply isn't there to breach.

35. GDPR Controller/Processor Analysis

Under GDPR, the question of who is the data controller and who is the processor matters for compliance obligations.

ANTON (the software) is neither controller nor processor. It is a tool installed on your hardware, like a word processor. The organisation deploying ANTON is the data controller for any personal data processed through it.

Ollama (local embeddings) is a locally-running process. No data leaves the machine. No third party is involved. There is no processor relationship — Ollama on your hardware is part of your own processing activity.

Cloud LLM providers (Claude, GPT, etc.) are data processors when called via API. Organisations should ensure that a Data Processing Agreement (DPA) is in place with their chosen provider. Anthropic, OpenAI, and Google all offer standard DPAs for API customers. The key terms to verify: data retention period, prohibition on training on API inputs, sub-processor list, and data deletion mechanisms.

Data subject access requests (DSARs). If knowledge atoms contain personal data — a client name, an employee's risk classification, a student's learning profile — the organisation must be able to fulfil DSAR requests against the APCI knowledge base. ANTON's knowledge graph is searchable by entity: a search for "Person: Jan Eriksson" returns every atom and entity relationship that references that individual. Atoms can be soft-deleted (setting `is_active = 0`) or hard-deleted, and the entity graph supports targeted node removal. This means that a DSAR response can identify, export, and if necessary delete all knowledge related to a specific data subject — fulfilling GDPR Articles 15 (right of access), 17 (right to erasure), and 20 (right to portability).

36. DORA Operational Resilience

For financial institutions subject to the Digital Operational Resilience Act (DORA), ANTON's local-first architecture offers specific advantages:

ICT concentration risk — DORA Article 29 requires financial entities to manage ICT concentration risk. A local-first platform that can switch between multiple LLM providers — or operate entirely on local models — reduces single-provider dependency compared to cloud-only AI tools.

Business continuity — ANTON with Ollama can operate completely offline. If a cloud LLM provider experiences an outage, the platform continues functioning on local models. Professional work is not interrupted by external service availability.

Audit trail preservation — DORA requires comprehensive ICT incident reporting. ANTON's reasoning trails, quality scores, and decision logs provide exactly the kind of auditable record that DORA's operational resilience testing expects.

Third-party risk management — DORA Article 28 requires due diligence on ICT service providers. When ANTON uses cloud LLM providers, those providers are ICT service providers under DORA. The model-agnostic architecture means the institution can switch providers if a particular provider's risk profile changes — a capability that platform-locked AI tools cannot offer. The ability to fall back to local Ollama models means the institution is never fully dependent on any external ICT service for its AI-assisted compliance work.

37. Deployment Checklist for Regulated Environments

For compliance officers deploying ANTON in a regulated environment:

31. **Execute DPAs** with any cloud LLM providers you plan to use (Anthropic, OpenAI, Azure OpenAI, Google, Mistral)
32. **Enable full-disk encryption** on the host machine (the database stores knowledge in plaintext)
33. **Set `OLLAMA_NO_CLOUD=1`** to prevent Ollama from accessing cloud-hosted models
34. **Keep Ollama on localhost** — never expose port 11434 to a network
35. **Configure `ALLOWED_FOLDER_PATHS`** to restrict filesystem access to necessary directories
36. **Use injection/collection toggles** to exclude personal data from the APCI knowledge base for sensitive sessions
37. **Review reasoning trails periodically** to verify that the Orchestrator's autonomous actions align with your governance framework
38. **Restrict network access** on the host machine — ANTON only needs outbound HTTPS to LLM API endpoints (api.anthropic.com, api.openai.com, etc.) and localhost:11434 for Ollama; block all other outbound traffic for maximum isolation
39. **Implement host-level access controls** — ANTON stores professional knowledge in its local database; the host machine should have user authentication, screen lock policies, and ideally encrypted user profiles
40. **Document your AI governance framework** — map ANTON's earned autonomy phases (Section 15) to your organisation's risk management framework, defining which phase is appropriate for which activity type and who authorises promotions
41. **Define a data retention policy for knowledge atoms** — atoms have `valid_from` and `valid_until` fields; establish a review cycle (e.g., annually) to assess whether aged atoms remain accurate and whether their legal basis for processing still applies. Atoms older than the retention period should be reviewed and either confirmed, updated, or deleted
42. **Establish a DSAR response procedure** — document how your organisation will search, export, and if necessary delete knowledge atoms and entity graph entries relating to a specific data subject (see Section 35)

Part 9: What Comes Next

38. Honesty About the Roadmap

We are committed to transparency about what exists and what doesn't. Everything described in Parts 1 through 8 of this document is implemented and working in ANTON v0.7.5. The following items are designed or planned but not yet built:

Voice-first interface for young learners (T1) — audio input and output for ages 6–12, with storytelling-based learning narratives. This matters because T1 learners may not be fluent typists. A voice interface would make ANTON accessible to children who can speak their questions but can't easily type them. Designed, not implemented.

SMS and WhatsApp gateway — text-only interface for feature phones in low-connectivity settings. This matters because the humanitarian deployment vision (Section 22) currently requires a web browser. An SMS gateway would extend access to the billions of people whose primary computing device is a feature phone. Designed, not implemented.

EU Sovereign Edition — a Mistral-primary deployment configuration for EU data residency requirements. This matters because some European organisations require all AI inference to occur within EU borders. Mistral, headquartered in Paris, offers EU-hosted inference. The multi-LLM adapter supports Mistral today; the sovereign deployment configuration — with appropriate defaults, documentation, and compliance attestations — does not yet exist.

.anton Marketplace discovery — the export/import infrastructure works today. Search, ratings, reviews, and community discovery are planned. This matters because the .anton bundle format's value scales with the size of the community using it. A single organisation creating bundles gets utility. A thousand organisations sharing bundles gets network effects. The discovery layer is what transforms the format from a file standard into an ecosystem.

PostgreSQL option — *Completed in v0.7.5.* ANTON now has full dual-database support. PostgreSQL is the primary database for production use with a complete adapter at `server/db/adapters/postgresql-adapter.ts`, 183 CREATE TABLE statements, and 64 PostgreSQL-specific migrations. Auto-detection: if `DATABASE_URL` starts with `postgresql://`, PostgreSQL is used; otherwise SQLite. PostgreSQL 16+ is required for production deployments.

Anthropic Economic Index integration — mapping the Anthropic Economic Index's occupation-level AI impact data to ANTON's module library. This would allow organisations to assess which of their professional functions are most affected by AI advancement and which ANTON modules address those functions. Concept stage — the Economic Index exists but the integration layer does not.

Desktop code signing and auto-update — ANTON's optional Electron desktop app currently requires manual installation. Code signing (for Windows SmartScreen and macOS Gatekeeper trust) and auto-update via GitHub Releases would make desktop distribution viable for non-technical users. The Electron shell exists and works; the distribution pipeline does not.

Team mode with role-based access — the current `DEPLOYMENT_MODE=team` enables JWT authentication, but fine-grained role-based access control (admin, analyst, reviewer, read-only) is not yet implemented. For team deployments, this means defining who can promote the Orchestrator, who can activate knowledge packs, who can delete atoms from the knowledge base, and who can view audit trails. The database columns and JWT infrastructure exist; the permission matrix and UI enforcement do not.

Multi-tenant knowledge isolation — in team deployments, some organisations need knowledge boundaries between business units or client teams. An audit team's knowledge atoms should not appear in a consulting team's retrievals, even though both teams share the same ANTON installation. For professional services firms, this extends to engagement-level boundaries: atoms from Client Nordea's engagement must be retrievable by the Nordea engagement team but not by the SEB engagement team, even though both work in the same FCP area. The architectural foundation exists (area and module scoping in the boost system, the injection toggle for session-level control), but explicit tenant-level and engagement-level isolation with configurable boundaries is not yet built.

AMLA supervisory integration — the EU Anti-Money Laundering Authority (AMLA), operational from mid-2025, will introduce a new layer of direct EU-level supervision for selected obliged entities. ANTON's regulatory radar and gap analysis modules currently reference AMLR and AMLD6, but AMLA's supervisory expectations — including its forthcoming Regulatory Technical Standards and supervisory methodology — will need dedicated integration as they are published. The framework architecture supports adding AMLA as a regulatory source; the specific content and supervisory mapping do not yet exist.

We would rather be honest about what's planned than imply that everything is finished. Professional trust is built on accuracy, not aspiration.

Part 10: Pathfinder — AI-Powered Search

40. Executive Summary

Pathfinder is ANTON's built-in search engine — a progressive reasoning search system that transforms how professionals find, validate, and use information. Unlike conventional search engines that return ranked links, or AI chatbots that answer without showing their work, Pathfinder searches the live web, cross-references results against your local knowledge, and synthesises findings through multi-phase reasoning with full transparency.

Every search in Pathfinder shows you exactly which models participated, how sources were ranked, what the reasoning process looked like, and what it cost. There are no ads, no tracking, no sponsored results, and no data leaves your machine except the API calls to Claude.

Pathfinder is designed for domain professionals aged 35-65 — compliance officers researching regulatory changes, lawyers verifying case law, analysts validating market data, consultants preparing engagement materials — who need search results they can trust and cite.

41. Architecture Overview

2.1 Claude-Only Model Stack

Pathfinder uses an exclusively Anthropic Claude architecture. All search, analysis, and synthesis runs through Claude models. This design ensures consistent quality, predictable costs, and a single API key requirement.

Layer	Model	Purpose
Web Search	Claude Haiku 4.5	Fast web retrieval with `web_search_20250305` tool
Analysis	Claude Haiku 4.5	Investigation-level analysis of search results
Chairman Synthesis	Claude Sonnet 4.6	Validates, cross-references, and produces final answer
Multi-Phase Reasoning	Claude Sonnet 4.6	IRE-style iterative reasoning with confidence gating

Why Claude-only? Web search and extended thinking are mutually exclusive in the Claude API. Pathfinder's architecture exploits this constraint by design: the search step uses Haiku with the `web_search_20250305` tool (no thinking), and the synthesis step uses thinking (no web search). Each model operates in its optimal mode.

2.2 Three Depth Modes

Pathfinder offers three progressively deeper search modes, each adding more reasoning layers:

Quick Mode

Pipeline: Haiku web search --> Haiku think_hard synthesis **Time:** 5-10 seconds | **Cost:** ~$0.001

Haiku searches the live web, retrieves sources, then synthesises the results with think_hard-level reasoning (budget_tokens: 10,000). Ideal for straightforward factual queries, quick lookups, and everyday research.

Thorough Mode

Pipeline: Haiku web search --> Haiku investigation analysis --> Sonnet chairman synthesis **Time:** 15-30 seconds | **Cost:** ~$0.01-0.03

After Haiku searches the web, a second Haiku call performs investigation-level analysis of the results — identifying key findings, assessing source quality, and noting contradictions. Sonnet then acts as chairman, validating the analysis, cross-referencing sources, and producing a comprehensive synthesis with reasoning transparency.

This two-step architecture mirrors how a senior analyst works: gather information, analyse it independently, then have a more experienced colleague validate and synthesise.

Deep Mode (Iterative Reasoning Engine)

Pipeline: Haiku web search --> Sonnet analyse --> Sonnet reflect (confidence scoring) --> Sonnet deepen (conditional) --> Sonnet synthesise **Time:** 30-90 seconds | **Cost:** ~$0.05-0.15

Deep mode implements a multi-phase Iterative Reasoning Engine (IRE) with confidence gating:

43. **Analyse Phase** — Sonnet performs a structured, multi-angle analysis of the search results. Identifies core findings, sub-problems, evidence quality, gaps, and risk factors.
44. **Reflect Phase** — Sonnet critically evaluates its own analysis. Challenges assumptions, identifies logical gaps, flags contradictions. Uses a structured `assess_confidence` tool to record a confidence score (0.0-1.0) and identify specific gaps.
45. **Deepen Phase** (conditional) — Only executes if the reflection confidence is below 0.8. Resolves the specific uncertainties and gaps identified during reflection. This phase is the confidence gate — if Sonnet is already confident, it skips directly to synthesis.
46. **Synthesise Phase** — Sonnet produces the final answer with full context from all prior phases. Streams live to the user interface.

The confidence gating mechanism ensures Deep mode adapts its cost to the query complexity. Simple queries that happen to be run in Deep mode will skip the deepen phase (confidence >= 0.8 after reflection), while genuinely complex queries get the additional reasoning layer.

2.3 System Flow Diagram

```
User Query
    |
    v
[Query Enrichment] -- adds domain terms based on active area
    |
    v
```

```
[Pre-Search Reasoning] -- explains search strategy to user (no API call)
    |
    v
[Haiku Web Search] --+-- [Local Knowledge Search]
  (web_search tool)     |     (BM25 + vector hybrid)
    |                   |
    v                   v
[Search Results + Web Sources] + [Local Knowledge Results]
    |
    v
[Depth-Specific Reasoning Pipeline]
    |
    +-- Quick:    Haiku think_hard synthesis
    +-- Thorough: Haiku investigation --> Sonnet chairman
    +-- Deep:     Sonnet analyse --> reflect --> [deepen] --> synthesise
    |
    v
[Streaming Synthesis to Client]
    |
    v
[Source Quality Scoring + Deduplication + Ranking]
    |
    v
[Persist to Database]
    |
    v
[User sees: Answer + Sources + Reasoning + Cost]
```

42. Seven Search Modes

Every Pathfinder search has both a depth (how deeply to reason) and a mode (what kind of answer the user needs). The seven modes shape how results are ranked, what metadata is extracted, and how the synthesis is structured.

Mode	Icon	Ranking Strategy	Extracted Metadata
Knowledge	BookOpen	Source authority first, then credibility	Confidence level, source type classification
Shopping	ShoppingBag	Price comparison, no sponsored results	Prices (local currency), availability, retailer comparison
Travel	Plane	Transport options by cost and time	Booking links, visa requirements, seasonal relevance
Food	UtensilsCrossed	Recipe reliability / restaurant rating	Cook time, difficulty, servings, dietary info, ingredients
Fix	Wrench	Official documentation preferred	Step-by-step guides, difficulty level, time estimate
News	Newspaper	Recency first, then credibility	Source classification (News/Opinion/Editorial/Analysis/Press Release), political positioning
Local	Navigation	Proximity first	Opening hours, ratings, address, phone, price range

Each mode injects specific instructions into both the search dispatch prompt and the synthesis prompt, ensuring the model's behaviour adapts to the user's intent.

Mode-specific examples:

- **Shopping** in Stockholm: "Compare prices across Swedish and international retailers. Include stores near Stockholm alongside online options. Flag suspiciously low prices."
- **News** for regulatory queries: "Classify each source as News Report, Opinion, or Analysis. Note the outlet's known political positioning. Flag single-source stories."
- **Local** for restaurants: "Rank by proximity to user. Extract opening hours, rating, address, phone. Flag if currently open/closed."

43. Knowledge Source Integration

Pathfinder doesn't just search the web. It simultaneously queries four knowledge layers:

4.1 Live Web Search

Claude's `web_search_20250305` tool retrieves current information from the public internet. Sources are scored for authority (government, academic, regulatory bodies score highest) and tagged with quality indicators.

4.2 Local Knowledge (BM25 + Vector Hybrid Search)

Every Pathfinder search also queries ANTON's local knowledge base — knowledge atoms, session outputs, checkpoint documents, and indexed local folders. The hybrid search combines:

- **BM25** (keyword matching) for precision
- **Vector embeddings** (nomic-embed-text via Ollama) for semantic similarity
- **Reciprocal Rank Fusion** to combine both rankings

Local results use a minimum similarity threshold of 0.5 and a keyword overlap filter to prevent false positives. Results are tagged as `sourceType: 'local'` so users can distinguish them from web results.

4.3 Knowledge Packs

Regulatory knowledge packs installed in ANTON (e.g., AMLR 2024, DORA, ISO 27001) are surfaced as `sourceType: 'knowledge_pack'` when relevant to the query.

4.4 Institutional Memory

Past decisions, session outputs, and documented institutional knowledge are surfaced as `sourceType: 'institutional_memory'`.

All four source types appear in a unified source list with filter tabs (All | Web | Local | Knowledge | Memory), allowing users to see exactly where each piece of information came from.

44. Source Quality Framework

Every source in Pathfinder receives three quality scores, each rated 0.0 to 1.0:

5.1 Authority Score (Domain-Based)

Score	Domain Category	Examples
0.9	Government, regulatory, academic	.gov, europa.eu, ecb.europa.eu, fatf-gafi.org, .edu, arxiv.org, nature.com
0.8	Major news, financial	reuters.com, bloomberg.com, ft.com
0.6	Default (most websites)	General commercial, professional sites
0.3	User-generated, social	reddit.com, quora.com, medium.com, twitter.com

5.2 Relevance Score

Assigned by the synthesis model based on how directly the source addresses the user's query. Considers domain match, content specificity, and recency.

5.3 Consensus Score

Calculated as the number of search models that independently found this URL divided by the total number of successful models. Higher consensus indicates the source is widely recognised as relevant. In the current Claude-only architecture, this represents whether the source appeared in the primary search results.

5.4 Visual Indicators

Sources are displayed with colour-coded dots:

- **Green** (>0.7): High quality / relevance / consensus
- **Amber** (0.4-0.7): Moderate
- **Red** (<0.4): Low — flagged for user awareness

45. Two-Step Reasoning Transparency

Every Pathfinder search provides reasoning transparency at two points:

6.1 Pre-Search Reasoning ("Search Strategy")

Before any API call is made, Pathfinder displays a search strategy panel explaining:

- How the query was analysed
- Whether domain enrichment was applied (and what terms were added)
- Which search mode is active and what it means
- The depth approach being used
- Context awareness (active area, user location)

This is generated locally with no API call — pure deterministic logic based on the search configuration.

6.2 Post-Search Reasoning ("Why These Results")

The synthesis model is instructed to include a "Why These Results" section explaining its ranking rationale. This section is automatically extracted from the synthesis text and displayed in a separate panel with a gold/amber border, making the reasoning visible without cluttering the main answer.

Example output:

> **Why These Results**: I ranked the European Commission's official AMLR text highest because it is the primary source. The EBA's technical standards draft was included because it provides implementation detail. The law firm commentary was included for practical interpretation but ranked below official sources due to potential commercial bias.

46. "Improve My Search" Wizard

Pathfinder includes a built-in search refinement wizard that helps users construct better queries before searching. The wizard runs a 3-step flow:

Step 1 — Analyse: Haiku analyses the user's query and generates 3-4 clarifying questions about scope, context, preferred answer type, and constraints.

Step 2 — Answer: The user answers the clarifying questions in a text area.

Step 3 — Build: Haiku takes the original query plus the user's answers and generates:

- An improved, more specific search query
- A recommended search mode (if the current mode isn't optimal)
- A recommended depth (if the query warrants deeper reasoning)
- An explanation of why these changes were suggested

The user can then apply the improved query, the mode/depth recommendations, or both.

47. Domain-Aware Query Enrichment

Pathfinder enriches queries based on the user's active ANTON area. This adds domain-specific terminology that improves search precision without requiring the user to know the exact regulatory or professional vocabulary.

Active Area	Terms Added (up to 3)
FCP (Financial Crime Prevention)	AML, CFT, AMLR, AMLA, financial crime prevention, compliance
Legal	legal analysis, case law, jurisdiction, regulatory
Healthcare	clinical, medical, patient safety, health regulation
Finance	financial regulation, MiFID, DORA, prudential
PE/VC	private equity, venture capital, deal flow, portfolio
Blockchain	MiCA, CASP, crypto-asset, DeFi, distributed ledger
Data Protection	GDPR, privacy, data processing, DPA
Sanctions	sanctions screening, OFAC, EU sanctions, SDN
Tax	tax compliance, transfer pricing, tax regulation
Education	curriculum, pedagogy, educational
NGO	humanitarian, development, social impact, donor
Creative Production	content creation, editorial, publishing

Enrichment is additive — terms already present in the query are not duplicated. The enriched query is shown to the user in the search strategy panel, and stored in the database for audit purposes.

48. Location-Aware Search

Users can configure their location (city and country) in ANTON's settings. When set, Pathfinder injects this context into both the search dispatch and synthesis prompts:

- **Shopping mode:** Includes local stores, uses local currency (SEK for Stockholm, GBP for London)
- **Local mode:** Ranks by proximity, flags currently open/closed businesses
- **Travel mode:** Routes calculated from user's city
- **Food mode:** Nearby restaurants included alongside online recipes
- **General queries:** Regional relevance applied where applicable

Location data is stored client-side only (localStorage) and is never persisted on the server. It flows through the search request body and is injected into the system prompt for that search only.

49. Document-Driven Search

Pathfinder supports uploading documents (PDF, DOCX, XLSX, CSV, TXT, MD, HTML) that provide additional context for searches. When documents are attached:

47. Text is extracted server-side using ANTON's `text-extractor` service
48. Token count is estimated for budget management
49. Document content is injected into the search context with token budgeting (max 30,000 tokens)
50. Longer documents are truncated with a "(truncated)" marker

Documents can be thread-scoped — all searches within a thread share the same document context. This enables workflows like "upload a regulatory text, then ask multiple questions about it."

Supported formats: PDF, DOCX, DOC, XLSX, XLS, CSV, TXT, MD, HTML (20MB limit)

50. Thread Management

Pathfinder organises searches into threads — persistent collections that group related searches together. Each thread has:

- A user-editable title
- A pinned/unpinned state (pinned threads appear first)
- A search count badge
- Optional attached documents (thread-scoped)
- Timestamps for creation and last update

Threads enable workflows like "AMLR Implementation Research" containing 15 related searches, all sharing the same document context and building on each other's results.

51. Follow-Up Questions

After any search completes, users can ask follow-up questions that maintain the context of the original search. The follow-up system:

51. Loads the original query and synthesis from the database
52. Builds a context message combining the original query, previous synthesis, and new question
53. Streams the response via Sonnet with thinking (budget_tokens: 4,096)
54. Persists the follow-up in the `pathfinder_followups` table

The synthesis model also generates 1-3 suggested follow-up questions at the end of each search, displayed as clickable pills below the results.

52. Pipe to Module

Search results can be piped directly into ANTON's modules for further processing. The "Use in..." dropdown offers five destinations:

Destination	Use Case
Open Chat	Continue the conversation with full context
Brief Me	Generate an executive briefing from search findings
Challenge This	Devil's advocate analysis of search conclusions
Review Engine	Formal review of the search synthesis
Sounding Board	Explore implications and alternatives

The pipe mechanism formats the synthesis and top 10 sources as markdown, stores it in sessionStorage, and navigates to the target page which reads and injects the context.

53. Proactive Suggestions

When Pathfinder is idle, it displays "You might want to know" suggestions — AI-generated search queries based on:

- The user's 5 most recent Pathfinder searches
- The user's 10 most recent ANTON work sessions (title + module)

Suggestions are generated by Haiku, cached in the database with a 6-hour expiry, and can be dismissed or refreshed. Each suggestion includes a brief context explanation of why it was suggested.

On the homepage, suggestions appear in compact mode (2 items). On the full Pathfinder tab, they appear in full mode with dismiss/refresh controls.

54. Homepage Integration

Pathfinder is embedded on the ANTON homepage as a compact search bar widget (`PathfinderBar`). The homepage widget:

- Runs in Quick mode only (for speed)
- Shows compact results (line-clamped to 6 lines of synthesis)
- Displays source count badges (e.g., "7 sources") with top 3 domain names
- Shows recent search history on input focus
- Includes the search mode selector in compact (icon-only) mode
- Features a manifesto line: "Your search. Your data. No ads. No agenda."

The "Open in Pathfinder — go deeper" button transfers state via the `?searchId=` URL parameter. The full Pathfinder page loads the existing result from the database without re-running the search, allowing the user to switch to Thorough or Deep mode, ask follow-ups, or explore individual sources.

55. Cost Transparency

Every Pathfinder search displays full cost information:

Metric	Displayed
Duration	Total wall-clock time in seconds

Input tokens	Total tokens sent to all models
Output tokens	Total tokens received from all models
Cost	Total USD cost across all API calls

For Thorough and Deep modes, a per-model cost breakdown is also shown, listing each step in the pipeline with its individual token counts and duration.

Estimated Cost Per Search

Mode	Typical Input Tokens	Typical Output Tokens	Estimated Cost
Quick	~1,500-3,000	~500-1,500	$0.001-0.003
Thorough	~8,000-15,000	~2,000-5,000	$0.01-0.03
Deep	~20,000-50,000	~5,000-15,000	$0.05-0.15

Daily professional usage estimate: $0.15-0.30 for a typical mix of 10-20 Quick, 3-5 Thorough, and 1-2 Deep searches.

56. Database Schema

Pathfinder uses 6 SQLite tables, created via migrations 046 and 047:

pathfinder_searches

The core table. Stores every search with its full context: query (original and enriched), depth mode, search mode, synthesis text, thinking text, model results (JSON), web sources (JSON), token counts, cost, duration, and status. Indexed by user + created_at for history queries.

pathfinder_sources

Individual source records with quality metrics. Each source has a URL, title, snippet, source type (web/local/knowledge_pack/institutional_memory), the model that found it, and three quality scores (relevance, quality, consensus). Final rank position is stored after synthesis.

pathfinder_threads

Thread metadata: title, pinned state, associated document IDs (JSON). Indexed by user + updated_at for recency-based listing.

pathfinder_documents

Uploaded document metadata and extracted text. Stores filename, file path, file size, MIME type, extracted text, word count, and token estimate. Thread-scoped via optional `thread_id` foreign key.

pathfinder_followups

Follow-up Q&A records linked to a parent search. Stores the question, answer text, thinking text, and token counts.

pathfinder_suggestions

Proactive suggestion cache. Stores generated queries with context, expiry timestamps, and dismissed state. Cleaned up on refresh.

57. Security and Privacy

Data Stays Local

- All search results, sources, threads, documents, and suggestions are stored in the local SQLite database
- Document uploads are stored on the local filesystem
- Location settings are stored in browser localStorage only
- No telemetry, no analytics, no third-party tracking

What Leaves the Network

- Search queries are sent to the Anthropic API (Claude) for web search and synthesis
- This is the only network traffic Pathfinder generates
- No data is sent to Google, Bing, or any other search provider

Access Control

- All database queries are parameterised (no SQL injection risk)
- User isolation on all queries (WHERE `user_id = ?`)
- File upload validation: extension whitelist, 20MB size limit, MIME type check
- Error responses use `safeError()` to strip stack traces and sensitive data

58. Technical Implementation

19.1 Server-Side Stack

Component	Technology
Runtime	Node.js 22 + Express 4
Database	SQLite (better-sqlite3)
AI Provider	Anthropic Claude SDK 0.78.0
File Processing	pdf-parse, mammoth (DOCX), xlsx
Streaming	Server-Sent Events (SSE)
Search	BM25 + vector hybrid via Ollama nomic-embed-text

19.2 Client-Side Stack

Component	Technology
Framework	React 18 + TypeScript 5.7 (strict)
Build	Vite 6
Styling	Tailwind CSS 4
State	Component-local (no global store needed)
Markdown	ReactMarkdown + remark-gfm
Streaming	Async generators over ReadableStream

19.3 File Inventory

Server (6 files):

- `server/services/pathfinder-engine.ts` — Core engine: dispatch, synthesis, IRE phases, quality scoring (~1,100 lines)
- `server/routes/pathfinder.ts` — 14 REST/SSE endpoints (~420 lines)
- `server/prompts/pathfinder-search.md` — Search dispatch system prompt
- `server/prompts/pathfinder-synthesis.md` — Synthesis/deliberation system prompt
- `server/db/migrations/046_pathfinder.sql` — 6 tables, indexes
- `server/db/migrations/047_pathfinder_enrichment.sql` — Quality scoring, context columns

Client (14 components + 1 API module):

- `src/pages/PathfinderPage.tsx` — Full search experience (~420 lines)
- `src/components/pathfinder/PathfinderBar.tsx` — Homepage widget (~255 lines)
- `src/components/pathfinder/PathfinderResultPanel.tsx` — Result display (~268 lines)
- `src/components/pathfinder/DepthSelector.tsx` — Depth mode toggle + info panel (~141 lines)
- `src/components/pathfinder/SearchModeSelector.tsx` — 7-mode intent toggle
- `src/components/pathfinder/ImproveSearchPanel.tsx` — Search refinement wizard
- `src/components/pathfinder/ProactiveSuggestions.tsx` — "You might want to know"
- `src/components/pathfinder/WebSourcesList.tsx` — Ranked source list with filter
- `src/components/pathfinder/SourceCard.tsx` — Per-model response card
- `src/components/pathfinder/FollowUpInput.tsx` — Follow-up Q&A input
- `src/components/pathfinder/PipeToModuleButton.tsx` — Route to modules
- `src/components/pathfinder/PathfinderCostDisplay.tsx` — Cost breakdown
- `src/components/pathfinder/PathfinderThreadTabs.tsx` — Thread management
- `src/components/pathfinder/DocumentUploadPanel.tsx` — Document upload
- `src/lib/pathfinder-api.ts` — Client API types and streaming (~263 lines)

19.4 SSE Event Protocol

Pathfinder uses Server-Sent Events for real-time streaming. Events are JSON objects prefixed with `data:` and terminated with `\n\n`. The stream ends with `data: [DONE]\n\n`.

Event	When	Key Fields
`search_start`	Search begins	`searchId`, `depth`
`pre_search_reasoning`	Strategy computed	`reasoning` (text)
`model_start`	A model begins work	`modelId`, `role`
`model_complete`	A model finishes	`modelId`, `role`, `status`, `durationMs`, `confidenceScore`
`synthesis_start`	Synthesis begins	—
`text_delta`	Synthesis token	`content` (text chunk)
`thinking_delta`	Thinking token	`content` (thinking chunk)
`search_complete`	Search finished	Full result: sources, tokens, cost, duration, follow-ups
`error`	Failure	`message`

59. Competitive Positioning

What Pathfinder Is NOT

- Not a Google/Bing competitor for trivial queries ("weather in Stockholm")
- Not a chatbot that happens to search
- Not a tool that optimises for engagement or ad revenue

What Pathfinder IS

- A domain-aware, reasoning-first search engine for professionals
- A transparent system that shows exactly how and why results were ranked
- A local-first tool where your data never leaves your machine (except API calls)
- A search engine that improves with your institutional knowledge

Key Differentiators vs. Perplexity / ChatGPT Search / Google AI Overviews

Dimension	Pathfinder	Perplexity	ChatGPT Search	Google AI
Reasoning depth	3 levels: Quick / Thorough / Deep IRE	Single-pass	Single-pass	Single-pass
Reasoning transparency	Full: pre-search strategy, per-phase reasoning, confidence scores, "Why These Results"	Minimal	None	None
Local knowledge	BM25+vector hybrid over your documents, atoms, sessions	None	None	None
Domain awareness	Enriches queries with 12 domain vocabularies	Generic	Generic	Generic
Cost transparency	Per-search token count and USD cost	Hidden	Hidden	Hidden
Search modes	7 intent-specific modes with tailored ranking	Generic	Generic	Generic
Data ownership	Local-first, SQLite, your machine	Cloud	Cloud	Cloud
Ad-free guarantee	Absolute	Partial (Pro)	Yes	No
Document context	Upload PDFs/DOCX for search-scoped context	Pro feature	Limited	None
Pipe to workflow	Direct integration with 150+ ANTON modules	None	None	None

60. User Experience Walkthrough

Scenario: Compliance Officer Researching AMLR Article 28 CDD Requirements

55. **Homepage:** The officer opens ANTON. Pathfinder's search bar is at the top. A proactive suggestion reads "AMLR Article 28 implementation timeline updates" (based on recent work in the Gap Assessment module).
56. **Search:** They type "CDD requirements under AMLR Article 28 for high-risk customers." Pathfinder enriches the query with "AML, CFT, AMLR" from the FCP domain context.
57. **Mode Selection:** They select Knowledge mode (default) and Thorough depth.
58. **Search Strategy:** The pre-search reasoning panel shows: "Analysing: CDD requirements under AMLR Article 28... Enriched with: AML, CFT, AMLR. Approach: Haiku web search, investigation analysis, then Sonnet chairman validates and synthesises."
59. **Progress:** Model status cards appear:
    - Web Search (Haiku) — running... complete (2.3s)
    - Investigation (Haiku) — running... complete (4.1s)
    - Synthesis starting...
60. **Results:** The synthesis streams in with markdown formatting. Links are clickable. A "Why These Results" section explains the ranking rationale.
61. **Sources:** 12 sources are listed — 9 from the web (europa.eu, eba.europa.eu, two law firm analyses) and 3 from local knowledge (a previously generated AMLR gap assessment and two knowledge pack entries).
62. **Follow-Up:** They click a suggested follow-up: "How does Article 28 interact with the EBA's draft technical standards?" The answer streams with full context from the original search.
63. **Pipe to Module:** They click "Use in..." and select "Gap Assessment" to pipe the research into an AMLR compliance gap assessment, pre-populated with the search findings.
64. **Cost:** The footer shows: 18.2s | 12,340 in / 3,210 out | $0.024.

61. API Reference

Search Endpoint

```
POST /api/pathfinder/search
Content-Type: application/json

{
  "query": "CDD requirements under AMLR Article 28",
  "depth": "thorough",
  "searchMode": "knowledge",
  "threadId": null,
  "documentIds": [],
  "activeAreaId": "fcp",
  "activeModuleId": "gap-analysis",
  "userLocation": "Stockholm, Sweden"
}

Response: SSE stream (text/event-stream)
```

Follow-Up Endpoint

```
POST /api/pathfinder/followup
Content-Type: application/json

{
  "searchId": "uuid",
  "question": "How does Article 28 interact with the EBA's draft technical standards?"
}

Response: SSE stream (text/event-stream)
```

History

```
GET /api/pathfinder/searches?limit=50&offset=0
GET /api/pathfinder/searches/:id
DELETE /api/pathfinder/searches/:id
```

Thread CRUD

```
GET /api/pathfinder/threads
POST /api/pathfinder/threads    { "title": "AMLR Research" }
PATCH /api/pathfinder/threads/:id   { "title": "...", "pinned": true }
DELETE /api/pathfinder/threads/:id
```

Document Upload

```
POST /api/pathfinder/documents   (multipart/form-data, field: "file")
GET /api/pathfinder/documents?threadId=uuid
DELETE /api/pathfinder/documents/:id
```

Pipe to Module

```
POST /api/pathfinder/searches/:id/to-module
{ "moduleId": "gap-analysis", "areaId": "fcp" }
```

Suggestions

```
GET /api/pathfinder/suggestions
POST /api/pathfinder/suggestions/:id/dismiss
POST /api/pathfinder/suggestions/refresh
```

62. Configuration

Pathfinder requires only one environment variable:

```
ANTHROPIC_API_KEY=sk-ant-...    # Required — enables all 3 depth modes
```

Optional configuration:

```
OLLAMA_BASE_URL=http://localhost:11434    # Enables local knowledge vector search
```

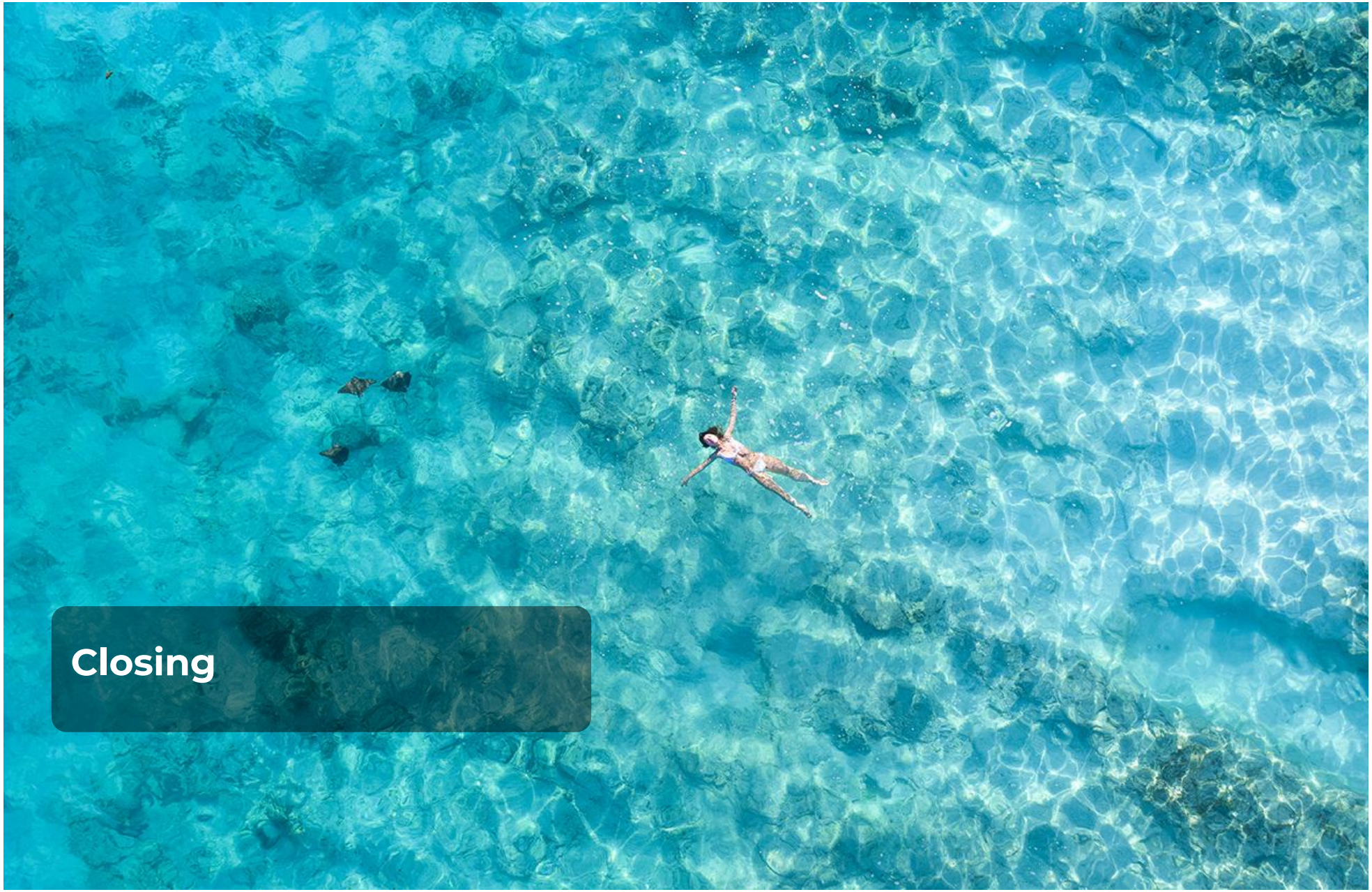
No Google API key. No Mistral API key. No OpenAI key. Pathfinder runs entirely on Claude.

63. Roadmap

Planned enhancements for future releases:

Feature	Description
Pathfinder Alerts	Save searches and get notified when results change
Team Search Sharing	Share searches and threads across team members
Search-to-Workflow	Automated pipelines from search results to module actions
Citation Builder	Generate APA, Harvard, Bluebook, or OSCOLA citations from sources
Pathfinder as MCP Tool	Expose Pathfinder as a Model Context Protocol tool for external agents
Comparative Search	Run the same query across multiple time periods and diff the results
Search Analytics	Dashboard showing search patterns, cost trends, and quality metrics

Pathfinder is part of ANTON by openEXPERT — the AI-powered expert workspace for 55+ professional domains. Built with Claude. Local-first. No ads. No tracking. No agenda.



Closing

39. The Compound Effect

The most important property of APCI is that it compounds.

The first week of using ANTON produces useful output. The prompt layer, the module expertise, and the output format system deliver professional-grade results from the very first session. That alone is valuable.

The first month starts producing better output, because the knowledge base has grown. A hundred atoms have been extracted. The retrieval feedback loop has received its first ratings. The knowledge graph has begun mapping the entities in your work. The system knows a little about your clients, your regulations, your standards.

After six months, the compound effect becomes substantial. The knowledge base holds thousands of atoms. The feedback loop has learned, from hundreds of professional judgements, which knowledge matters in which contexts. Entity relationships span dozens of sessions. Pattern detection has identified trends that no individual analyst noticed. The quality trajectory shows measurable improvement. The Orchestrator, if promoted, has earned Phase 2 or Phase 3 trust through demonstrated competence.

After a year, APCI holds institutional knowledge that no individual team member possesses — not because it's smarter than any of them, but because it remembers across all of them. Every session that every analyst has ever run, every decision that every senior partner has ever made, every pattern that has emerged across hundreds of engagements — it's all there, structured, searchable, and actively applied to make every new session better than it would have been without that context.

After two years, the APCI layer is a genuine strategic asset. It contains knowledge that would otherwise walk out the door when experienced team members move on. It holds institutional judgement that no training programme can replicate. It has detected patterns that no individual analyst could see. And it has done all of this visibly, verifiably, and under the professional's control.

This compound effect is ANTON's genuine strategic contribution. Not a better chatbot. Not a smarter model. A professional context layer that gets more valuable the longer you use it — and that belongs to you, not to a model provider.

Every session makes it better. Every correction sharpens it. Every decision enriches it.

Full Circle

We started this document with a question: what happens after the first session ends?

The answer, we now hope, is clear. After the first session, APCI begins. Knowledge atoms are extracted. Embeddings are created. The knowledge base starts its long accumulation. After the tenth session, retrieval quality improves because feedback ratings have started to differentiate useful knowledge from noise. After the hundredth session, entity relationships reveal structural patterns in your professional domain. After the thousandth session, the quality trajectory, the earned trust progression, and the cross-workflow intelligence represent something that no model upgrade, no prompt engineering, and no competitor subscription can replicate: your organisation's accumulated professional context intelligence.

That is what APCI means in practice. And that is what we are building.

The AI industry's current trajectory — pouring investment into ever-more-intelligent models — is not wrong. It is incomplete. Intelligence is necessary. But intelligence without context is a brilliant stranger who starts over every morning. What professionals need is a trusted colleague who remembers, learns, and improves.

APCI is the architecture for that colleague. ANTON is the open-source implementation. And every session that every professional runs on it makes it a little bit better at the work that matters to them.

Daniel Bardun Founder, FutureChain AB Creator of ANTON by openEXPERT March 2026